

Laboratory Record

submitted as part of

Programming for Data Science Lab

LAB ASSESSMENT-8

by



Reg No: 26MML0031
Name: Hasnain H. Makada

To



Dr M Rajasekhara Babu

School of Computer Science and Engineering



VIT[®]
Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)

JULY 2026

2026 FALL SEMESTER
Programming for Data Science Lab (MACSE502) Record
INDEX

Week No	Date	Sl. No.	Type	Topic	Title with download Link for document	Problem Statement	Page No.
1	09-07-2026	1	PP	Pandas	Basic Data Manipulation with Pandas	Create a DataFrame with columns Name, Age, and City containing data for five individuals. Perform the following operations: select only the Name and Age columns, filter rows where Age is greater than 25, add a new column Country with a default value, and sort the DataFrame by Age in descending order. Finally, calculate the average age of the individuals.	1
1	09-07-2026	2	EP	Pandas	AI Model Performance Analysis Using Pandas	An AI company maintains model performance records containing: • ModelID • Accuracy • Algorithm Perform: 1. Select ModelID and Accuracy. 2. Filter models with Accuracy greater than 90%. 3. Add Status with default value "Production Ready". 4. Sort models by Accuracy. 5. Calculate average model accuracy.	6
2	16-07-2026	3	PP	Pandas	Data Cleaning and Preprocessing with Pandas	Create a DataFrame with some missing values in columns Name, Age, and City. Perform the following operations: fill missing values in the Age column with the mean age, drop rows where Name or City is missing, and convert the Age column to integer type. Finally, normalize the Age column using min-max scaling	14
2	16-07-2026	4	EP	Pandas	Movie Rating Data Preprocessing Using Pandas	A streaming platform maintains movie records containing: • MovieID • Rating • Genre Perform: 1. Create a dataset with missing values. 2. Fill missing Rating values using average rating. 3. Remove rows with missing MovieID or Genre. 4. Convert Rating into integer type. Normalize rating values.	18

3	23-07-2026	5	PP	NumPy	Creating and Manipulating Arrays Using NumPy	Create a 2D array of shape (3, 4) with random integers between 0 and 10. Perform the following operations: calculate the mean and standard deviation of the entire array, slice the array to get the first two rows and last two columns, reshape the array to shape (4, 3), and perform element-wise multiplication with another array of the same shape.	30
3	23-07-2026	6	EP	NumPy	Warehouse Shipment and Logistic Analytics Using Numpy	A logistics company tracks shipments from 4 warehouses across 7 weeks. Create a 4 x 7 shipment matrix. Perform the following operations: (1) compute average shipments, (2) compute standard deviation, (3) slice the first two warehouses and last three weeks, (4) reshape the matrix into a 7 x 4 matrix, and (5) perform element-wise multiplication with a transportation-cost matrix of the same shape. As an extra	35
4	30/7/2026	7	PP	Pandas, Numpy, Matplotlib	Feature Engineering and Exploratory Data Analysis	Create a DataFrame with columns Feature1, Feature2, and Target containing random data. Perform the following operations: create a new feature that is the logarithm of Feature1, bin Feature2 into three categories (low, medium, high), and calculate the correlation matrix of the DataFrame. Finally, create a scatter plot of Feature1 vs. Feature2 colored by Target, and interpret any visible patterns.	46

4	30/7/2026	8	EP	Pandas, NumPy, Matplotlib	Employee Attrition Prediction Using Workforce Indicators	A company wants to analyze workforce behavior and predict employee attrition. Create a DataFrame containing: OvertimeHours JobSatisfactionScore SalaryLevel TrainingHours WorkLifeBalance YearsAtCompany PromotionCount AttritionStatus Perform: 1. Create Log_OvertimeHours. 2. Categorize JobSatisfactionScore into: - o Disengaged - o Satisfied - o Highly Engaged 3. Generate correlation matrix. 4. Create a Treemap Visualization of employee categories. Analyze workforce trends.	54
5	06/08/2026	9	PP	Web Scraping	requests, BeautifulSoup, pandas, json	Scrape data from a webpage and store it in a structured format like CSV or JSON	63
5	06/08/2026	10	EP	Web Scraping	Online Job Market Analytics	A recruitment agency wants to analyze job opportunities posted on a dynamic job portal. Develop a Python application using Selenium to scrape job listings, extract Job Title, Company Name, Location and Salary (if available), store the collected data in a Pandas DataFrame, save the dataset as a CSV file, and visualize job demand by location.	70
6	20/08/2026	11	PP	Plotly	Interactive Bar Chart with Plotly	Create an interactive bar chart showing the population of different cities.	80
6	20/08/2026	12	PP	Plotly	Interactive Scatter Plot with Plotly	Create an interactive scatter plot showing the relationship between house size and price.	88

7	27/08/2026	13	PP	Bokeh	Interactive Scatter Plot with Bokeh	Create an interactive scatter plot showing the relationship between petal length and petal width from the Iris dataset using Bokeh.	101
7	27/08/2026	14	EP	Bokeh	Interactive Line Plot with Bokeh	Create an interactive line plot showing the daily temperatures over a week using Bokeh.	105
7	27/08/2026	15	EP	Bokeh	Interactive Bar Chart with Bokeh	Create an interactive bar chart showing the sales figures of different products using Bokeh.	109
8	03/09/2026	16	PP	scikit-learn and Plotly	Visualizing Clusters with scikit-learn and Plotly	Perform Principal Component Analysis (PCA) on the Iris dataset and visualize the first two principal components using an interactive 2D scatter plot in Plotly.	113
8	03/09/2026	17	EP	scikit-learn and Plotly	Visualizing Clusters with scikit-learn and Plotly	Perform K-means clustering on the Iris dataset and visualize the clusters using an interactive 3D scatter plot in Plotly.	122

PP: Practice Program; EP: Exercise Program; CP: Challenging Program

2026 FALL SEMESTER
Programming for Data Science Lab (MACSE502)
Record Practice Program

Date: 09/07/2026

Week No.: 1

Program S. No. 1

Title	Data Manipulation and Analysis using the Starwars Dataset in R																					
Problem Statement	Basic Data Manipulation with Pandas: Create a DataFrame with columns Name, Age, and City containing data for five individuals. Perform the following operations: select only the Name and Age columns, filter rows where Age is greater than 25, add a new column Country with a default value, and sort the DataFrame by Age in descending order. Finally, calculate the average age of the individuals.																					
Objectives	- To understand how to create a Pandas DataFrame from a Python dictionary. - To learn column selection techniques using Pandas. - To apply filtering conditions on DataFrame rows. - To create and populate new columns in a DataFrame. - To sort data based on column values. - To perform basic statistical analysis using Pandas aggregate functions.																					
Dataset URL Link / File	<table><tr><td>Name</td><td>Age</td><td>City</td></tr><tr><td>Alice</td><td>23</td><td>New York</td></tr><tr><td>Bob</td><td>30</td><td>London</td></tr><tr><td>Charlie</td><td>28</td><td>Paris</td></tr><tr><td>David</td><td>35</td><td>Tokyo</td></tr><tr><td>Emma</td><td>22</td><td>Sydney</td></tr></table>				Name	Age	City	Alice	23	New York	Bob	30	London	Charlie	28	Paris	David	35	Tokyo	Emma	22	Sydney
Name	Age	City																				
Alice	23	New York																				
Bob	30	London																				
Charlie	28	Paris																				
David	35	Tokyo																				
Emma	22	Sydney																				
Functions Details	Sl. No.	Function	Package	Description of function																		
	1	DataFrame()	pandas	Creates a DataFrame																		
	2	sort_values()	pandas	Sorts DataFrame rows																		
	3	mean()	pandas	Calculates the average value																		
	4	sort_values()	pandas	Sorts DataFrame rows																		
Steps/ Process/ Procedure/ Algorithm/ Flowchart/ Model	Step 1 Import the pandas library and create a dictionary containing Name, Age and City Data Step 2 Convert the dictionary into a pandas DataFrame Step 3 Select only the Name and Age columns from the DataFrame. Step 4 Filter the DataFrame to display individuals whose age is greater than 25.																					

Student Reg. No: 26MML0031 Name: Makada Hasnain Husainbhai

Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT), Vellore

2026 FALL SEMESTER
Programming for Data Science Lab (MACSE502)
Record Practice Program

Date: 09/07/2026

Week No.: 1

Program S. No. 1

	Step 5 Add a new column named Country and assign a default value of India. Step 6 Sort the DataFrame in descending order based on the Age column. Step 7 Calculate the average age of all individuals using the mean() function. Step 8 – Display all intermediate and final results.
Program	<pre># Load Necessary Packages import pandas as pd # Step 1: Create Dataset dataframe1 = { "Name": ["Alice", "Bob", "Charlie", "David", "Emma"], "Age": [23, 30, 28, 35, 22], "City": ["New York", "London", "Paris", "Tokyo", "Sydney"] } # Step 2: Create DataFrame df = pd.DataFrame(dataframe1) print("Original DataFrame") print(df) print() # Step 3: Select Name and Age Columns name_age = df[["Name", "Age"]] print("Name and Age") print(name_age) print() # Step 4: Filter Rows where Age > 25 filtered_df = df[df["Age"] > 25] print("Age > 25") print(filtered_df) print()</pre>

Student Reg. No: 26MML0031 Name: Makada Hasnain Husainbhai

Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT), Vellore

2026 FALL SEMESTER
Programming for Data Science Lab (MACSE502)

Record Practice Program

Date: 09/07/2026

Week No.: 1

Program S. No. 1

```
# Step 5: Add Country
Column df["Country"] =
"India"

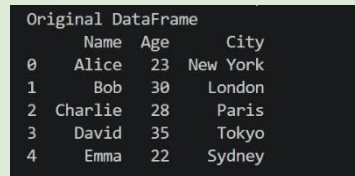
print("Country
Added") print(df)
print()

# Step 6: Sort by Age in Descending Order
sorted_df = df.sort_values(by="Age", ascending=False)

print("Sorted
Data")
print(sorted_df)
print()

# Step 7: Calculate Mean Age
mean_df = df["Age"].mean()

print("Mean Age")
print(mean_df)
```



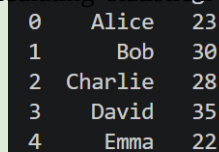
	Name	Age	City
0	Alice	23	New York
1	Bob	30	London
2	Charlie	28	Paris
3	David	35	Tokyo
4	Emma	22	Sydney

Figure 1.1 Original Dataframe

Figure 1.1 displays the original DataFrame created from a Python dictionary containing information about five individuals. The DataFrame consists of three columns: Name, Age, and City, where each row represents a different individual and their corresponding details. The pd.DataFrame() function converts

the dictionary data into a structured tabular format, making it easier to store, view, and manipulate data using Pandas. This DataFrame serves as the starting dataset for all subsequent operations such as Name and Age selection, filtering, adding new columns, sorting, and calculating Name and Age measures.

Figure 1.2 shows the DataFrame after selecting only the Name and Age columns from the original dataset using Pandas column indexing. This operation extracts the required columns and creates a new DataFrame named name_age, removing the City column from the view. Such column selection helps focus on relevant attributes for analysis and reduces unnecessary data processing.



	Name	Age
0	Alice	23
1	Bob	30
2	Charlie	28
3	David	35
4	Emma	22

Figure 1.2 Selected Name and Age Columns

**Output
Snapshots
With
Explanation/
Observations**

Student Reg. No: 26MML0031 Name: Makada Hasnain Husainbhai

Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT), Vellore

2026 FALL SEMESTER
Programming for Data Science Lab (MACSE502)
Record Practice Program

Date: 09/07/2026

Week No.: 1

Program S. No. 1

	Name	Age	City
1	Bob	30	London
2	Charlie	28	Paris
3	David	35	Tokyo

Figure 1.3: Filtered Data (Age > 25)

Figure 1.3 presents the filtered DataFrame containing only those individuals whose Age is greater than 25. This filtering operation is performed using a conditional expression in Pandas, where the Age column is checked against the condition `Age > 25`. As a result, only the rows

satisfying the condition are retained in the new DataFrame `filtered_df`. Figure 1.3 operation demonstrates how Pandas can be used to extract specific records based on user-defined criteria, which is a common requirement in data analysis and data preprocessing tasks.

	Name	Age	City	Country
0	Alice	23	New York	India
1	Bob	30	London	India
2	Charlie	28	Paris	India
3	David	35	Tokyo	India
4	Emma	22	Sydney	India

Figure 1.4: DataFrame with Country Column

Figure 1.4 displays the DataFrame after adding a new column named `Country` with a default value of "India" for all records. This operation is performed using Pandas column assignment, where the same value is assigned to every row in the newly created column. Adding new columns is a common data transformation technique used to enrich datasets with additional information. Figure 1.4 output confirms that the `Country` column has been successfully added to the DataFrame, and each individual has been assigned the default value India. The enriched dataset is now ready for further operations such as sorting and statistical analysis.

	Name	Age	City	Country
3	David	35	Tokyo	India
1	Bob	30	London	India
2	Charlie	28	Paris	India
0	Alice	23	New York	India
4	Emma	22	Sydney	India

Figure 1.5 Sorted Data by Age (Descending Order)

Figure 1.5 illustrates the DataFrame after sorting the records based on the Age column in descending order using the Pandas `sort_values()` function. In this operation, individuals are arranged from the highest age to the lowest age. Sorting helps organize data systematically and makes it easier to identify records with

the largest or smallest values. Figure 1.5 output confirms that the DataFrame has been successfully sorted in descending order of Age, placing the oldest individual at the top and the youngest individual at the bottom. Such sorting operations are commonly used in data analysis records efficiently.

Figure 1.6 Average Age of Individuals

Figure 1.6 presents the average age of all individuals in the dataset, calculated using the Pandas `mean()` function. The average (mean) age is obtained by summing all the age values and dividing the total by the number of individuals in the DataFrame. This operation demonstrates the use of aggregate statistical functions in Pandas for summarizing numerical data. Figure 1.6 output confirms that the Pandas `mean()` function has successfully computed the average age of the five individuals as 27.6 years. Such aggregate functions are widely used in data analysis to summarize and interpret numerical information efficiently.

Student Reg. No: 26MML0031 Name: Makada Hasnain Husainbhai

Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT), Vellore

2026 FALL SEMESTER
Programming for Data Science Lab (MACSE502)
Record Practice Program

Date: 09/07/2026

Week No.: 1

Program S. No. 1

Conclusions	This experiment successfully demonstrated the use of Pandas for basic data manipulation and analysis. A DataFrame containing the details of five individuals was created using the columns Name, Age, and City. Various DataFrame operations were then performed, including selecting specific columns, filtering rows based on a condition, adding a new column with default values, sorting records, and calculating statistical measures. These operations showcased the flexibility and efficiency of Pandas in handling tabular data. The results showed that: • The Name and Age columns could be extracted independently for focused analysis. • Filtering enabled the identification of individuals whose Age was greater than 25 years. • A new Country column was successfully added to enrich the dataset. • Sorting arranged the records in descending order of age, making it easy to identify the oldest and youngest individuals. • The average age of the individuals was calculated as 27.6 years, demonstrating the use of aggregate functions in data analysis. Overall, this exercise provided a strong foundation in DataFrame creation, data selection, filtering, transformation, sorting, and statistical analysis using Pandas. These fundamental data manipulation techniques are widely used in Data Science, Machine Learning, Data Analytics, and Business Intelligence applications, making them essential skills for working with real-world datasets.
References	1. Michael Freeman and Joel Ross, Programming Skills for Data Science: Start Writing Code to Wrangle, Analyze, and Visualize Data with R, Addison-Wesley, 2018 2. Pandas Documentation: https://pandas.pydata.org/docs/ 3. Python Documentation: https://docs.python.org/3/ 4. Python Program (Google Colab link) https://colab.research.google.com/drive/1sjdIVacPTNg2JO_nGS8gic99hGEtMuRe?usp=sharing 5. Class lecture link https://drive.google.com/file/d/15diEukCtrPLimox63oa9ce1U-t4qVMUy/view

2025 WINTER SEMESTER
Programming for Data Science Lab (MACSE502)
Record Exercise Program

Date: 09/07/2026

Week No.: 1

Program S. No. 2

Title	Data Manipulation and Analysis using the Starwars Dataset in R				
Problem Statement	An AI company maintains model performance records containing: • ModelID • Accuracy • Algorithm Perform: 1. Select ModelID and Accuracy. 2. Filter models with Accuracy greater than 90%. 3. Add Status with default value "Production Ready". 4. Sort models by Accuracy. 5. Calculate average model accuracy.				
Objectives	• Understand how AI model performance data is stored and represented in a Pandas DataFrame. • Filter machine learning models based on conditions, such as accuracy greater than 90%. • Learn how to select specific columns such as ModelID and Accuracy. • Sort AI models according to their accuracy values. • Calculate statistical values such as average, minimum, and maximum model accuracy. • Rank machine learning models based on their performance.				
Dataset URL Link / File	ModelID	Algorithm	PreviousAccuracy	Accuracy	
	M101	Logistic Regression	84.5	87.2	
	M102	Decision Tree	87.0	89.5	
	M103	Random Forest	90.2	93.8	
	M104	Support Vector Machine	88.5	91.4	
	M105	K-Nearest Neighbors	82.5	85.6	
	M106	Random Forest	91.0	94.5	
	M107	Decision Tree	85.5	88.3	
	M108	Neural Network	92.0	96.1	
	M109	Logistic Regression	86.0	90.8	
	M110	Neural Network	93.1	95.4	
	M111	Gradient Boosting	89.4	92.7	
	M112	Gradient Boosting	91.8	94.1	
Functions Details	Sl. No.	Function	Package	Description of function	
	1	pd.DataFrame()	Pandas	Creates a DataFrame from the AI model data dictionary.	
	2	pd.read_csv()	Pandas	Reads AI model records from a CSV file.	
	3	dropna()	Pandas	Removes rows containing missing or invalid values	
	4	drop_duplicates()	Pandas	Removes duplicate records based on ModelID.	

Student Reg. No: 26MML0031 Name: Hasnain Makada

Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT), Vellore

2025 WINTER SEMESTER
Programming for Data Science Lab (MACSE502)
Record Exercise Program

Date: 09/07/2026

Week No.: 1

Program S. No. 2

	5	sort_values()	Pandas	Sorts the models according to their accuracy values.
	6	mean()	Pandas	Calculates the average accuracy of all AI models.
	7	groupby() and agg()	Pandas	Groups models by algorithm and calculates performance statistics.
	8	to_csv()	Pandas	Saves the analysed model results into a CSV file.
Steps/ Process/ Procedure/ Algorithm/ Flowchart/ Model	1. Import the Pandas package. 2. Create the AI model dataset containing ModelID, Algorithm, PreviousAccuracy and Accuracy. 3. Convert the dataset into a Pandas DataFrame. 4. Clean the dataset by removing missing values, duplicate ModelIDs and invalid accuracy values. 5. Select only the ModelID and Accuracy columns. 6. Filter the models whose Accuracy is greater than 90%. 7. Add a Status column with the default value "Production Ready". 8. Change the status to "Needs Improvement" for models having Accuracy below 90%. 9. Sort the models in descending order based on Accuracy. 10. Assign ranks to the models, where the highest-accuracy model receives Rank 1. 11. Calculate the improvement of each model using: $\text{Improvement} = \text{Accuracy} - \text{PreviousAccuracy}$ 12. Calculate the average accuracy of all models. 13. Group the models by Algorithm and calculate minimum, maximum and average accuracy. 14. Identify the best-performing, lowest-performing and most-improved models. 15. Display all the analysis results in the console. 16. Generate an automated model evaluation report. 17. Save the analysed results in a CSV file and the report in a text file.			
Program	<pre>import sys from pathlib import Path import pandas as pd from analysis_core import (analyse_models, clean_data, create_algorithm_summary, create_report, create_sample_data, perform_basic_operations)</pre>			

Student Reg. No: 26MML0031 Name: Hasnain Makada

Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT), Vellore

2025 WINTER SEMESTER
Programming for Data Science Lab (MACSE502)
Record Exercise Program

Date: 09/07/2026

Week No.: 1

Program S. No. 2

```
#
=====
=====
# HEADING FUNCTION
# This keeps console output neat and
readable. #
=====
=====
def print_heading(title):
    print("\n" + "=" * 72)
    print(title)
    print("=" * 72)

#
=====
=====
# MAIN CONSOLE
PROGRAM #
# Run with sample data:
# python console_app.py
#
# Run with CSV data:
# python console_app.py
sample_ai_models.csv #
=====
=====
def main():
    print_heading("AI MODEL PERFORMANCE ANALYSIS
- CONSOLE MODE")

    csv_file = None

    # If the user gives a file path, use that
    CSV. if len(sys.argv) > 1:
        csv_file = sys.argv[1]

    if csv_file:
        csv_path = Path(csv_file)

        if not csv_path.exists():
            print(f"Error: File not found:
{csv_path}") return

        print(f"Using CSV file: {csv_path}")
        raw_data = pd.read_csv(csv_path)

    else:
        print("Using built-in sample data.")
```

Student Reg. No: 26MML0031 Name: Hasnain Makada

Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT), Vellore

2025 WINTER SEMESTER
Programming for Data Science Lab (MACSE502)
Record Exercise Program

Date: 09/07/2026

Week No.: 1

Program S. No. 2

```
raw_data = create_sample_data()

try:
    cleaned_data =
clean_data(raw_data) except
ValueError as error:
    print(f'Data error: {error}')
    return

analysed_data =
    analyse_models( cleaned_data
    , production_threshold=90.0
)

results =
    perform_basic_operations( analy
    sed_data
)

print_heading("ORIGINAL CLEANED DATA")
print(
    cleaned_data.to_string(index=False)
)

print_heading("1. SELECT MODELID AND
ACCURACY") print(
    results["selected_columns"]
    .to_string(index=False)
)

print_heading(
    "2. FILTER MODELS WITH ACCURACY GREATER
    THAN 90%"
)
print(
    results["models_above_90"][[
        ["ModelID", "Algorithm",
        "Accuracy"]
    ]].to_string(index=False)
)

print_heading(
    '3. ADD STATUS WITH DEFAULT "PRODUCTION READY"'
)
print(
    results["status_table"]
    .to_string(index=False)
)
print_heading("4. SORT MODELS BY ACCURACY") print(
```

Student Reg. No: 26MML0031 Name: Hasnain Makada

Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT), Vellore

2025 WINTER SEMESTER
Programming for Data Science Lab (MACSE502)
Record Exercise Program

Date: 09/07/2026

Week No.: 1

Program S. No. 2

```
results["sorted_models"][\n    ["ModelID", "Algorithm", "Accuracy"]\n].to_string(index=False)\n)\n\nprint_heading("5. CALCULATE AVERAGE MODEL\nACCURACY")\nprint(\n    f'Average model accuracy: '\n    f'{results["average_accuracy"]:.2f}%\n')\n\nprint_heading("MODEL RANKING")\nprint(\n    analysed_data[\n        [\n            "Rank",\n            "ModelID",\n            "Algorithm",\n            "PreviousAccuracy",\n            "Accuracy",\n            "Improvement",\n            "Status"\n        ]\n    ].to_string(index=False)\n)\n\nprint_heading("ALGORITHM PERFORMANCE COMPARISON")\nalgorithm_summary =\n    create_algorithm_summary( analysed_data\n)\n\nprint(\n    algorithm_summary.to_string(index=False)\n)\n\nprint_heading("ACCURACY IMPROVEMENT TRACKING")\nprint(\n    analysed_data[\n        [\n            "ModelID",\n            "PreviousAccuracy",\n            "Accuracy",\n            "Improvement",\n            "ImprovementStatus"\n        ]\n    ].to_string(index=False)\n)
```

Student Reg. No: 26MML0031 — Name: Hasnain Makada

Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT), Vellore

2025 WINTER SEMESTER
Programming for Data Science Lab (MACSE502)
Record Exercise Program

Date: 09/07/2026

Week No.: 1

Program S. No. 2

```
report = create_report(
    analysed_data,
    production_threshold=90.0
)

print_heading("AUTOMATED EVALUATION
REPORT") print(report)

# Save analysed
data.
analysed_data.to_csv(
    "console_ai_model_results.csv",
    index=False
)

# Save the report.
with open(
    "console_ai_model_report.txt",
    "w",
    encoding="utf-8"
) as report_file:
    report_file.write(report)

print_heading("FILES CREATED")
print("console_ai_model_results.csv")
print("console_ai_model_report.txt")

if __name__ == "__main__":
    main()
```

**Output
Snapshots
With
Explanation/
Observations**

ORIGINAL CLEANED DATA

ModelID	Algorithm	PreviousAccuracy	Accuracy
M101	Logistic Regression	84.5	87.2
M102	Decision Tree	87.0	89.5
M103	Random Forest	90.2	93.8
M104	Support Vector Machine	88.5	91.4
M105	K-Nearest Neighbors	82.5	85.6
M106	Random Forest	91.0	94.5
M107	Decision Tree	85.5	88.3
M108	Neural Network	92.0	96.1
M109	Logistic Regression	86.0	90.8
M110	Neural Network	93.1	95.4
M111	Gradient Boosting	89.4	92.7
M112	Gradient Boosting	91.8	94.1

Figure 2.1 Original Cleaned Dataset

The ModelID and Accuracy columns were selected successfully. Models having accuracy greater than 90% were filtered from the dataset.

The program successfully loaded and cleaned the AI model performance dataset. The dataset contained ModelID, Algorithm, PreviousAccuracy and Accuracy.

1. SELECT MODELID AND ACCURACY

ModelID	Accuracy
M108	96.1
M110	95.4
M106	94.5
M112	94.1
M103	93.8
M111	92.7
M104	91.4
M109	90.8
M102	89.5
M107	88.3
M101	87.2
M105	85.6

Figure 1.2 Selected ModelID and Accuracy

Student Reg. No: 26MML0031

Name: Hasnain Makada

Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT), Vellore

2025 WINTER SEMESTER
Programming for Data Science Lab (MACSE502)
Record Exercise Program

Date: 09/07/2026

Week No.: 1

Program S. No. 2

2. FILTER MODELS WITH ACCURACY GREATER THAN 90%

ModelID	Algorithm	Accuracy
M108	Neural Network	96.1
M110	Neural Network	95.4
M106	Random Forest	94.5
M112	Gradient Boosting	94.1
M103	Random Forest	93.8
M111	Gradient Boosting	92.7
M104	Support Vector Machine	91.4
M109	Logistic Regression	90.8

Figure 1.3 Models Accuracy Greater than 90%

A Status column was added. Models with accuracy greater than or equal to 90% were marked as "Production Ready", while the remaining

3. ADD STATUS WITH DEFAULT "PRODUCTION READY"

ModelID	Accuracy	Status
M108	96.1	Production Ready
M110	95.4	Production Ready
M106	94.5	Production Ready
M112	94.1	Production Ready
M103	93.8	Production Ready
M111	92.7	Production Ready
M104	91.4	Production Ready
M109	90.8	Production Ready
M102	89.5	Needs Improvement
M107	88.3	Needs Improvement
M101	87.2	Needs Improvement
M105	85.6	Needs Improvement

Figure 1.4 Status Column and Models sorted by accuracy

5. CALCULATE AVERAGE MODEL ACCURACY
 Average model accuracy: 91.62%
 Improvement

MODEL RANKING

Rank	ModelID	Algorithm	PreviousAccuracy	Accuracy	Improvement	Status
1	M108	Neural Network	92.0	96.1	4.1	Production Ready
2	M110	Neural Network	93.1	95.4	2.3	Production Ready
3	M106	Random Forest	91.0	94.5	3.5	Production Ready
4	M112	Gradient Boosting	91.8	94.1	2.3	Production Ready
5	M103	Random Forest	90.2	93.8	3.6	Production Ready
6	M111	Gradient Boosting	89.4	92.7	3.3	Production Ready
7	M104	Support Vector Machine	88.5	91.4	2.9	Production Ready
8	M109	Logistic Regression	86.0	90.8	4.8	Production Ready
9	M102	Decision Tree	87.0	89.5	2.5	Needs Improvement
10	M107	Decision Tree	85.5	88.3	2.8	Needs Improvement
11	M101	Logistic Regression	84.5	87.2	2.7	Needs Improvement
12	M105	K-Nearest Neighbors	82.5	85.6	3.1	Needs Improvement

Figure 1.5 Average Model Accuracy

The average accuracy of all models was calculated. Accuracy improvement was also calculated by subtracting PreviousAccuracy from the current Accuracy.

2025 WINTER SEMESTER
Programming for Data Science Lab (MACSE502)
Record Exercise Program

Date: 09/07/2026

Week No.: 1

Program S. No. 2

ALGORITHM PERFORMANCE COMPARISON						
Algorithm	ModelCount	MinimumAccuracy	MaximumAccuracy	AverageAccuracy	AverageImprovement	
Neural Network	2	95.4	96.1	95.75	3.20	
Random Forest	2	93.8	94.5	94.15	3.55	
Gradient Boosting	2	92.7	94.1	93.40	2.80	
Support Vector Machine	1	91.4	91.4	91.40	2.90	
Logistic Regression	2	87.2	90.8	89.00	3.75	
Decision Tree	2	88.3	89.5	88.90	2.65	
K-Nearest Neighbors	1	85.6	85.6	85.60	3.10	

Figure 1.6 Models Grouped according to their algorithm and other metrics

The models were grouped according to their algorithms, and minimum, maximum and average accuracy values were calculated for each algorithm.

```
SUMMARY
Total models analysed: 12
Production threshold: 90.0%
Average model accuracy: 91.62%
Production-ready models: 8
Models needing improvement: 4

BEST MODEL
Model ID: M108
Algorithm: Neural Network
Accuracy: 96.10%

LOWEST-PERFORMING MODEL
Model ID: M105
Algorithm: K-Nearest Neighbors
Accuracy: 85.60%

MOST IMPROVED MODEL
Model ID: M109
Algorithm: Logistic Regression
Improvement: 4.80 percentage points

BEST ALGORITHM
Algorithm: Neural Network
Average accuracy: 95.75%
```

The program identified the best-performing model, lowest-performing model, most-improved model and best-performing algorithm.

Figure 1.7 Final Output

Conclusions

The exercise successfully demonstrated how Pandas can be used to analyse AI model performance data. The program selected, filtered, sorted and ranked the models, calculated average accuracy, compared algorithm performance and tracked accuracy improvement. It also generated an automated evaluation report and saved the final results in CSV and text formats.

References

1. Michael Freeman and Joel Ross, Programming Skills for Data Science: Start Writing Code to Wrangle, Analyze, and Visualize Data with R, Addison-Wesley, 2018
2. Python Program
https://github.com/hasnainmakada-99/Data-Science-Codes/blob/main/WEEK%201/Excercise_Program_Console.py
3. Class lecture link
https://drive.google.com/file/d/15diEukCtrPLimox63oa9ce1U-t4qVMUy/view?usp=drive_link

Student Reg. No: 26MML0031 Name: Hasnain Makada

Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT), Vellore

2026 FALL SEMESTER
Programming for Data Science Lab (MACSE502)
Record Practice Program

Date: 16/07/2026

Week No.: 2

Program S. No. 3

Title	Student Marks Data Manipulation and Analysis using Pandas																																	
Problem Statement	Data Cleaning and Preprocessing with Pandas: Create a DataFrame with some missing values in columns Name, Age, and City. Perform the following operations: fill missing values in the Age column with the mean age, drop rows where Name or City is missing, and convert the Age column to integer type. Finally, normalize the Age column using min-max scaling.																																	
Objectives	● To understand how to create a Pandas DataFrame containing missing values. ● To learn how to identify and handle missing data using the fillna() function. ● To remove incomplete records using the dropna() function. ● To convert a DataFrame column to the required data type using the astype() function. ● To normalize numerical data using the Min-Max Scaling technique. To understand the importance of data preprocessing before performing data analysis and machine learning tasks.																																	
Dataset URL Link / File	<table><tr><th>Name</th><th>Age</th><th>City</th></tr><tr><td>Arjun</td><td>np.nan</td><td>np.nan</td></tr><tr><td>Priya</td><td>28</td><td>Mumbai</td></tr><tr><td>Rohan</td><td>35</td><td>Delhi</td></tr><tr><td>Sneha</td><td>np.nan</td><td>np.nan</td></tr><tr><td>Amit</td><td>40</td><td>Bangalore</td></tr><tr><td>Deepa</td><td>30</td><td>Chennai</td></tr><tr><td>Rahul</td><td>np.nan</td><td>np.nan</td></tr><tr><td>np.nan</td><td>33</td><td>Kolkata</td></tr><tr><td>np.nan</td><td>29</td><td>Ahmedabad</td></tr></table>				Name	Age	City	Arjun	np.nan	np.nan	Priya	28	Mumbai	Rohan	35	Delhi	Sneha	np.nan	np.nan	Amit	40	Bangalore	Deepa	30	Chennai	Rahul	np.nan	np.nan	np.nan	33	Kolkata	np.nan	29	Ahmedabad
Name	Age	City																																
Arjun	np.nan	np.nan																																
Priya	28	Mumbai																																
Rohan	35	Delhi																																
Sneha	np.nan	np.nan																																
Amit	40	Bangalore																																
Deepa	30	Chennai																																
Rahul	np.nan	np.nan																																
np.nan	33	Kolkata																																
np.nan	29	Ahmedabad																																
Functions Details	Sl. No.	Function	Package	Description of function																														
	1	DataFrame()	pandas	Creates a DataFrame from structured data																														
	2	Mean()	pandas	Calculates the average value																														
	3	fillna()	pandas	Replaces missing values with a specified value.																														
	4	dropna()	pandas	Removes rows containing missing values.																														

Student Reg. No: 26MML0031

Name: Makada Hasnain Husainbhai

Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT), Vellore

2026 FALL SEMESTER
Programming for Data Science Lab (MACSE502)
Record Practice Program

Date: 16/07/2026

Week No.: 2

Program S. No. 3

Steps/ Process/ Procedure/ Algorithm/ Flowchart/ Model	Step 1 Import the pandas library. Step 2 Create a Python dictionary containing StudentID, Name, and Marks for five students. Step 3 Convert the dictionary into a Pandas DataFrame and display the original dataset. Step 4 Select and display only the Name and Marks columns. Step 5 Filter and display students whose Marks are greater than 70. Step 6 Add a Grade column by assigning grades A to E according to the marks obtained. Step 7 Display the updated DataFrame containing the Grade column. Step 8 Sort the DataFrame by Marks in ascending order and display the result. Step 9 Calculate and display the average marks using the mean() function.
Program	<pre># Import the Pandas library import pandas as pd # Create a dictionary containing student details dictionary_1 = { "StudentID": [10, 11, 12, 13, 14], "Name": ["Hasnain", "John", "Alice", "Sahil", "Cook"], "Marks": [45, 86, 90, 67, 97] } # Convert the dictionary into a Pandas DataFrame dataframe = pd.DataFrame(dictionary_1) # Display the original dataset print("Displaying the Original Dataset\ n") print(dataFrame, "\n") # Select and display only the Name and Marks columns only_name_colum = pd.DataFrame(dataframe, columns=["Name", "Marks"]) print("Displaying only name and marks columns\n") print(only_name_colum, "\n") # Filter students whose marks are greater than 70 marks_greater_than_70 = pd.DataFrame(dataframe[dataFrame["Marks"] > 70]) print("Displaying students with marks greater than 70\n") print(marks_greater_than_70, "\n") # Assign grades based on the marks obtained dataFrame["Grade"] = ["A" if marks >= 90 else "B" if marks >= 80 else "C" if marks >= 70 else "D" if marks >= 60 else "E" for marks in dataFrame["Marks"]]</pre>

Student Reg. No: 26MML0031

Name: Makada Hasnain Husainbhai

Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT), Vellore

2026 FALL SEMESTER
Programming for Data Science Lab (MACSE502)
Record Practice Program

Date: 16/07/2026

Week No.: 2

Program S. No. 3

```
]

# Display the updated dataset with the Grade column
print("Displaying the updated dataset\n")
print(dataFrame, "\n")

# Sort the dataset by Marks in ascending order
sort_by_marks = dataFrame.sort_values(
    by="Marks", ascending=True
)
print("Displaying the sorted dataset\n")
print(sort_by_marks, "\n")

# Calculate and display the average
marks average =
dataFrame["Marks"].mean()
print("Average Marks \n")
print(average)
```

Displaying the Original Dataset

```
***
   StudentID  Name  Marks
0         10  Hasnain    45
1         11   John    86
2         12   Alice    90
3         13   Sahil    67
4         14    Cook    97
```

Figure 2.1 Original Student Dataset

Figure 2.1 displays the original DataFrame created from student dictionary. It contains the columns StudentID, Name, and Marks for five students. The pd.DataFrame function converts the dictionary into a structured format that can be analysed using Pandas.

2.2 shows only the Name and Marks columns selected from the original DataFrame. This demonstrates column selection and allows the analysis to focus on the student names and their corresponding marks without displaying StudentID.

Displaying only name and marks columns

```
   Name  Marks
0  Hasnain    45
1    John    86
2   Alice    90
3   Sahil    67
4    Cook    97
```

Figure 2.2 Selected Name and Marks Columns

**Output
Snapshots
With
Explanation/
Observations**

Displaying students with marks greater than 70

```
   StudentID  Name  Marks
1         11   John    86
2         12   Alice    90
4         14    Cook    97
```

Figure 2.3 Filtering student with marks > 70

Figure 2.3 presents the filtered records for students whose marks are greater than 70. The Boolean condition `dataFrame["Marks"] > 70` retains John, Alice, and Cook because their marks satisfy the specified condition.

2.4 displays the updated DataFrame after adding the Grade column. Grades are assigned using conditional logic: A for marks of 90 or above, B for 80-89, C for 70-79, D for 60-69, and marks below 60.

Displaying the updated dataset

```
   StudentID  Name  Marks  Grade
0         10  Hasnain    45     E
1         11   John    86     B
2         12   Alice    90     A
3         13   Sahil    67     D
4         14    Cook    97     A
```

Figure 2.4 Dataset after Adding the Grade Column

Displaying the sorted dataset

```
   StudentID  Name  Marks  Grade
0         10  Hasnain    45     E
3         13   Sahil    67     D
1         11   John    86     B
2         12   Alice    90     A
4         14    Cook    97     A
```

2.5 Dataset Sorted by Marks in Ascending Order

Figure 2.5 shows the records sorted from the lowest to the highest marks using `sort_values()`. Hasnain appears first with 45 marks, while Cook appears last with 97 marks. The original row indexes are retained by Pandas.

Student Reg. No: 26MML0031 Name: Makada Hasnain Husainbhai

Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT), Vellore

2026 FALL SEMESTER
Programming for Data Science Lab (MACSE502)
Record Practice Program

Date: 16/07/2026

Week No.: 2

Program S. No. 3

Figure using five of	2.6 displays the average marks calculated the mean() function. The total marks of all students divided by five gives an average 77.0.	Figure 2.6 Average Marks
Conclusions	This exercise successfully demonstrated basic student-data manipulation and analysis using Pandas. A DataFrame containing StudentID, Name, and Marks was created from a Python dictionary, and multiple operations were performed on the dataset. The results showed that: • The Name and Marks columns were selected successfully for focused analysis. • Students with marks greater than 70 were identified using Boolean filtering. • A Grade column was generated using conditional logic based on each student's marks. • The records were arranged in ascending order of marks using sort_values(). • The average marks of the five students were calculated as 77.0 using mean(). Overall, the exercise provided practical experience in DataFrame creation, column selection, row filtering, conditional transformation, sorting, and statistical aggregation. These operations are fundamental for data preprocessing and exploratory data analysis in Data Science and Machine Learning applications.	
References	1. Wes McKinney, Python for Data Analysis: Data Wrangling with pandas, NumPy, and Jupyter, 3rd Edition, O'Reilly Media, 2022. 2. Pandas Documentation: https://pandas.pydata.org/docs/ 3. Python Documentation: https://docs.python.org/3/ 4. Python Program (Google Colab Link) https://colab.research.google.com/drive/1qBzvtH6ZEis8Jv20WDiYXKI4PXGSXcWJ?usp=sharing 4. Pandas DataFrame API Reference: https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.html 5. Class lecture link: https://drive.google.com/file/d/15diEukCtrPLimox63oa9ce1U-t4qVMUy/view	

Student Reg. No: 26MML0031 Name: Makada Hasnain Husainbhai

Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT), Vellore

2025 WINTER SEMESTER
Programming for Data Science Lab (MACSE502)
Record Exercise Program

Date: 16/07/2026

Week No.: 2

Program S. No. 4

Title	Data Manipulation and Analysis using the Starwars Dataset in R					
Problem Statement	A streaming platform maintains movie records containing: • MovieID • Rating • Genre Perform: 1. Create a dataset with missing values. 2. Fill missing Rating values using average rating. 3. Remove rows with missing MovieID or Genre. 4. Convert Rating into integer type. 5. Normalize rating values.					
Objectives	• To understand movie data preprocessing using the Pandas library. • To create a dataset containing missing values. • To identify and handle missing values in a dataset. • To fill missing Rating values using the average rating. • To remove records with missing MovieID or Genre. • To convert the Rating column into integer data type. • To normalize movie rating values using Min-Max normalization. • To identify top-rated movies based on ratings.					
Dataset URL Link / File	MovieID	MovieTitle	Rating	Genre	ViewCount	Likes
	MV101	The Last Horizon	8.5	Science Fiction	125000	11200
	MV102	City Lights		Comedy	98000	7500
	MV103	Hidden Truth	7.8	Thriller	110000	8900
	None	Lost Kingdom	9.1	Fantasy	150000	13500
	MV105	Digital Dreams	8.9	Science Fiction	175000	16000
	MV106	Silent River		Drama	87000	6100
	MV107	Future Code	9.3	Science Fiction	210000	19800
	MV108	Broken Compass	6.7		65000	4200
	MV109	Midnight Journey	8.1	Adventure	132000	10400
	MV110	Laugh Again	7.5	Comedy	105000	8100
	MV111	Ocean Mystery	8.8	Mystery	143000	12100
	MV112	Final Mission		Action	190000	17400

Student Reg. No: 26MML0031 Name: Makada Hasnain H

Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT), Vellore

2025 WINTER SEMESTER
Programming for Data Science Lab (MACSE502)
Record Exercise Program

Date: 16/07/2026

Week No.: 2

Program S. No. 4

	MV113	Parallel Lives	8.4	Drama	118000	9600
	MV114	Wild Trails	7.2	Adventure	99000	7200
	MV115	The Great Escape	9.0	Action	205000	18800
Functions Details	Sl. No.	Function	Package	Description of function		
	1	pd.DataFrame()	Pandas	Creates a DataFrame from the movie dataset stored in dictionary format.		
	2	isnull()	Pandas	Identifies missing values present in the movie dataset.		
	3	mean()	Pandas	Fills missing Rating values using the calculated average rating.		
	4	dropna()	Pandas	Removes rows containing missing MovieID or Genre values.		
	5	pd.to_numeric()	Pandas	Converts Rating values into numeric format and handles invalid entries.		
	6	round()	Pandas	Rounds decimal Rating values before converting them into integers.		
	7	astype()	Pandas	Converts the Rating column into integer data type.		
	8	min()	Pandas	Finds the minimum Rating value required for normalization.		
	9	max()	Pandas	Finds the maximum Rating value required for normalization.		
	10	groupby()	Pandas	Groups movie records according to Genre for genre-wise analysis.		
	11	sort_values()	Pandas	Sorts movies based on rating, popularity score or number of views.		
	12	fillna()	Pandas	Fills missing Rating values using the calculated average rating.		
Steps/ Process/ Procedure/ Algorithm/ Flowchart/ Model	Step 1: Import the Pandas library and create the movie dataset in dictionary format.					
	Step 2: Convert the dictionary into a Pandas DataFrame using pd.DataFrame().					
	Step 3: Display the original dataset containing missing values.					
	Step 4: Calculate the average value of the Rating column using the mean() function.					
	Step 5: Fill the missing Rating values using fillna() with the calculated average rating.					

Student Reg. No: 26MML0031 Name: Makada Hasnain H

Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT), Vellore

2025 WINTER SEMESTER
Programming for Data Science Lab (MACSE502)
Record Exercise Program

Date: 16/07/2026

Week No.: 2

Program S. No. 4

	Step 6: Remove rows containing missing MovieID or Genre using the dropna() function. Step 7: Convert the Rating column into numeric format using pd.to_numeric() and then convert it into integer type using round() and astype(int). Step 8: Normalize the Rating values using the Min-Max Normalization formula: Normalized Rating = (Rating – Minimum Rating) / (Maximum Rating – Minimum Rating) Step 9: Calculate the Engagement Rate and Popularity Score for each movie based on views, likes and ratings. Step 10: Sort the movies according to Popularity Score and Rating using sort_values() and assign popularity ranks. Step 11: Generate a genre-wise report using the groupby() function to compare movie performance across different genres. Step 12: Recommend the top movies based on selected genre and minimum rating, generate the automated analytics report, and save the processed results for future analysis.
Program	<pre>import pandas as pd # # 1. CREATE MOVIE DATASET IN DICTIONARY FORMAT # movie_data = { "MovieID": ["MV101", "MV102", "MV103", None, "MV105", "MV106", "MV107", "MV108", "MV109", "MV110", "MV111", "MV112", "MV113", "MV114", "MV115"], "MovieTitle": ["The Last Horizon", "City Lights", "Hidden Truth", "Lost Kingdom", "Digital Dreams", "Silent River", "Future Code", "Broken Compass", "Midnight Journey",] }</pre>

Student Reg. No: 26MML0031 Name: Makada Hasnain H

Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT), Vellore

2025 WINTER SEMESTER
Programming for Data Science Lab (MACSE502)
Record Exercise Program

Date: 16/07/2026

Week No.: 2

Program S. No. 4

```
"Laugh Again",
"Ocean Mystery",
"Final Mission",
"Parallel Lives",
"Wild Trails",
"The Great
Escape"
],

"Rating": [
    8.5, None, 7.8, 9.1, 8.9,
    None, 9.3, 6.7, 8.1, 7.5,
    8.8, None, 8.4, 7.2, 9.0
],

"Genre": [
    "Science
    Fiction",
    "Comedy",
    "Thriller",
    "Fantasy",
    "Science
    Fiction",
    "Drama",
    "Science
    Fiction", None,
    "Adventure",
    "Comedy",
    "Mystery",
    "Action",
    "Drama",
    "Adventure",
    "Action"
],

"ViewCount": [
    125000, 98000, 110000, 150000, 175000,
    87000, 210000, 65000, 132000, 105000,
    143000, 190000, 118000, 99000, 205000
],

"Likes": [
    11200, 7500, 8900, 13500, 16000,
    6100, 19800, 4200, 10400, 8100,
    12100, 17400, 9600, 7200, 18800
]
} -----
# 2. CONVERT DICTIONARY INTO DATAFRAME
```

Student Reg. No: 26MML0031 Name: Makada Hasnain H

Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT), Vellore

2025 WINTER SEMESTER
Programming for Data Science Lab (MACSE502)
Record Exercise Program

Date: 16/07/2026

Week No.: 2

Program S. No. 4

```
# -----
movies = pd.DataFrame(movie_data)

print("\nORIGINAL MOVIE DATASET")
print("=" * 90)
print(movies.to_string(index=False))

# -----
# 3. CALCULATE AVERAGE
RATING # -----
average_rating = movies["Rating"].mean()

print("\nAVERAGE RATING")
print("=" * 90)
print("Average Rating:", round(average_rating, 2))

# -----
# 4. FILL MISSING RATING
VALUES # -----
movies["Rating"] = movies["Rating"].fillna(average_rating)

print("\nDATASET AFTER FILLING MISSING RATING VALUES")
print("=" * 90)
print(
    movies[
        ["MovieID", "MovieTitle", "Rating", "Genre"]
    ].to_string(index=False)
)

# -----
# 5. REMOVE ROWS WITH MISSING MOVIEID OR
GENRE # -----
movies = movies.dropna(subset=["MovieID", "Genre"])

print("\nDATASET AFTER REMOVING MISSING MOVIEID OR
GENRE")
print("=" * 90)
print(
    movies[
        ["MovieID", "MovieTitle", "Rating", "Genre"]
    ].to_string(index=False)
)

# -----
```

Student Reg. No: 26MML0031 — Name: Makada Hasnain H

Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT), Vellore

2025 WINTER SEMESTER
Programming for Data Science Lab (MACSE502)
Record Exercise Program

Date: 16/07/2026

Week No.: 2

Program S. No. 4

```
# 6. CONVERT RATING INTO INTEGER
TYPE #-----
movies["Rating"] = movies["Rating"].round().astype(int)

print("\nRATING CONVERTED INTO INTEGER TYPE")
print("=" * 90)
print(
    movies[
        ["MovieID", "MovieTitle", "Rating"]
    ].to_string(index=False)
)

print("\nRating Data Type:", movies["Rating"].dtype)

# -----
# 7. NORMALIZE RATING VALUES
# Min-Max
Normalization #-----
minimum_rating = movies["Rating"].min()
maximum_rating = movies["Rating"].max()

movies["NormalizedRating"] = (
    (movies["Rating"] -
     minimum_rating)
    / (maximum_rating - minimum_rating)
).round(3)

print("\nNORMALIZED RATING VALUES")
print("=" * 90)
print(
    movies[
        [
            "MovieID",
            "MovieTitle",
            "Rating",
            "NormalizedRating"
        ]
    ].to_string(index=False)
)

# -----
# 8. CALCULATE ENGAGEMENT
RATE #-----
movies["EngagementRate"] = (
    movies["Likes"] / movies["ViewCount"] * 100
).round(2)
```

Student Reg. No: 26MML0031 — Name: Makada Hasnain H

Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT), Vellore

2025 WINTER SEMESTER
Programming for Data Science Lab (MACSE502)
Record Exercise Program

Date: 16/07/2026

Week No.: 2

Program S. No. 4

```
print("\nVIEWER ENGAGEMENT")
print("=" * 90)
print(
    movies[
        [
            "MovieTitle",
            "ViewCount",
            "Likes",
            "EngagementRate"
        ]
    ].to_string(index=False)
)

# -----
# 9. CALCULATE POPULARITY SCORE
# -----
maximum_engagement = movies["EngagementRate"].max()

movies["NormalizedEngagement"] = (
    movies["EngagementRate"] / maximum_engagement
).round(3)

movies["PopularityScore"] = (
    movies["NormalizedRating"] *
    0.60
    + movies["NormalizedEngagement"] * 0.40
).round(3)

# -----
# 10. SORT AND RANK
MOVIES # -----
movies = movies.sort_values(
    by=["PopularityScore", "Rating"],
    ascending=[False, False]
).reset_index(drop=True)

movies["PopularityRank"] = range(1, len(movies) +

1) print("\nTOP-RATED AND POPULAMOVIES")
print("=" * 90)
print(
    movies[
        [
            "PopularityRank",
            "MovieID",
            "MovieTitle",
            "Genre",
```

Student Reg. No: 26MML0031 Name: Makada Hasnain H

Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT), Vellore

2025 WINTER SEMESTER
Programming for Data Science Lab (MACSE502)
Record Exercise Program

Date: 16/07/2026

Week No.: 2

Program S. No. 4

```
"Rating",
"ViewCount",
"PopularityScore"
]
].to_string(index=False)
)

# -----
# 11. GENERATE GENRE-WISE REPORT
# -----

genre_report =
    movies.groupby( "Genre",
        as_index=False
    ).agg(
        MovieCount=("MovieID", "count"),
        AverageRating=("Rating", "mean"),
        HighestRating=("Rating", "max"),
        TotalViews=("ViewCount", "sum"),
        TotalLikes=("Likes", "sum")
    )

genre_report["AverageRating"] = (
    genre_report["AverageRating"].round(2)
)

genre_report =
    genre_report.sort_values( by="Avera
        geRating", ascending=False
    )

print("\nGENRE-WISE MOVIE REPORT")
print("=" * 90)
print(genre_report.to_string(index=False))

# -----
# 12. SIMPLE MOVIE
RECOMMENDATION # -----
preferred_genre = "Science Fiction"
minimum_required_rating = 7

recommended_movies = movies[
    (movies["Genre"] ==
        preferred_genre)
    & (movies["Rating"] >= minimum_required_rating)
].sort_values( by="PopularityScor
    e", ascending=False
```

Student Reg. No: 26MML0031 Name: Makada Hasnain H

Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT), Vellore

2025 WINTER SEMESTER
Programming for Data Science Lab (MACSE502)
Record Exercise Program

Date: 16/07/2026

Week No.: 2

Program S. No. 4

```
)

print("\nMOVIE RECOMMENDATIONS")
print("=" * 90)
print("Preferred Genre:", preferred_genre)
print("Minimum Rating:",
minimum_required_rating)

print(
    recommended_movies[
        [
            "MovieID",
            "MovieTitle",
            "Genre",
            "Rating",
            "PopularityScore"
        ]
    ].to_string(index=False)
)

# -----
# 13. AUTOMATED ANALYTICS REPORT
# -----
top_movie = movies.iloc[0]

most_viewed_movie =
    movies.loc[ movies["ViewCount"].idxmax()
]

best_genre = genre_report.iloc[0]

print("\nAUTOMATED MOVIE ANALYTICS REPORT")
print("=" * 90)

print("Total Movies Analysed:", len(movies))
print("Average Movie Rating:", round(movies["Rating"].mean(), 2))
print("Total Genres:", movies["Genre"].nunique())

print("\nTop-Rated and Most Popular Movie")
print("Movie Title:", top_movie["MovieTitle"])
print("Genre:", top_movie["Genre"])
print("Rating:", top_movie["Rating"])
print("Popularity Score:", top_movie["PopularityScore"])

print("\nMost Viewed Movie")
print("Movie Title:",
most_viewed_movie["MovieTitle"]) print("Views:",
most_viewed_movie["ViewCount"])
```

Student Reg. No: 26MML0031 Name: Makada Hasnain H

Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT), Vellore

2025 WINTER SEMESTER
Programming for Data Science Lab (MACSE502)
Record Exercise Program

Date: 16/07/2026

Week No.: 2

Program S. No. 4

```
print("\nBest Performing Genre")
print("Genre:", best_genre["Genre"])
print("Average Rating:", best_genre["AverageRating"])

# -----
# 14. SAVE RESULTS
# -----

movies.to_csv( "processed_movie
_data.csv", index=False
)

genre_report.to_csv( "genre_wise
_report.csv", index=False
)

print("\nFILES CREATED")
print("=" * 90)
print("processed_movie_data.csv")
print("genre_wise_report.csv")
```

**Output
Snapshots
With
Explanation/
Observations**

1. ORIGINAL DATASET WITH MISSING VALUES

MovieID	MovieTitle	Rating	Genre	ViewCount	Likes
MV101	The Last Horizon	8.5	Science Fiction	125000	11200
MV102	City Lights	NaN	Comedy	98000	7500
MV103	Hidden Truth	7.8	Thriller	110000	8900
NaN	Lost Kingdom	9.1	Fantasy	150000	13500
MV105	Digital Dreams	8.9	Science Fiction	175000	16000
MV106	Silent River	NaN	Drama	87000	6100
MV107	Future Code	9.3	Science Fiction	210000	19800
MV108	Broken Compass	6.7	NaN	65000	4200
MV109	Midnight Journey	8.1	Adventure	132000	10400
MV110	Laugh Again	7.5	Comedy	105000	8100
MV111	Ocean Mystery	8.8	Mystery	143000	12100
MV112	Final Mission	NaN	Action	190000	17400
MV113	Parallel Lives	8.4	Drama	110000	9600
MV114	Wild Trails	7.2	Adventure	99000	7200
MV115	The Great Escape	9.0	Action	205000	18800

Figure 1.1 Original dataset printed after creating the DataFrame (containing missing values).

The average rating was calculated successfully and was used to replace all missing Rating values.

2. FILL MISSING RATING USING AVERAGE RATING

Average Rating used: 8.28

MovieID	MovieTitle	Rating
MV107	Future Code	9
MV115	The Great Escape	9
MV105	Digital Dreams	9
MV111	Ocean Mystery	9
MV112	Final Mission	8
MV101	The Last Horizon	8
MV113	Parallel Lives	8
MV103	Hidden Truth	8
MV109	Midnight Journey	8
MV110	Laugh Again	8
MV102	City Lights	8
MV106	Silent River	8
MV114	Wild Trails	7

Figure 1.2 Output showing the calculated average rating.

Student Reg. No: 26MML0031 Name: Makada Hasnain H

Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT), Vellore

2025 WINTER SEMESTER
Programming for Data Science Lab (MACSE502)
Record Exercise Program

Date: 16/07/2026

Week No.: 2

Program S. No. 4

3. REMOVE ROWS WITH MISSING MOVIEID OR GENRE

MovieID	MovieTitle	Genre	Rating
MV107	Future Code	Science Fiction	9
MV115	The Great Escape	Action	9
MV105	Digital Dreams	Science Fiction	9
MV111	Ocean Mystery	Mystery	9
MV112	Final Mission	Action	8
MV101	The Last Horizon	Science Fiction	8
MV113	Parallel Lives	Drama	8
MV103	Hidden Truth	Thriller	8
MV109	Midnight Journey	Adventure	8
MV110	Laugh Again	Comedy	8
MV102	City Lights	Comedy	8
MV106	Silent River	Drama	8
MV114	Wild Trails	Adventure	7

Figure 1.3 Dataset after fillna() operation.

Missing Rating values were replaced with the calculated average rating while preserving all other records.

Rating values were converted into integer type and normalized using the Min-Max Normalization technique.

5. NORMALIZE RATING VALUES

MovieID	MovieTitle	Rating	NormalizedRating
MV107	Future Code	9	1.0
MV115	The Great Escape	9	1.0
MV105	Digital Dreams	9	1.0
MV111	Ocean Mystery	9	1.0
MV112	Final Mission	8	0.5
MV101	The Last Horizon	8	0.5
MV113	Parallel Lives	8	0.5
MV103	Hidden Truth	8	0.5
MV109	Midnight Journey	8	0.5
MV110	Laugh Again	8	0.5
MV102	City Lights	8	0.5
MV106	Silent River	8	0.5
MV114	Wild Trails	7	0.0

Figure 1.4 Integer Rating output along with the **NormalizedRating** column.

TOP-RATED AND POPULAR MOVIES

PopularityRank	MovieID	MovieTitle	Genre	Rating	ViewCount	PopularityScore
1	MV107	Future Code	Science Fiction	9	210000	1.000
2	MV115	The Great Escape	Action	9	205000	0.989
3	MV105	Digital Dreams	Science Fiction	9	175000	0.988
4	MV111	Ocean Mystery	Mystery	9	143000	0.959
5	MV112	Final Mission	Action	8	190000	0.689
6	MV101	The Last Horizon	Science Fiction	8	125000	0.680
7	MV113	Parallel Lives	Drama	8	118000	0.645
8	MV103	Hidden Truth	Thriller	8	110000	0.643
9	MV109	Midnight Journey	Adventure	8	132000	0.634
10	MV110	Laugh Again	Comedy	8	105000	0.627
11	MV102	City Lights	Comedy	8	98000	0.624
12	MV106	Silent River	Drama	8	87000	0.597
13	MV114	Wild Trails	Adventure	7	99000	0.308

Figure 1.5 Sorted movie list showing **Popularity Rank**.

Movies were ranked successfully based on their popularity score, allowing easy identification of top-performing movies.

Conclusions

The Movie Rating Data Preprocessing project was successfully implemented using the Pandas library. The dataset was created with missing values, and the missing Rating values were replaced using the average rating. Records with missing MovieID or Genre were removed to improve data quality. The Rating column was converted into integer type, and the values were normalized using the Min-Max normalization technique. The project also identified top-rated movies, generated genre-wise reports, analysed viewer preferences, recommended

Student Reg. No: 26MML0031 Name: Makada Hasnain H

Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT), Vellore

2026 WINTER SEMESTER
Programming for Data Science Lab (MACSE502)
Record Exercise Program

Date: 16/07/2026

Week No.: 2

Program S. No. 4

	movies based on genre and rating, and visualized movie popularity trends. This exercise provided practical knowledge of data preprocessing, data analysis, and dashboard development using Python, Pandas, and Streamlit.
References	1. Michael Freeman and Joel Ross, Programming Skills for Data Science: Start Writing Code to Wrangle, Analyze, and Visualize Data with R, Addison-Wesley, 2018 2. Python Program (https://github.com/hasnainmakada-99/Data-Science-Codes/blob/main/WEEK%202/EXERCISE_PROGRAM.py) 3. Class lecture link (https://drive.google.com/file/d/15diEukCtrPLimox63oa9ce1U-t4qVMUy/view)

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Practice Program

Date: 23/07/2026

Week No.: 3

Program S. No. 5

Title	Creating and Manipulating Arrays Using NumPy				
Problem Statement	Create a 2D array of shape (3, 4) with random integers between 0 and 10. Perform the following operations: calculate the mean and standard deviation of the entire array, slice the array to get the first two rows and last two columns, reshape the array to shape (4, 3), and perform element-wise multiplication with another array of the same shape				
Objectives	• To create and manipulate two-dimensional NumPy arrays. • To represent predefined array values using a Python dictionary. • To calculate the mean and standard deviation of the entire array. • To slice the array and extract selected rows and columns. • To reshape an array from shape (3, 4) to shape (4, 3). • To perform element-wise multiplication between arrays of the same shape. • To understand array shape, slicing, reshaping and numerical operations using NumPy.				
Dataset URL Link / File	Predefined dictionary dataset used directly inside the Python program:				
	Array	Row 1	Row 2	Row 3	Row 4
	Array1 (3 x 4)	2, 5, 7, 1	8, 3, 6, 4	9, 0, 5, 2	—
	Array2 (4 x 3)	1, 2, 3	4, 5, 6	7, 8, 9	2, 3, 4
	No external dataset file or URL is required.				
Functions Details	Sl. No.	Function	Package	Description of function	
	1	np.array()	NumPy	Converts dictionary values into NumPy arrays.	
	2	shape	NumPy	Returns the dimensions of an array as rows and columns.	
	3	np.mean()	NumPy	Calculates the mean of all elements in the array.	
	4	np.std()	NumPy	Calculates the standard deviation of all array elements.	
	5	Array slicing	NumPy	Extracts the first two rows and last two columns.	
	6	reshape()	NumPy	Changes the array shape from (3, 4) to (4, 3).	
	7	round()	Python	Rounds numerical output to the specified decimal places.	
	8	print()	Python	Displays arrays, calculations and headings in the console.	
	9	range()	Python	Creates a sequence of integer values when required.	
	10	len()	Python	Returns the number of items in an object.	
	11	dtype	NumPy	Shows the data type of array elements.	

Student Reg. No: 26MML0031 Name: Makada Hasnain

Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT), Vellore

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Practice Program

Date: 23/07/2026

Week No.: 3

Program S. No. 5

	12	Element-wise *	NumPy	Multiplies corresponding elements of equal-shaped arrays.
Steps/ Process/ Procedure/ Algorithm/ Flowchart/ Model	Step 1: Import NumPy using import numpy as np. Step 2: Create a predefined dictionary containing Array1 and Array2. Step 3: Convert the dictionary values into NumPy arrays using np.array(). Step 4: Display Array1 and verify its shape as (3, 4). Step 5: Calculate the mean using np.mean(). Step 6: Calculate the standard deviation using np.std(). Step 7: Slice Array1 using array1[:2, -2:] to get the first two rows and last two columns. Step 8: Reshape Array1 to (4, 3) using reshape(4, 3). Step 9: Display Array2 and verify its shape as (4, 3). Step 10: Perform element-wise multiplication using reshaped_array * array2. Step 11: Display all results and their respective shapes. Step 12: Verify that all requested NumPy operations are completed successfully.			
Program	<pre>import numpy as np array_data = { "Array1": [[2, 5, 7, 1], [8, 3, 6, 4], [9, 0, 5, 2]], "Array2": [[1, 2, 3], [4, 5, 6], [7, 8, 9], [2, 3, 4]] } array1 = np.array(array_data["Array1"]) array2 = np.array(array_data["Array2"]) print("ORIGINAL 2D\nARRAY") print(array1) print("Shape:", array1.shape) mean_value = np.mean(array1) standard_deviation = np.std(array1) print("\nMean:", round(mean_value, 2)) print("Standard Deviation:", round(standard_deviation, 2)) sliced_array = array1[:2, -2:] print("\nFIRST TWO ROWS AND LAST TWO COLUMNS") print(sliced_array) print("Shape:", sliced_array.shape) reshaped_array = array1.reshape(4, 3) print("\nRESHAPED ARRAY")</pre>			

Student Reg. No: 26MML0031 — Name: Makada Hasnain

Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT), Vellore

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Practice Program

Date: 23/07/2026

Week No.: 3

Program S. No. 5

```
print(reshaped_array)
print("Shape:", reshaped_array.shape)

print("\nSECOND ARRAY")
print(array2)

multiplied_array = reshaped_array * array2
print("\nELEMENT-WISE
MULTIPLICATION")
print(multiplied_array)
print("Shape:", multiplied_array.shape)

print("\nFINAL SUMMARY")
print("Original Shape:",
array1.shape) print("Mean:",
round(mean_value, 2))
print("Standard Deviation:", round(standard_deviation, 2))
print("Sliced Shape:", sliced_array.shape)
print("Reshaped Shape:", reshaped_array.shape)
print("Multiplied Shape:", multiplied_array.shape)
```

Screenshot 1: Original Array and Shape

ORIGINAL 2D ARRAY

```
=====
[[2 5 7 1]
 [8 3 6 4]
 [9 0 5 2]]
```

Shape of Array: (3, 4)

Figure 1.1 Original 2D Array

The predefined dictionary values were converted into a NumPy array. The output confirms that the array contains 3 rows and 4 columns.

Screenshot 2: Mean and Standard Deviation

MEAN OF THE ENTIRE ARRAY

```
=====
Mean: 4.33
```

STANDARD DEVIATION OF THE ENTIRE ARRAY

```
=====
Standard Deviation: 2.72
```

Figure 1.2 Mean & Standard Deviation of the Array

The mean represents the average of all elements in the array, while the standard deviation shows how widely the values are distributed around the mean.

**Output
Snapshots
With
Explanation/
Observations**

Student Reg. No: 26MML0031 Name: Makada Hasnain

Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT), Vellore

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Practice Program

Date: 23/07/2026

Week No.: 3

Program S. No.

5 Screenshot 3: Array Slicing

```
FIRST TWO ROWS AND LAST TWO COLUMNS
```

```
=====
[[7 1]
 [6 4]]
```

Figure 1.3 Sliced Array

Array slicing was performed using `array1[:2, -2:]`. The resulting sliced array has a shape of 2×2 .

Screenshot 4: Reshaped Array

```
RESHAPED ARRAY
```

```
=====
[[2 5 7]
 [1 8 3]
 [6 4 9]
 [0 5 2]]
```

```
Shape After Reshaping: (4, 3)
```

Figure 1.4 Reshaped Array

The `reshape()` function changed the arrangement of the 12 elements without changing their values or total count.

Screenshot 5: Element-wise Multiplication

```
ELEMENT-WISE MULTIPLICATION
```

```
=====
[[ 2 10 21]
 [ 4 40 18]
 [42 32 81]
 [ 0 15  8]]
```

Figure 1.5 Element Wise Multiplication

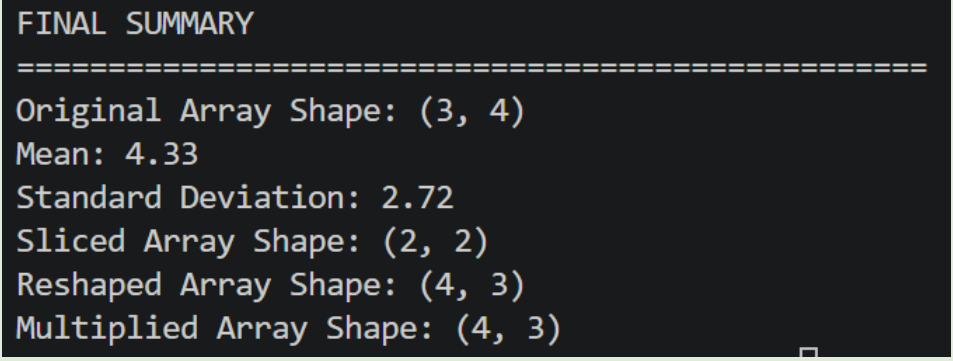
Corresponding elements of the two arrays with shape (4, 3) were multiplied using the `*` operator.

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Practice Program

Date: 23/07/2026

Week No.: 3

Program S. No. 5

	Screenshot 6: Final Summary  <i>Figure 1.6 Final Summary</i> The final output verifies that all statistical, slicing, reshaping and multiplication operations were completed successfully.
Conclusions	The NumPy array manipulation exercise was completed successfully. A two-dimensional array of shape (3, 4) was created using predefined dictionary data. The mean and standard deviation were calculated, the first two rows and last two columns were extracted through slicing, and the array was reshaped to (4, 3). Element-wise multiplication was performed with another array of the same shape. The exercise provided practical understanding of NumPy array creation, statistical calculations, slicing, reshaping and element-wise operations.
References	1. Michael Freeman and Joel Ross, Programming Skills for Data Science: Start Writing Code to Wrangle, Analyze, and Visualize Data with R, Addison-Wesley, 2018. 2. NumPy documentation for array creation, statistics, slicing, reshaping and mathematical operations. 3. Python program link (https://github.com/hasnainmakada-99/Data-Science-Codes/blob/main/WEEK%203/PRACTICE_PROGRAM.py) 4. Programming for Data Science (MACSE502) class lecture notes.

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Exercise Program

Date: 23/07/2026

Week No.: 3

Program S. No.: 6

Title	Warehouse Shipment and Logistics Analytics Using NumPy				
Problem Statement	A logistics company tracks shipments from 4 warehouses across 7 weeks. Create a 4 x 7 shipment matrix. Perform the following operations: (1) compute average shipments, (2) compute standard deviation, (3) slice the first two warehouses and last three weeks, (4) reshape the matrix into a 7 x 4 matrix, and (5) perform element-wise multiplication with a transportation-cost matrix of the same shape. As an extra practice mini project, develop a Supply Chain Analytics Dashboard that tracks inventory movement, optimizes transportation costs, identifies logistics bottlenecks, generates warehouse performance reports, and supports supply-chain planning.				
Objectives	• To create a 4 x 7 warehouse shipment matrix using NumPy. • To store shipment, transportation-cost, and capacity data locally using predefined Python dictionaries. • To calculate the mean and standard deviation of the complete shipment matrix. • To extract the first two warehouses and last three weeks using NumPy slicing. • To reshape the shipment matrix from 4 x 7 into 7 x 4 without changing its elements. • To calculate warehouse-week transportation expenditure using element-wise multiplication. • To track warehouse totals, weekly inventory movement, transportation cost, and capacity utilization. • To identify high-performing warehouses, cost-efficient warehouses, peak weeks, and logistics bottlenecks. • To develop an additional Streamlit dashboard for warehouse reporting and supply-chain planning. • Learning Outcome: Students understand numerical analytics in logistics management.				
Dataset URL Link / File	The dataset is stored locally inside the console and dashboard programs using predefined dictionaries. No external CSV file, URL, or database is required. A. Shipment Matrix (units shipped)				
		Warehouse 1	120	135	128
		Warehouse 2	98	105	110
		Warehouse 3	140	145	150
		Warehouse 4	80	85	90

Student Reg. No.: 26MML0031

Name: Makada Hasnain Husainbhai

Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT), Vellore

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Exercise Program

Date: 23/07/2026

Week No.: 3

Program S. No.: 6

B. Transportation Cost Matrix (cost per shipment unit)

Warehouse 1	12	12	13	13	14	14	15
Warehouse 2	10	11	11	12	12	13	13
Warehouse 3	14	14	15	15	16	16	17
Warehouse 4	9	9	10	10	11	11	12

C. Weekly Capacity Dictionary

Warehouse	Weekly Capacity
Warehouse 1	175
Warehouse 2	145
Warehouse 3	180
Warehouse 4	120

Functions Details

Sl. No.	Function	Package	Description of function
1	np.array()	NumPy	Converts dictionary values into two-dimensional NumPy matrices.
2	np.mean()	NumPy	Calculates average shipments for the full matrix or a selected axis.
3	np.std()	NumPy	Measures the variation of shipment values around the mean.
4	Array slicing	NumPy	Extracts the first two warehouses and last three weeks.
5	reshape()	NumPy	Reorganizes the 4 x 7 matrix into a 7 x 4 matrix.
6	np.multiply()	NumPy	Performs element-wise multiplication of shipments and unit costs.
7	np.sum()	NumPy	Calculates warehouse totals, weekly totals, and total transportation cost.
8	np.argmax()	NumPy	Returns the index of the maximum shipment or utilization value.

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Exercise Program

Date: 23/07/2026

Week No.: 3

Program S. No.: 6

	9	np.where()	NumPy	Identifies warehouse-week combinations that cross the bottleneck threshold.
	10	pd.DataFrame()	Pandas	Creates labeled warehouse and weekly reports for dashboard display.
	11	st.metric()	Streamlit	Displays important supply-chain key performance indicators.
	12	st.line_chart()/ bar_chart()	Streamlit	Visualizes inventory movement, cost, utilization, and warehouse performance.
Steps/ Process/ Procedure/ Algorithm/ Flowchart/ Model	Step 1: Import NumPy for numerical operations. Step 2: Create local dictionaries containing warehouse names, week names, shipment data, transportation costs, and weekly capacities. Step 3: Convert dictionary values into NumPy arrays. Step 4: Display the original 4 x 7 shipment and transportation-cost matrices. Step 5: Calculate the mean of the entire shipment matrix using np.mean(). Step 6: Calculate the standard deviation using np.std(). Step 7: Slice shipments[:, -3:] to obtain the first two warehouses and last three weeks. Step 8: Reshape the 4 x 7 shipment matrix into a 7 x 4 matrix. Step 9: Use np.multiply() to multiply each shipment value by its corresponding transportation cost. Step 10: Calculate warehouse-wise shipments, week-wise shipments, total costs, average costs, and capacity utilization. Step 11: Identify the highest-performing warehouse and peak shipment week using np.argmax(). Step 12: Identify logistics bottlenecks where utilization is at least 90 percent of weekly capacity. Step 13: Generate a warehouse performance report for console output. Step 14: Create a Streamlit dashboard for inventory movement, transportation-cost analysis, bottleneck reporting, and planning. Step 15: Execute the console program and dashboard, then capture each required output screenshot.			
Program	A. Console Program <pre>import numpy as np # Local dataset stored using predefined dictionaries shipment_data = { "Warehouse 1": [120, 135, 128, 142, 150, 160, 155], "Warehouse 2": [98, 105, 110, 115, 120, 118, 125], "Warehouse 3": [140, 145, 150, 155, 165, 170, 175], "Warehouse 4": [80, 85, 90, 95, 100, 105, 110] } transport_cost_data = { "Warehouse 1": [12, 12, 13, 13, 14, 14, 15], "Warehouse 2": [10, 11, 11, 12, 12, 13, 13],</pre>			

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Exercise Program

Date: 23/07/2026

Week No.: 3

Program S. No.: 6

```
"Warehouse 3": [14, 14, 15, 15, 16, 16, 17],
"Warehouse 4": [9, 9, 10, 10, 11, 11, 12]
}

capacity_data =
{ "Warehouse 1":
  175,
  "Warehouse 2": 145,
  "Warehouse 3": 180,
  "Warehouse 4": 120
}

warehouses = np.array(list(shipment_data.keys()))
weeks = np.array(["Week 1", "Week 2", "Week 3", "Week 4",
                  "Week 5", "Week 6", "Week 7"])
shipments = np.array(list(shipment_data.values()))
transport_cost =
np.array(list(transport_cost_data.values())) capacity =
np.array(list(capacity_data.values()))

print("WAREHOUSE SHIPMENT MATRIX")
print(shipments)
print("Shape:", shipments.shape)

print("\nTRANSPORTATION COST MATRIX")
print(transport_cost)
print("Shape:", transport_cost.shape)

average_shipments = np.mean(shipments)
shipment_std = np.std(shipments)

print("\nAVERAGE AND STANDARD DEVIATION")
print("Average shipments:", round(average_shipments, 2))
print("Standard deviation:", round(shipment_std, 2))

sliced_shipments = shipments[:2, -3:]
print("\nFIRST TWO WAREHOUSES AND LAST THREE WEEKS")
print(sliced_shipments)
print("Sliced shape:", sliced_shipments.shape)

reshaped_shipments = shipments.reshape(7,
4) print("\nRESHAPED SHIPMENT
MATRIX")
print(reshaped_shipments)
print("Reshaped shape:", reshaped_shipments.shape)
transportation_expense = np.multiply(shipments, transport_cost)
print("\nELEMENT-WISE TRANSPORTATION EXPENSE")
print(transportation_expense)
warehouse_totals = np.sum(shipments, axis=1)
```

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Exercise Program

Date: 23/07/2026

Week No.: 3

Program S. No.: 6

```
weekly_totals = np.sum(shipments, axis=0)
warehouse_costs = np.sum(transportation_expense, axis=1)
average_unit_cost = warehouse_costs / warehouse_totals
utilization = shipments / capacity[:, None] * 100
maximum_utilization = np.max(utilization, axis=1)

best_warehouse_index = np.argmax(warehouse_totals)
peak_week_index = np.argmax(weekly_totals)
efficient_warehouse_index = np.argmin(average_unit_cost)
bottleneck_index = np.argmax(maximum_utilization)
bottleneck_points = np.where(utilization >= 90)

print("\nWAREHOUSE PERFORMANCE REPORT")
for i in range(len(warehouses)):
    print(warehouses[i])
    print(" Total shipments:", warehouse_totals[i])
    print(" Average weekly shipments:", round(np.mean(shipments[i]), 2))
    print(" Transportation expense:", warehouse_costs[i])
    print(" Average cost per shipment:", round(average_unit_cost[i], 2))
    print(" Maximum capacity utilization:",
          round(maximum_utilization[i], 2), "%")

print("\nWEEKLY INVENTORY MOVEMENT")
for i in range(len(weeks)):
    print(weeks[i], ":", weekly_totals[i])

print("\nLOGISTICS BOTTLENECK POINTS")
for warehouse_i, week_i in zip(*bottleneck_points):
    print(warehouses[warehouse_i], weeks[week_i],
          "Utilization:", round(utilization[warehouse_i, week_i], 2), "%")

print("\nSUPPLY CHAIN PLANNING SUMMARY")
print("Highest-performing warehouse:",
      warehouses[best_warehouse_index])
print("Peak shipment week:", weeks[peak_week_index])
print("Most cost-efficient warehouse:",
      warehouses[efficient_warehouse_index])
print("Highest bottleneck risk:", warehouses[bottleneck_index])
print("Total shipments:", np.sum(shipments))
print("Total transportation expense:", np.sum(transportation_expense))

B. Extra Practice: Streamlit Supply Chain Analytics Dashboard
import numpy as np
import pandas as pd
import streamlit as st

st.set_page_config(
    page_title="Supply Chain Analytics Dashboard",
    layout="wide"
```

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Exercise Program

Date: 23/07/2026

Week No.: 3

Program S. No.: 6

```
)

st.title("Warehouse Shipment and Supply Chain Analytics")

# The dashboard uses the same local dictionary dataset
shipment_data = {
    "Warehouse 1": [120, 135, 128, 142, 150, 160, 155],
    "Warehouse 2": [98, 105, 110, 115, 120, 118, 125],
    "Warehouse 3": [140, 145, 150, 155, 165, 170, 175],
    "Warehouse 4": [80, 85, 90, 95, 100, 105, 110]
}

transport_cost_data = {
    "Warehouse 1": [12, 12, 13, 13, 14, 14, 15],
    "Warehouse 2": [10, 11, 11, 12, 12, 13, 13],
    "Warehouse 3": [14, 14, 15, 15, 16, 16, 17],
    "Warehouse 4": [9, 9, 10, 10, 11, 11, 12]
}

capacity_data =
{ "Warehouse 1":
175,
"Warehouse 2": 145,
"Warehouse 3": 180,
"Warehouse 4": 120
}

weeks = ["Week 1", "Week 2", "Week 3", "Week 4",
        "Week 5", "Week 6", "Week 7"]

shipment_df = pd.DataFrame.from_dict(
    shipment_data, orient="index", columns=weeks
)
cost_rate_df = pd.DataFrame.from_dict(
    transport_cost_data, orient="index", columns=weeks
)
expense_df = shipment_df * cost_rate_df
capacity_series = pd.Series(capacity_data)
utilization_df = shipment_df.div(capacity_series, axis=0) * 100

warehouse_totals = shipment_df.sum(axis=1)
weekly_totals = shipment_df.sum(axis=0)
warehouse_expense = expense_df.sum(axis=1)
average_cost = warehouse_expense / warehouse_totals
maximum_utilization = utilization_df.max(axis=1)

performance_report = pd.DataFrame({
    "Total Shipments": warehouse_totals,
    "Average Weekly Shipments": shipment_df.mean(axis=1).round(2),
```

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Exercise Program

Date: 23/07/2026

Week No.: 3

Program S. No.: 6

```
"Transportation Expense": warehouse_expense,
"Average Cost per Shipment": average_cost.round(2),
"Maximum Utilization (%)": maximum_utilization.round(2)
})

best_warehouse = warehouse_totals.idxmax()
peak_week = weekly_totals.idxmax()
cost_efficient = average_cost.idxmin()
bottleneck = maximum_utilization.idxmax()

selected_warehouse = st.sidebar.selectbox(
    "Select warehouse", shipment_df.index
)
threshold = st.sidebar.slider(
    "Bottleneck threshold (%)", 70, 100, 90
)

c1, c2, c3, c4 = st.columns(4)
c1.metric("Total Shipments", f"{int(shipment_df.values.sum());}")
c2.metric("Top Warehouse", best_warehouse)
c3.metric("Peak Week", peak_week)
c4.metric("Bottleneck Risk", bottleneck)

movement_tab, cost_tab, bottleneck_tab, report_tab, planning_tab
= st.tabs([
    "Inventory Movement", "Transportation
    Costs", "Logistics Bottlenecks", "Warehouse
    Reports", "Supply Chain Planning"
])

with movement_tab:
    st.subheader("Weekly Inventory Movement")
    st.line_chart(shipment_df.T)
    st.bar_chart(weekly_totals)
    st.dataframe(shipment_df, use_container_width=True)

with cost_tab:
    st.subheader("Transportation Expense by Warehouse")
    st.bar_chart(warehouse_expense)
    st.write("Most cost-efficient warehouse:", cost_efficient)
    st.dataframe(expense_df, use_container_width=True)

with bottleneck_tab:
    st.subheader("Capacity
    Utilization")
    st.line_chart(utilization_df.T)
    bottleneck_table = utilization_df[utilization_df >= threshold]
    st.dataframe(bottleneck_table, use_container_width=True)
    if bottleneck_table.notna().any().any():
```

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Exercise Program

Date: 23/07/2026

Week No.: 3

Program S. No.: 6

```
st.warning("High-utilization warehouse-week combinations
detected.")
else:
    st.success("No bottlenecks detected at the selected threshold.")

with report_tab:
    st.subheader("Warehouse Performance Report")
    st.dataframe(performance_report, use_container_width=True)
    st.bar_chart(performance_report[["Total Shipments"]])

with planning_tab:
    st.subheader("Supply Chain Planning Summary")
    st.write("Selected warehouse:", selected_warehouse)
    st.write("Highest-performing warehouse:",
best_warehouse) st.write("Peak shipment week:",
peak_week) st.write("Most cost-efficient warehouse:",
cost_efficient) st.write("Highest bottleneck risk:",
bottleneck)
st.write(
    "Planning action: Review vehicle allocation, dispatch frequency, "
    "capacity, and route cost for high-utilization warehouses."
)
```

Run using: python -m streamlit run dashboard.py

**Output
Snapshots
With
Explanation/
Observations**

Screenshot 1: Original Shipment and Transportation-Cost Matrices

```
WAREHOUSE SHIPMENT MATRIX
[[120 135 128 142 150 160 155]
 [ 98 105 110 115 120 118 125]
 [140 145 150 155 165 170 175]
 [ 80  85  90  95 100 105 110]]
Shape: (4, 7)
```

```
TRANSPORTATION COST MATRIX
[[12 12 13 13 14 14 15]
 [10 11 11 12 12 13 13]
 [14 14 15 15 16 16 17]
 [ 9  9 10 10 11 11 12]]
Shape: (4, 7)
```

Figure 1.1 Original Shipment & Transportation Cost Matrix

Explanation/Observation: The local dictionary data was converted into two NumPy matrices of shape (4, 7), representing four warehouses across seven weeks.

Screenshot 2: Average Shipment and Standard Deviation

```
AVERAGE AND STANDARD DEVIATION  
Average shipments: 126.64  
Standard deviation: 26.66
```

Figure 1.2 Average & Standard Deviation

Explanation/Observation: The mean represents the average shipment volume across all warehouse-week observations, while the standard deviation measures shipment variation.

Screenshot 3: Sliced Shipment Matrix

```
FIRST TWO WAREHOUSES AND LAST THREE WEEKS  
[[150 160 155]  
 [120 118 125]]  
Sliced shape: (2, 3)
```

Figure 1.3 Sliced Shipment Matrix

Explanation/Observation: The expression `shipments[:2, -3:]` selected the first two warehouses and the last three weeks, producing a matrix of shape (2, 3).

Screenshot 4: Reshaped 7 x 4 Shipment Matrix

```
RESHAPED SHIPMENT MATRIX  
[[120 135 128 142]  
 [150 160 155 98]  
 [105 110 115 120]  
 [118 125 140 145]  
 [150 155 165 170]  
 [175 80 85 90]  
 [95 100 105 110]]  
Reshaped shape: (7, 4)
```

Figure 1.4 Reshaped Matrix

Explanation/Observation: The reshape operation reorganized all 28 shipment values into seven rows and four columns without adding or removing data.

Screenshot 5: Element-wise Transportation Expense Matrix

```
ELEMENT-WISE TRANSPORTATION EXPENSE
[[1440 1620 1664 1846 2100 2240 2325]
 [ 980 1155 1210 1380 1440 1534 1625]
 [1960 2030 2250 2325 2640 2720 2975]
 [ 720  765  900  950 1100 1155 1320]]
```

Figure 1.5 Element Wise Matrix

Explanation/Observation: Each shipment quantity was multiplied by its corresponding transportation cost to estimate warehouse-week transportation expenditure.

Screenshot 6: Warehouse Performance and Bottleneck Report

```
WAREHOUSE PERFORMANCE REPORT
Warehouse 1
  Total shipments: 990
  Average weekly shipments: 141.43
  Transportation expense: 13235
  Average cost per shipment: 13.37
  Maximum capacity utilization: 91.43 %
Warehouse 2
  Total shipments: 791
  Average weekly shipments: 113.0
  Transportation expense: 9324
  Average cost per shipment: 11.79
  Maximum capacity utilization: 86.21 %
Warehouse 3
  Total shipments: 1100
  Average weekly shipments: 157.14
  Transportation expense: 16900
  Average cost per shipment: 15.36
  Maximum capacity utilization: 97.22 %
Warehouse 4
  Total shipments: 665
  Average weekly shipments: 95.0
  Transportation expense: 6910
  Average cost per shipment: 10.39
  Maximum capacity utilization: 91.67 %
```

Figure 1.6 Warehouse Performance Report

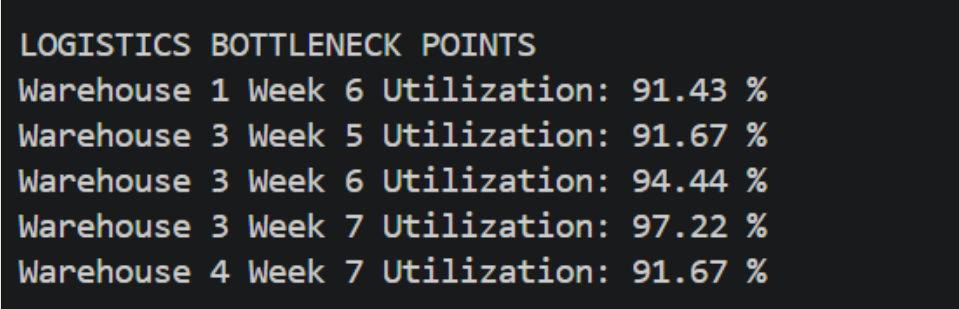
Explanation/Observation: The console report compared total shipments, average shipments, transportation expense, cost efficiency, and maximum capacity utilization for every warehouse.

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Exercise Program

Date: 23/07/2026

Week No.: 3

Program S. No.: 6

	Screenshot 7: Cost, Bottleneck, and Planning Views  <i>Figure 1.7 Logistics Bottleneck Points</i> Explanation/Observation: We visualized transportation expenses and capacity utilization, identified high-utilization periods, and generated planning recommendations.
Conclusions	The Warehouse Shipment and Logistics Analytics exercise was completed successfully using NumPy. Shipment, transportation-cost, and capacity data were stored locally using predefined dictionaries and converted into structured arrays. The mean and standard deviation of the shipment matrix were calculated, the first two warehouses and last three weeks were extracted through slicing, and the matrix was reshaped from 4 x 7 to 7 x 4. Element-wise multiplication was used to calculate transportation expenditure. Additional warehouse-wise and week-wise calculations identified the highest-performing warehouse, peak shipment week, most cost-efficient warehouse, and high-utilization bottlenecks.
References	1. NumPy documentation for array creation, descriptive statistics, slicing, reshaping, aggregation, and element-wise operations. 2. Pandas documentation for DataFrame creation and warehouse performance reporting. 3. Python Program Link https://github.com/hasnainmakada-99/Data-Science-Codes/blob/main/WEEK%203/EXERCISE_PROGRAM.py

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Practice Program

Date: 30/07/2026

Week No.: 4

Program S. No. 7

Title	Feature Engineering and Correlation Analysis Using Pandas			
Problem Statement	Create a DataFrame with columns Feature1, Feature2, and Target containing predefined sample data stored in a Python dictionary. Create a new feature that is the logarithm of Feature1, bin Feature2 into three categories (Low, Medium, and High), and calculate the correlation matrix of the DataFrame. Finally, create a scatter plot of Feature1 versus Feature2 distinguished by Target and interpret the visible patterns.			
Objectives	• To create a Pandas DataFrame using a predefined Python dictionary. • To understand feature engineering using logarithmic transformation. • To transform Feature1 using the natural logarithm function. • To divide Feature2 into Low, Medium, and High categories. • To calculate and interpret correlations among numerical variables. • To visualize the relationship between Feature1 and Feature2 using a scatter plot. • To distinguish Target classes using a black-and-white plotting format. • To interpret visible trends, class overlap, and feature relationships.			
Dataset URL Link / File	Predefined dictionary dataset used directly inside the Python program:			
	Feature1	Feature2	Target	
	5	10	0	
	8	18	0	
	12	15	1	
	15	25	0	
	18	30	1	
	22	28	1	
	25	35	0	
	30	40	1	
	35	38	1	
	40	45	0	
	45	50	1	
	50	55	1	
	55	60	0	
	60	65	1	
	70	75	1	
No external CSV file, URL, database, or online dataset is required.				
Functions Details	Sl. No.	Function	Package	Description of function
	1	pd.DataFrame()	Pandas	Creates a DataFrame from the predefined dictionary data.
	2	np.log()	NumPy	Calculates the natural logarithm of each Feature1 value.
	3	pd.cut()	Pandas	Bins Feature2 into Low, Medium, and High categories.

Student Reg. No: 26MML0031 — Name: Makada Hasnain

Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT), Vellore

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Practice Program

Date: 30/07/2026

Week No.: 4

Program S. No. 7

	4	corr()	Pandas	Calculates pairwise correlations between numerical columns.
	5	value_counts()	Pandas	Counts the number of records in each Feature2 category.
	6	sort_index()	Pandas	Displays category counts in their defined category order.
	7	round()	Python/ Pandas	Rounds logarithmic and correlation values for readable output.
	8	plt.figure()	Matplotlib	Creates a plotting area with the specified dimensions.
	9	plt.scatter()	Matplotlib	Creates the Feature1 versus Feature2 scatter plot.
	10	plt.xlabel()/plt.ylabel()	Matplotlib	Adds descriptive labels to the plot axes.
	11	plt.legend()	Matplotlib	Identifies the symbols used for each Target class.
	12	plt.show()	Matplotlib	Displays the completed scatter plot on the screen.
Steps/ Process/ Procedure/ Algorithm/ Flowchart/ Model	Step 1: Import Pandas, NumPy, and Matplotlib. Step 2: Define Feature1, Feature2, and Target values inside a Python dictionary. Step 3: Convert the dictionary into a DataFrame using pd.DataFrame(). Step 4: Display the original DataFrame and confirm the available columns. Step 5: Create Log_Feature1 by applying np.log() to Feature1. Step 6: Bin Feature2 into three equal-width categories using pd.cut(). Step 7: Label the categories as Low, Medium, and High. Step 8: Select the numerical columns required for correlation analysis. Step 9: Calculate and display the correlation matrix using corr(). Step 10: Count the number of observations in each Feature2 category. Step 11: Create a black-and-white scatter plot of Feature1 versus Feature2 grouped by Target. Step 12: Interpret the correlation, trend, and overlap visible in the output.			
Program	<pre>import pandas as pd import numpy as np import matplotlib.pyplot as plt # Predefined dictionary dataset data = { "Feature1": [5, 8, 12, 15, 18, 22, 25, 30, 35, 40, 45, 50, 55, 60, 70], "Feature2": [10, 18, 15, 25, 30, 28, 35, 40, 38, 45, 50, 55, 60, 65, 75], "Target": [0, 0, 1, 0, 1, 1, 0, 1, 1, 0, 1, 1, 0, 1, 1] }</pre>			

Student Reg. No: 26MML0031 Name: Makada Hasnain

Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT), Vellore

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Practice Program

Date: 30/07/2026

Week No.: 4

Program S. No. 7

```
# Convert dictionary into a DataFrame
df = pd.DataFrame(data)
print("ORIGINAL DATAFRAME")
print(df)
print("DataFrame Shape:", df.shape)

# Create logarithmic feature
df["Log_Feature1"] = np.log(df["Feature1"])
print("\nDATAFRAME WITH LOG FEATURE")
print(df[["Feature1", "Log_Feature1"]].round(3))

# Bin Feature2 into three categories
df["Feature2_Category"] = pd.cut(
    df["Feature2"],
    bins=3,
    labels=["Low", "Medium", "High"]
)
print("\nFEATURE2 CATEGORIES")
print(df[["Feature2", "Feature2_Category"]])

# Calculate correlation matrix
numeric_columns = [
    "Feature1", "Feature2", "Target", "Log_Feature1"
]
correlation_matrix = df[numeric_columns].corr()
print("\nCORRELATION MATRIX")
print(correlation_matrix.round(3))

# Display category-wise
counts category_count = (
    df["Feature2_Category"]
    .value_counts()
    .sort_index()
)
print("\nFEATURE2 CATEGORY COUNT")
print(category_count)

# Create black-and-white scatter plot
class_zero = df["Target"] == 0
class_one = df["Target"] == 1

plt.figure(figsize=(8, 6))
plt.scatter(
    df.loc[class_zero, "Feature1"],
    df.loc[class_zero, "Feature2"],
    facecolors="white",
    edgecolors="black",
    marker="o",
```

Student Reg. No: 26MML0031 Name: Makada Hasnain

Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT), Vellore

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Practice Program

Date: 30/07/2026

Week No.: 4

Program S. No. 7

```
label="Target = 0"
)
plt.scatter(
    df.loc[class_one, "Feature1"],
    df.loc[class_one, "Feature2"],
    color="black",
    marker="s",
    label="Target =
1"
)
plt.title("Feature1 vs. Feature2 Grouped by Target")
plt.xlabel("Feature1")
plt.ylabel("Feature2")
plt.grid(True, linestyle="--", alpha=0.5)
plt.legend()
plt.tight_layout()
plt.show()

# Interpret visible patterns
feature_correlation = df["Feature1"].corr(df["Feature2"]) print("\n
PATTERN INTERPRETATION")
print(
    "Correlation between Feature1 and Feature2:",
    round(feature_correlation, 3)
)
print(
    "Feature1 and Feature2 have a strong positive "
    "relationship. Target classes overlap, although "
    "Target = 1 appears more often at medium and high values."
)
```

Student Reg. No: 26MML0031 Name: Makada Hasnain

Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT), Vellore

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Practice Program

Date: 30/07/2026

Week No.: 4

Program S. No. 7

**Output
Snapshots
With
Explanation/
Observations**

Screenshot 1: Original DataFrame

ORIGINAL DATAFRAME

	Feature1	Feature2	Target
0	5	10	0
1	8	18	0
2	12	15	1
3	15	25	0
4	18	30	1
5	22	28	1
6	25	35	0
7	30	40	1
8	35	38	1
9	40	45	0
10	45	50	1
11	50	55	1
12	55	60	0
13	60	65	1
14	70	75	1

DataFrame Shape: (15, 3)

Figure 1.1 - Original DataFrame

Explanation/Observation: The dictionary was converted into a DataFrame containing 15 observations and the three required columns: Feature1, Feature2, and Target.

Screenshot 2: Logarithmic Feature

LOGARITHMIC FEATURE CREATED FROM FEATURE1

	Feature1	Log_Feature1
0	5	1.609
1	8	2.079
2	12	2.485
3	15	2.708
4	18	2.890
5	22	3.091
6	25	3.219
7	30	3.401
8	35	3.555
9	40	3.689
10	45	3.807
11	50	3.912
12	55	4.007
13	60	4.094
14	70	4.248

Figure 1.2 - Logarithmic Transformation of Feature1

Explanation/Observation: The natural logarithm of Feature1 was calculated using np.log(). The transformed values increase gradually and reduce the scale difference among larger values.

Student Reg. No: 26MML0031 Name: Makada Hasnain

Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT), Vellore

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Practice Program

Date: 30/07/2026

Week No.: 4

Program S. No. 7

Screenshot 3: Feature2 Binning

FEATURE2 BINNED INTO LOW, MEDIUM AND HIGH

	Feature2	Feature2_Category
0	10	Low
1	18	Low
2	15	Low
3	25	Low
4	30	Low
5	28	Low
6	35	Medium
7	40	Medium
8	38	Medium
9	45	Medium
10	50	Medium
11	55	High
12	60	High
13	65	High
14	75	High

Figure 1.3 - Feature2 Categorized as Low, Medium, and High

Explanation/Observation: The `pd.cut()` function divided Feature2 into three equal-width categories. The output contains 6 Low, 5 Medium, and 4 High observations.

Screenshot 4: Correlation Matrix

CORRELATION MATRIX

	Feature1	Feature2	Target	Log_Feature1
Feature1	1.000	0.990	0.336	0.943
Feature2	0.990	1.000	0.314	0.946
Target	0.336	0.314	1.000	0.400
Log_Feature1	0.943	0.946	0.400	1.000

Figure 1.4 - Correlation Matrix of Numerical Features

Explanation/Observation: Feature1 and Feature2 have a very strong positive correlation of approximately 0.990. Log_Feature1 also has strong positive relationships with both original features.

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Practice Program

Date: 30/07/2026

Week No.: 4

Program S. No.

7 Screenshot 5: Category Count and Pattern Interpretation

CATEGORY COUNT AND PATTERN INTERPRETATION

Feature2_Category

Low 6

Medium 5

High 4

Correlation between Feature1 and Feature2: 0.990

Interpretation: Feature1 and Feature2 show a strong positive relationship. Target classes overlap, although Target = 1 occurs more often at medium and high feature values.

Figure 1.5 - Category Counts and Console Interpretation

Explanation/Observation: The category distribution and calculated correlation confirm an increasing relationship between Feature1 and Feature2. Target classes overlap, so the features do not completely separate the two classes.

Screenshot 6: Scatter Plot

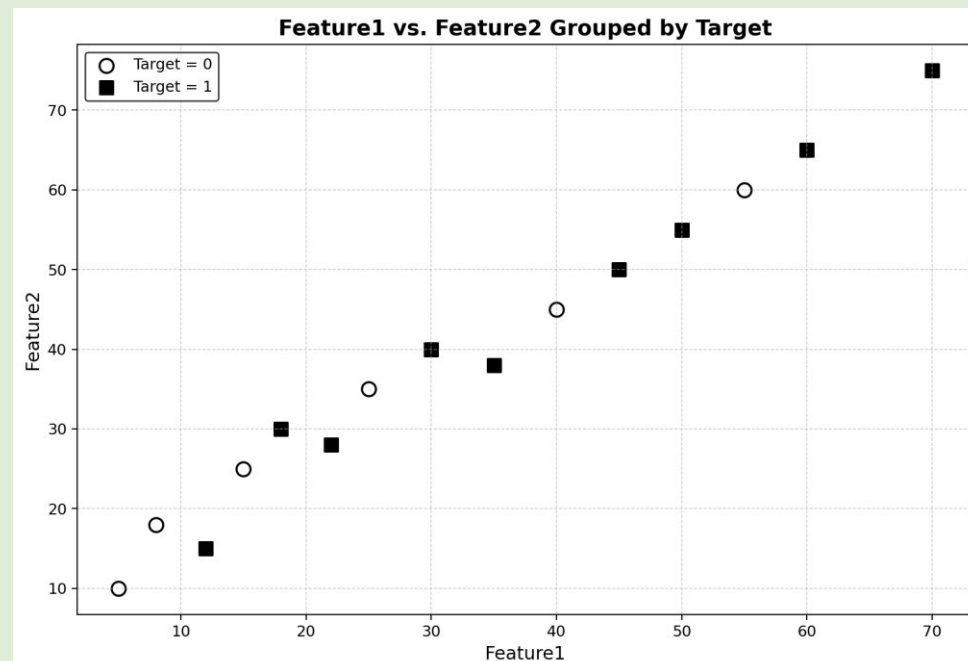


Figure 1.6 - Feature1 versus Feature2 Grouped by Target

Explanation/Observation: The points follow a clear upward pattern, showing that Feature2 generally increases as Feature1 increases. Black squares represent Target = 1, while white circles represent Target = 0. The classes overlap, but Target = 1 occurs more frequently at medium and high feature values.

Conclusions

The feature engineering and correlation analysis exercise was completed successfully using Pandas, NumPy, and Matplotlib. A DataFrame was created from a locally defined Python dictionary. A logarithmic feature was generated from Feature1, Feature2 was categorized into Low, Medium, and

Student Reg. No: 26MML0031 Name: Makada Hasnain

Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT), Vellore

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Practice Program

Date: 30/07/2026

Week No.: 4

Program S. No. 7

	High groups, and a correlation matrix was calculated for the numerical variables. The correlation value of approximately 0.990 showed a very strong positive relationship between Feature1 and Feature2. The black-and-white scatter plot confirmed the upward trend and showed that the Target classes overlap, although Target = 1 appears more frequently at medium and high feature values. The exercise provided practical understanding of feature transformation, binning, correlation analysis, visualization, and pattern interpretation.
References	1. Pandas documentation for DataFrame creation, binning, category counting, and correlation analysis. 2. NumPy documentation for logarithmic transformations. 3. Matplotlib documentation for scatter plots, legends, labels, and plot formatting. 4. Python Program (https://github.com/hasnainmakada-99/Data-Science-Codes/blob/main/WEEK%204/PRACTICE_PROGRAM.py) 5. Programming for Data Science (MACSE502) class lecture notes (https://drive.google.com/file/d/1PqsD1tC9aCgGFZo7QIJblBu1pIjg4UBs/view)

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Exercise Program

Date: 30/07/2026

Week No.: 4

Program S. No. 8

Title	Employee Attrition Prediction Using Workforce Indicators																																																																																																									
Problem Statement	A company wants to analyze workforce behavior and predict employee attrition. Create a DataFrame containing OvertimeHours, JobSatisfactionScore, SalaryLevel, TrainingHours, WorkLifeBalance, YearsAtCompany, PromotionCount, and AttritionStatus. Create Log_OvertimeHours, categorize JobSatisfactionScore as Disengaged, Satisfied, or Highly Engaged, generate a correlation matrix, create a treemap visualization of employee categories, and analyze workforce trends to identify practical retention insights.																																																																																																									
Objectives	• To create an employee workforce DataFrame from a predefined Python dictionary. • To perform logarithmic transformation on employee overtime hours. • To segment employees using job satisfaction scores. • To encode attrition status for numerical analysis. • To generate and interpret a correlation matrix of workforce indicators. • To analyze attrition rates across employee engagement categories. • To identify employees with high attrition risk using workforce conditions. • To create a treemap visualization of employee categories. • To derive retention insights related to workload, satisfaction, salary, training, promotion, and work-life balance.																																																																																																									
Dataset URL Link / File	Predefined dictionary dataset used directly inside the Python program: Table A: Overtime, Satisfaction, Salary and Training Indicators <table><tr><th>Emp.</th><th>Overtime Hours</th><th>Job Satisfaction Score</th><th>Salary Level</th><th>Training Hours</th></tr><tr><td>1</td><td>5</td><td>5</td><td>5</td><td>40</td></tr><tr><td>2</td><td>12</td><td>4</td><td>4</td><td>35</td></tr><tr><td>3</td><td>18</td><td>3</td><td>3</td><td>28</td></tr><tr><td>4</td><td>26</td><td>2</td><td>2</td><td>16</td></tr><tr><td>5</td><td>8</td><td>4</td><td>4</td><td>34</td></tr><tr><td>6</td><td>32</td><td>1</td><td>1</td><td>10</td></tr><tr><td>7</td><td>15</td><td>3</td><td>3</td><td>24</td></tr><tr><td>8</td><td>28</td><td>2</td><td>2</td><td>18</td></tr><tr><td>9</td><td>10</td><td>5</td><td>5</td><td>45</td></tr><tr><td>10</td><td>24</td><td>2</td><td>4</td><td>30</td></tr><tr><td>11</td><td>35</td><td>1</td><td>1</td><td>8</td></tr><tr><td>12</td><td>6</td><td>4</td><td>4</td><td>38</td></tr><tr><td>13</td><td>20</td><td>3</td><td>3</td><td>26</td></tr><tr><td>14</td><td>34</td><td>3</td><td>2</td><td>12</td></tr><tr><td>15</td><td>38</td><td>2</td><td>2</td><td>14</td></tr></table> Table B: Work-Life, Tenure, Promotion and Attrition Indicators <table><tr><th>Emp.</th><th>Work-Life Balance</th><th>Years at Company</th><th>Promotion Count</th><th>Attrition Status</th></tr><tr><td>1</td><td>5</td><td>10</td><td>3</td><td>No</td></tr><tr><td>2</td><td>4</td><td>7</td><td>2</td><td>No</td></tr><tr><td>3</td><td>3</td><td>5</td><td>1</td><td>No</td></tr><tr><td>4</td><td>2</td><td>2</td><td>0</td><td>Yes</td></tr></table>	Emp.	Overtime Hours	Job Satisfaction Score	Salary Level	Training Hours	1	5	5	5	40	2	12	4	4	35	3	18	3	3	28	4	26	2	2	16	5	8	4	4	34	6	32	1	1	10	7	15	3	3	24	8	28	2	2	18	9	10	5	5	45	10	24	2	4	30	11	35	1	1	8	12	6	4	4	38	13	20	3	3	26	14	34	3	2	12	15	38	2	2	14	Emp.	Work-Life Balance	Years at Company	Promotion Count	Attrition Status	1	5	10	3	No	2	4	7	2	No	3	3	5	1	No	4	2	2	0	Yes
Emp.	Overtime Hours	Job Satisfaction Score	Salary Level	Training Hours																																																																																																						
1	5	5	5	40																																																																																																						
2	12	4	4	35																																																																																																						
3	18	3	3	28																																																																																																						
4	26	2	2	16																																																																																																						
5	8	4	4	34																																																																																																						
6	32	1	1	10																																																																																																						
7	15	3	3	24																																																																																																						
8	28	2	2	18																																																																																																						
9	10	5	5	45																																																																																																						
10	24	2	4	30																																																																																																						
11	35	1	1	8																																																																																																						
12	6	4	4	38																																																																																																						
13	20	3	3	26																																																																																																						
14	34	3	2	12																																																																																																						
15	38	2	2	14																																																																																																						
Emp.	Work-Life Balance	Years at Company	Promotion Count	Attrition Status																																																																																																						
1	5	10	3	No																																																																																																						
2	4	7	2	No																																																																																																						
3	3	5	1	No																																																																																																						
4	2	2	0	Yes																																																																																																						

Student Reg. No: 26MML0031 Name: Makada Hasnain

— Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT), Vellore —

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Exercise Program

Date: 30/07/2026

Week No.: 4

Program S. No. 8

	5	4	8	2	No	
	6	1	1	0	Yes	
	7	3	4	1	No	
	8	2	3	0	Yes	
	9	5	11	3	No	
	10	3	6	1	No	
	11	1	1	0	Yes	
	12	4	9	2	No	
	13	3	6	1	No	
	14	2	2	0	Yes	
	15	2	2	0	Yes	
	No external CSV file, URL, database, or online dataset is required.					
	Functions Details	Sl. No.	Function	Package	Description of function	
		1	pd.DataFrame()	Pandas	Creates the workforce DataFrame from the predefined dictionary.	
		2	np.log()	NumPy	Calculates the natural logarithm of OvertimeHours.	
		3	pd.cut()	Pandas	Categorizes job satisfaction and attrition risk into defined groups.	
4		map()	Pandas	Converts AttritionStatus values from Yes/No into 1/0.		
5		corr()	Pandas	Generates the correlation matrix for numerical workforce indicators.		
6		groupby()	Pandas	Groups employee records by engagement or attrition category.		
7		agg()	Pandas	Calculates employee count, attrition count, averages, and summaries.		
8		mean()	Pandas	Calculates overall attrition rate and average workforce indicators.		
9		astype()	Pandas	Converts Boolean risk conditions into integer risk points.		
10		value_counts()	Pandas	Counts employees within each attrition-risk category.		
11		Rectangle()	Matplotli b	Creates proportional rectangles for the treemap visualization.		
12	savefig()/show ()	Matplotli b	Saves and displays the employee-category treemap.			
Steps/ Process/ Procedure/ Algorithm/ Flowchart/ Model	Step 1: Import Pandas, NumPy, Matplotlib, and Rectangle from matplotlib.patches. Step 2: Create the workforce dataset locally in dictionary format. Step 3: Convert the dictionary into a DataFrame and assign employee numbers as the index. Step 4: Create Log_OvertimeHours using the natural logarithm of OvertimeHours. Step 5: Categorize JobSatisfactionScore as Disengaged, Satisfied, or Highly Engaged. Step 6: Encode AttritionStatus as AttritionFlag using No = 0 and Yes = 1.					

Student Reg. No: 26MML0031 Name: Makada Hasnain

Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT), Vellore

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Exercise Program

Date: 30/07/2026

Week No.: 4

Program S. No. 8

	Step 7: Select numerical workforce indicators and generate the correlation matrix. Step 8: Calculate the overall attrition rate and category-wise workforce summaries. Step 9: Compare average workforce indicators for employees who stayed and employees who left. Step 10: Create a rule-based risk score from overtime, satisfaction, salary, work-life balance, and promotion conditions. Step 11: Create a black-background treemap showing employee-category size and attrition rate. Step 12: Interpret workforce trends and recommend suitable employee-retention strategies.
Program	<pre>import pandas as pd import numpy as np import matplotlib.pyplot as plt from matplotlib.patches import Rectangle def print_heading(title): print("\n" + "=" * 100) print(title) print("=" * 100) # 1. Workforce dataset stored locally as a Python dictionary. employee_data = { "OvertimeHours": [5, 12, 18, 26, 8, 32, 15, 28, 10, 24, 35, 6, 20, 34, 38], "JobSatisfactionScore": [5, 4, 3, 2, 4, 1, 3, 2, 5, 2, 1, 4, 3, 3, 2], "SalaryLevel": [5, 4, 3, 2, 4, 1, 3, 2, 5, 4, 1, 4, 3, 2, 2], "TrainingHours": [40, 35, 28, 16, 34, 10, 24, 18, 45, 30, 8, 38, 26, 12, 14], "WorkLifeBalance": [5, 4, 3, 2, 4, 1, 3, 2, 5, 3, 1, 4, 3, 2, 2], "YearsAtCompany": [10, 7, 5, 2, 8, 1, 4, 3, 11, 6, 1, 9, 6, 2, 2], "PromotionCount": [3, 2, 1, 0, 2, 0, 1, 0, 3, 1, 0, 2, 1, 0, 0], "AttritionStatus": ["No", "No", "No", "Yes", "No", "Yes", "No", "Yes", "No", "No", "Yes", "No", "No", "Yes", "Yes"] } # 2. Create and display the DataFrame. df = pd.DataFrame(employee_data) df.index = range(1, len(df) + 1) df.index.name = "Employee" print_heading("ORIGINAL WORKFORCE DATAFRAME") print(df.to_string()) print("\nDataFrame Shape:", df.shape) # 3. Feature engineering and workforce segmentation. df["Log_OvertimeHours"] = np.log(df["OvertimeHours"]).round(3) df["EmployeeCategory"] = pd.cut(df["JobSatisfactionScore"], bins=[0, 2, 3, 5], labels=["Disengaged", "Satisfied", "Highly Engaged"], include_lowest=True)</pre>

Student Reg. No: 26MML0031 Name: Makada Hasnain

Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT), Vellore

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Exercise Program

Date: 30/07/2026

Week No.: 4

Program S. No. 8

```
df["AttritionFlag"] = df["AttritionStatus"].map({"No": 0, "Yes": 1})

print_heading("FEATURE ENGINEERING AND EMPLOYEE SEGMENTATION")
print(df[["OvertimeHours", "Log_OvertimeHours", "JobSatisfactionScore",
"EmployeeCategory", "AttritionStatus"]].to_string())

# 4. Correlation analysis.
numeric_columns = [
    "OvertimeHours", "JobSatisfactionScore", "SalaryLevel",
    "TrainingHours", "WorkLifeBalance", "YearsAtCompany",
    "PromotionCount", "Log_OvertimeHours", "AttritionFlag"
]
correlation_matrix = df[numeric_columns].corr()

print_heading("CORRELATION MATRIX")
print(correlation_matrix.round(3).to_string())

# 5. Workforce trend analysis.
overall_attrition_rate = df["AttritionFlag"].mean() * 100
category_summary = (
    df.groupby("EmployeeCategory", observed=False)
    .agg(
        EmployeeCount=("AttritionFlag", "size"),
        AttritionCount=("AttritionFlag", "sum"),
        AverageOvertime=("OvertimeHours", "mean"),
        AverageSalaryLevel=("SalaryLevel", "mean")
    )
    .reset_index()
)
category_summary["AttritionRatePercent"] = (
    category_summary["AttritionCount"]
    / category_summary["EmployeeCount"] * 100
).round(2)
category_summary["AverageOvertime"] =
category_summary["AverageOvertime"].round(2)
category_summary["AverageSalaryLevel"] =
category_summary["AverageSalaryLevel"].round(2)

status_summary = df.groupby("AttritionStatus")[["OvertimeHours", "JobSatisfactionScore", "SalaryLevel",
"TrainingHours", "WorkLifeBalance", "YearsAtCompany",
"PromotionCount"]].mean().round(2)

print_heading("WORKFORCE TREND ANALYSIS")
print("Overall Attrition Rate: {:.2f}%".format(overall_attrition_rate))
print("\nEmployee Category Summary")
print(category_summary.to_string(index=False))
print("\nAverage Workforce Indicators by Attrition Status")
print(status_summary.to_string())

# 6. Rule-based attrition risk analysis.
df["RiskScore"] = (
    (df["OvertimeHours"] >= 25).astype(int)
    + (df["JobSatisfactionScore"] <= 2).astype(int)
    + (df["SalaryLevel"] <= 2).astype(int)
    + (df["WorkLifeBalance"] <= 2).astype(int)
    + (df["PromotionCount"] == 0).astype(int)
)
df["AttritionRisk"] = pd.cut(
    df["RiskScore"], bins=[-1, 1, 4, 5], labels=["Low", "Medium",
"High"]
)
```

Student Reg. No: 26MML0031 — Name: Makada Hasnain

Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT), Vellore

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Exercise Program

Date: 30/07/2026

Week No.: 4

Program S. No. 8

```
high_risk_employees = df[df["AttritionRisk"] == "High"][["OvertimeHours", "JobSatisfactionScore", "SalaryLevel", "WorkLifeBalance", "PromotionCount", "RiskScore", "AttritionRisk", "AttritionStatus"]]

print_heading("ATTRITION RISK ANALYSIS")
print("Risk Category Distribution")
print(df["AttritionRisk"].value_counts().sort_index().to_string())
print("\nHigh-Risk Employees")
print(high_risk_employees.to_string())

# 7. Treemap visualization using proportional Matplotlib
rectangles, total_employees = category_summary["EmployeeCount"].sum()
fig, ax = plt.subplots(figsize=(11, 6))
fig.patch.set_facecolor("black")
ax.set_facecolor("black")
x_position = 0.0
grayscale_colors = ["#2F2F2F", "#555555", "#7A7A7A"]

for index, row in category_summary.iterrows():
    width = row["EmployeeCount"] / total_employees
    ax.add_patch(Rectangle(
        (x_position, 0), width, 1,
        facecolor=grayscale_colors[index], edgecolor="white",
        linewidth=2
    ))
    label = (
        f'{row["EmployeeCategory"]}\n'
        f'Employees: {int(row["EmployeeCount"])}\n'
        f'Attrition Rate: {row["AttritionRatePercent"]:.1f}%'
    )
    ax.text(
        x_position + width / 2, 0.5, label,
        color="white", ha="center",
        va="center", fontsize=12,
        fontweight="bold"
    )
    x_position += width

ax.set_xlim(0, 1)
ax.set_ylim(0, 1)
ax.axis("off")
ax.set_title(
    "Treemap of Employee Engagement Categories",
    color="white", fontsize=16, fontweight="bold", pad=18
)
plt.tight_layout()
plt.savefig(
    "hr_employee_category_treemap.png",
    dpi=300, facecolor="black", bbox_inches="tight"
)
plt.show()

# 8. Final interpretation.
attrition_correlations = (
    correlation_matrix["AttritionFlag"]
    .drop("AttritionFlag")
    .sort_values(ascending=False)
)
print_heading("FINAL WORKFORCE INSIGHTS")
print("Correlation of Workforce Indicators with Attrition")
print(attrition_correlations.round(3).to_string())
print("\nInterpretation:")
print(
    "1. Higher overtime is associated with greater attrition risk.\n"
```

Student Reg. No: 26MML0031 — Name: Makada Hasnain

Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT), Vellore

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Exercise Program

Date: 30/07/2026

Week No.: 4

Program S. No. 8

```
"2. Lower job satisfaction, salary level, work-life balance,
training,\n"
" tenure, and promotion opportunities are associated
with attrition.\n"
"3. Disengaged employees show the highest attrition rate.\n"
"4. Retention strategies should prioritize workload control,
employee\n"
" engagement, career growth, training, and work-life balance."
)
```

**Output
Snapshots
With
Explanation/
Observations**

Screenshot 1: Original Workforce DataFrame

Employee	OverTimeHours	JobSatisfactionScore	SalaryLevel	TrainingHours	WorkLifeBalance	TenureCompany	PromotionCount	AttritionStatus
1	5	3	1	10	4	10	1	No
2	12	4	4	15	4	7	2	No
3	18	5	3	10	3	3	1	Yes
4	26	2	3	10	2	2	0	Yes
5	18	2	2	10	1	1	0	No
6	32	1	1	10	1	1	0	Yes
7	15	3	2	10	2	4	1	Yes
8	20	2	2	10	2	4	1	Yes
9	20	5	5	40	5	11	1	No
10	24	4	4	30	4	9	1	No
11	35	1	1	8	1	1	0	Yes
12	6	4	4	30	4	9	1	No
13	20	3	3	20	3	6	1	Yes
14	30	2	2	12	2	2	0	Yes
15	38	2	2	14	2	2	0	Yes

Figure 1.1 - Original Workforce DataFrame

Explanation/Observation: The locally defined dictionary was converted into a DataFrame containing 15 employee records and all eight required workforce indicators.

Screenshot 2: Feature Engineering and Employee Segmentation

Employee	OverTimeHours	Log_OvertimeHours	JobSatisfactionScore	EmployeeCategory	AttritionStatus
1	5	1.609	5	Highly Engaged	No
2	12	2.485	4	Highly Engaged	No
3	18	2.890	3	Satisfied	No
4	26	3.258	2	Disengaged	Yes
5	18	2.890	5	Highly Engaged	No
6	32	3.466	1	Disengaged	Yes
7	15	2.708	3	Satisfied	No
8	20	3.302	2	Disengaged	Yes
9	20	2.996	5	Highly Engaged	No
10	24	3.178	2	Disengaged	No
11	35	3.555	1	Disengaged	Yes
12	6	1.792	4	Highly Engaged	No
13	20	2.996	3	Satisfied	No
14	30	3.526	3	Satisfied	Yes
15	38	3.638	2	Disengaged	Yes

Figure 1.2 - Logarithmic Overtime Feature and Employee Categories

Explanation/Observation: Log_OvertimeHours was generated using np.log(). JobSatisfactionScore was categorized as Disengaged, Satisfied, or Highly Engaged, and AttritionStatus was retained for comparison.

Screenshot 3: Correlation Matrix

	OverTimeHours	JobSatisfactionScore	SalaryLevel	TrainingHours	WorkLifeBalance	TenureCompany	PromotionCount	Log_OvertimeHours	AttritionFlag
OverTimeHours	1.000	-0.364	-0.287	-0.321	-0.154	0.308	0.303	0.364	-0.854
JobSatisfactionScore	-0.364	1.000	0.000	0.000	0.000	0.000	0.000	-0.874	-0.729
SalaryLevel	-0.287	0.000	1.000	0.000	0.000	0.000	0.000	-0.854	-0.861
TrainingHours	-0.321	0.000	0.000	1.000	0.000	0.000	0.000	-0.889	-0.870
WorkLifeBalance	-0.154	0.000	0.000	0.000	1.000	0.000	0.000	-0.903	-0.826
TenureCompany	0.308	0.000	0.000	0.000	0.000	1.000	0.000	-0.801	-0.810
PromotionCount	0.303	0.000	0.000	0.000	0.000	0.000	1.000	-0.821	-0.824
Log_OvertimeHours	0.364	-0.874	-0.854	-0.889	-0.903	-0.801	-0.821	1.000	0.772
AttritionFlag	-0.854	-0.729	-0.861	-0.870	-0.826	-0.810	-0.824	0.772	1.000

Figure 1.3 - Correlation Matrix of Workforce Indicators

Explanation/Observation: OverTimeHours has a strong positive relationship with attrition, while satisfaction, salary, training, work-life balance, tenure, and promotion indicators show negative relationships with attrition.

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Exercise Program

Date: 30/07/2026

Week No.: 4

Program S. No. 8

Screenshot 4: Workforce Trend Analysis

WORKFORCE TREND ANALYSIS						
Overall Attrition Rate: 40.00%						
Employee Category Summary						
EmployeeCategory	EmployeeCount	AttritionCount	AverageOvertime	AverageSalaryLevel	AttritionRatePercent	
Disengaged	6	5	16.50	2.00	83.33	
Satisfied	4	1	21.75	2.75	25.00	
Highly Engaged	5	0	8.20	4.40	0.00	
Average Workforce Indicators by Attrition Status						
AttritionStatus	OvertimeHours	JobSatisfactionScore	SalaryLevel	TrainingHours	WorkLifeBalance	YearsAtCompany
No	13.11	3.67	3.89	33.33	3.78	7.33
Yes	32.17	1.83	1.67	13.00	1.67	1.83

Figure 1.4 - Attrition Rate and Workforce Category Summary

Explanation/Observation: The overall attrition rate is 40%. Disengage employees have the highest attrition rate and the highest average overtime while highly engaged employees show no attrition in the sample.

Screenshot 5: Attrition Risk Analysis

Attrition Risk Analysis								
Risk Category Distribution								
AttritionRisk								
Low	9							
Medium	1							
High	5							
High-Risk Employees								
Employee	OvertimeHours	JobSatisfactionScore	SalaryLevel	WorkLifeBalance	PromotionCount	RiskScore	AttritionRisk	AttritionStatus
4	26	2	2	2	0	5	High	Yes
6	32	1	1	1	0	5	High	Yes
9	28	2	2	2	0	5	High	Yes
11	35	1	1	1	0	5	High	Yes
13	38	2	2	2	0	5	High	Yes

Figure 1.5 - Rule-Based Attrition Risk Distribution

Explanation/Observation: Employees were assigned risk points based on overtime, satisfaction, salary, work-life balance, and promotion conditions. Five employees were identified as high-risk, and all five have an observed attrition status of Yes.

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Exercise Program

Date: 30/07/2026

Week No.: 4

Program S. No. 8

Screenshot 6: Employee Category Treemap

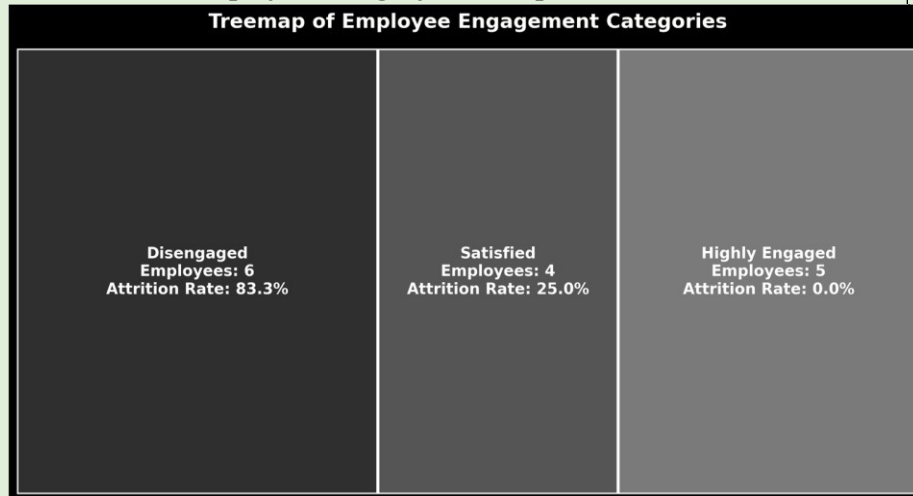


Figure 1.6 - Treemap of Employee Engagement Categories

Explanation/Observation: The treemap displays the relative size of each employee category. Each rectangle also reports the number of employees and the attrition rate for that category.

Screenshot 7: Final Workforce Insights

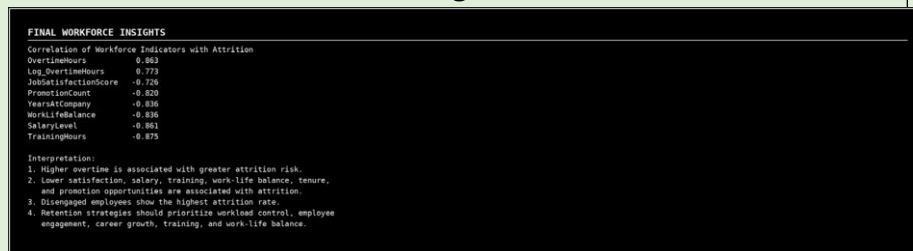


Figure 1.7 - Correlation-Based Workforce Interpretation

Explanation/Observation: The final analysis indicates that overtime is the strongest positive attrition indicator. Retention efforts should focus on workload control, engagement, compensation, training, career growth, and work-life balance.

Conclusions

The Human Resource Analytics exercise was completed successfully using Pandas, NumPy, and Matplotlib. A workforce DataFrame was created from a predefined Python dictionary, Log_OvertimeHours was generated, and employees were segmented using job satisfaction scores. The correlation matrix showed that higher overtime was positively associated with attrition, whereas job satisfaction, salary level, training, work-life balance, tenure, and promotion opportunities were negatively associated with attrition. The treemap highlighted the distribution of engagement categories, and the trend analysis showed that disengaged employees had the highest attrition rate. The rule-based risk analysis identified employees requiring immediate attention. These findings support retention strategies focused on workload reduction, employee

Student Reg. No: 26MML0031 Name: Makada Hasnain

Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT), Vellore

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Exercise Program

Date: 30/07/2026

Week No.: 4

Program S. No. 8

	engagement, fair compensation, career progression, training opportunities, and improved work-life balance.
References	1. Pandas documentation for DataFrame creation, categorization, grouping, aggregation, mapping, and correlation analysis. 2. NumPy documentation for logarithmic transformation. 3. Matplotlib documentation for figures, rectangles, text, and image export. 4. Python program supplied with this exercise https://github.com/hasnainmakada-99/Data-Science-Codes/blob/main/WEEK%204/EXERCISE_PROGRAM.py 5. Programming for Data Science (MACSE502) class lecture notes https://drive.google.com/file/d/1PqsD1tC9aCgGFZo7QIJbBu1pIjg4UBs/view

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Practice Program

Date: 06/08/2026

Week No.: 5

Program S. No. 9

Title	Web Scraping and Data Collection			
Problem Statement	Scrape data from a webpage and store it in a structured format like CSV or JSON format			
Objectives	• To understand the concept of web scraping and data collection. • To retrieve webpage content using the Requests library. • To extract required information from HTML using BeautifulSoup. • To organize the scraped data using Pandas DataFrame. • To store the collected data in structured formats such as CSV and JSON.			
Dataset URL Link / File	https://quotes.toscrape.com			
Function Details	Sl. No.	Function	Package	Description of Function
	1	requests.get()	requests	Retrieves the HTML content of a webpage
	2	BeautifulSoup()	bs4	Parses the HTML content of the webpage
	3	find_all()	BeautifulSoup	Finds all matching HTML elements
	4	find()	BeautifulSoup	Finds the first matching HTML element
	5	DataFrame()	pandas	Converts collected data into tabular format
	6	to_csv()	pandas	Stores data in CSV format
	7	json.dump()	json	Stores Python data in JSON format
	8	json.load()	json	Reads data from a JSON file
Steps/ Process/ Procedure/	1. Import the required libraries: requests, BeautifulSoup, pandas and json. 2. Specify the URL of the webpage to be scraped.			

Student Reg. No: 26MML0031 Name: Hasnain Makada

Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT), Vellore

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Practice Program

Date: 06/08/2026

Week No.: 5

Program S. No. 9

Algorithm/ Flowchart/ Model	3. Send an HTTP GET request using requests.get(). 4. Parse the HTML content using BeautifulSoup. 5. Locate the required HTML elements containing quotes, authors and tags. 6. Extract the required data from the webpage. 7. Store the extracted data in a Python list. 8. Convert the collected data into a Pandas DataFrame. 9. Save the structured data in CSV format using to_csv(). 10. Save the same data in JSON format using json.dump(). 11. Display the collected data to verify the output.
Program	<pre> import requests from bs4 import BeautifulSoup import pandas as pd import json url = "https://quotes.toscrape.com/" response = requests.get(url) print("Status Code:", response.status_code) soup = BeautifulSoup(response.text, "html.parser") print("Website Title:", soup.title.text) quotes = [] for item in soup.find_all("div", class_="quote"): quote = item.find("span", class_="text").text author = item.find("small", class_="author").text tags = [tag.text for tag in item.find_all("a", class_="tag")] quotes.append({ "Quote": quote, "Author": author, "Tags": " ".join(tags) }) print("Number of quotes collected:", len(quotes)) </pre>

Student Reg. No: 26MML0031 Name: Hasnain Makada

*Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT),
Vellore*

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Practice Program

Date: 06/08/2026

Week No.: 5

Program S. No. 9

```
df = pd.DataFrame(quotes)

df

df.to_csv("quotes.csv", index=False)

print("Data saved successfully in CSV format.")

with open("quotes.json", "w", encoding="utf-8") as file:
    json.dump(quotes, file, indent=4, ensure_ascii=False)

print("Data saved successfully in JSON format.")

csv_data = pd.read_csv("quotes.csv")

csv_data

all_quotes = []

page = 1

while True:

    url = f"https://quotes.toscrape.com/page/{page}/"

    response = requests.get(url)
    soup = BeautifulSoup(response.text, "html.parser")

    items = soup.find_all("div", class_="quote")

    if not items:
        break

    for item in items:

        quote = item.find("span", class_="text").text
        author = item.find("small", class_="author").text

        tags = [
            tag.text
            for tag in item.find_all("a", class_="tag")
        ]

        all_quotes.append({
            "Quote": quote,
```

Student Reg. No: 26MML0031 Name: Hasnain Makada

*Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT),
Vellore*

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Practice Program

Date: 06/08/2026

Week No.: 5

Program S. No. 9

	<pre> "Author": author, "Tags": ", ".join(tags) }) page += 1 print("Total quotes collected:", len(all_quotes)) df = pd.DataFrame(all_quotes) df df.to_csv("quotes_dataset.csv", index=False) with open("quotes_dataset.json", "w", encoding="utf-8") as file: json.dump(all_quotes, file, indent=4, ensure_ascii=False) print("CSV and JSON files created successfully.") </pre>
Output Snapshots With Explanation/ Observations	The required libraries requests, BeautifulSoup, pandas and json are imported for web scraping, HTML parsing, data organization and structured data storage. <pre> 1 import requests 2 from bs4 import BeautifulSoup 3 import pandas as pd 4 import json </pre> <pre> 1 url = "https://quotes.toscrape.com/" 2 3 response = requests.get(url) 4 5 print("Status Code:", response.status_code) </pre> Status Code: 200 <i>Fig. 1.1: Importing Required Python Libraries</i>

Student Reg. No: 26MML0031 Name: Hasnain Makada

Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT),
Vellore

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Practice Program

Date: 06/08/2026

Week No.: 5

Program S. No. 9

The webpage <https://quotes.toscrape.com/> is accessed using `requests.get()`. The status code 200 confirms that the webpage was successfully retrieved.

```
[7]: 1 quotes = []
      2
      3 for item in soup.find_all("div", class_="quote"):
      4
      5     quote = item.find("span", class_="text").text
      6     author = item.find("small", class_="author").text
      7
      8     tags = [
      9         tag.text
     10         for tag in item.find_all("a", class_="tag")
     11     ]
     12
     13 quotes.append({
     14     "Quote": quote,
     15     "Author": author,
     16     "Tags": ", ".join(tags)
     17 })
     18
     19 print("Number of quotes collected:", len(quotes))

Number of quotes collected: 10
```

Fig. 1.2: Accessing the Webpage

The HTML content of the webpage is parsed using BeautifulSoup. The webpage title confirms that the correct page has been successfully loaded.

Student Reg. No: 26MML0031 Name: Hasnain Makada

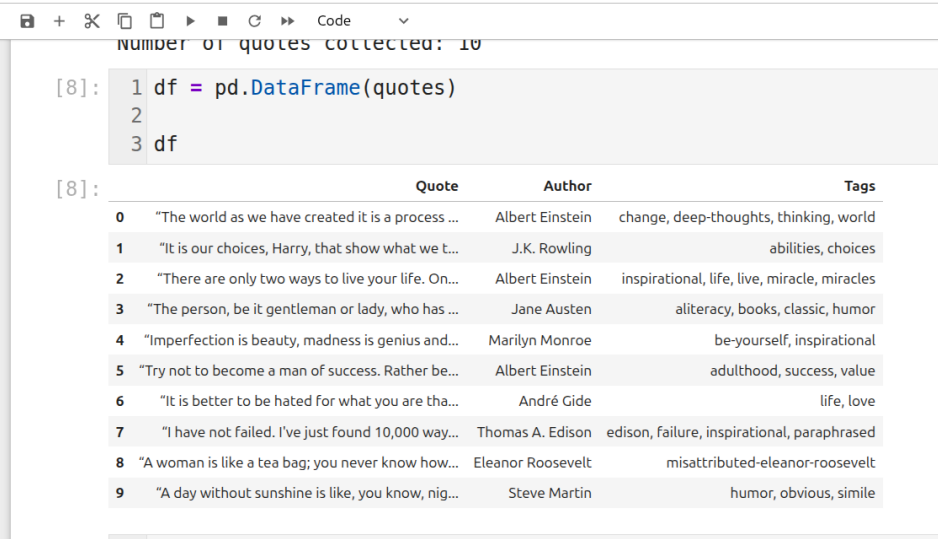
Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT),
Vellore

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Practice Program

Date: 06/08/2026

Week No.: 5

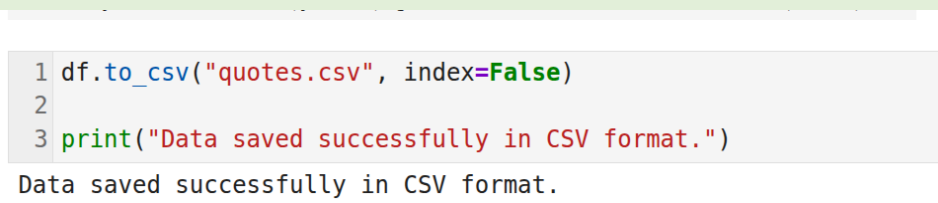
Program S. No. 9



```
[8]: 1 df = pd.DataFrame(quotes)
      2
      3 df
```

	Quote	Author	Tags
0	"The world as we have created it is a process ...	Albert Einstein	change, deep-thoughts, thinking, world
1	"It is our choices, Harry, that show what we t...	J.K. Rowling	abilities, choices
2	"There are only two ways to live your life. On...	Albert Einstein	inspirational, life, live, miracle, miracles
3	"The person, be it gentleman or lady, who has ...	Jane Austen	aliteracy, books, classic, humor
4	"Imperfection is beauty, madness is genius and...	Marilyn Monroe	be-yourself, inspirational
5	"Try not to become a man of success. Rather be...	Albert Einstein	adulthood, success, value
6	"It is better to be hated for what you are tha...	André Gide	life, love
7	"I have not failed. I've just found 10,000 way...	Thomas A. Edison	edison, failure, inspirational, paraphrased
8	"A woman is like a tea bag; you never know how...	Eleanor Roosevelt	misattributed-eleanor-roosevelt
9	"A day without sunshine is like, you know, nig...	Steve Martin	humor, obvious, simile

Fig. 1.3: Parsing the HTML and displaying the dataset Content



```
1 df.to_csv("quotes.csv", index=False)
2
3 print("Data saved successfully in CSV format.")
```

Data saved successfully in CSV format.

Fig. 1.4: Saving Data in CSV Format

The scraped data is successfully stored in CSV format using the pandas `to_csv()` function.

The scraped data is successfully stored in JSON format using the `json.dump()` function.

Student Reg. No: 26MML0031 Name: Hasnain Makada

Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT),
Vellore

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Practice Program

Date: 06/08/2026

Week No.: 5

Program S. No. 9

```
1 with open("quotes.json", "w", encoding="utf-8") as file:
2     json.dump(quotes, file, indent=4, ensure_ascii=False)
3
4 print("Data saved successfully in JSON format.")
```

Data saved successfully in JSON format.

Fig. 1.5: Saving Data in JSON Format

	Quote	Author	Tags
0	"The world as we have created it is a process ...	Albert Einstein	change, deep-thoughts, thinking, world
1	"It is our choices, Harry, that show what we t...	J.K. Rowling	abilities, choices
2	"There are only two ways to live your life. On...	Albert Einstein	inspirational, life, live, miracle, miracles
3	"The person, be it gentleman or lady, who has ...	Jane Austen	aliteracy, books, classic, humor
4	"Imperfection is beauty, madness is genius and...	Marilyn Monroe	be-yourself, inspirational
5	"Try not to become a man of success. Rather be...	Albert Einstein	adulthood, success, value
6	"It is better to be hated for what you are tha...	André Gide	life, love
7	"I have not failed. I've just found 10,000 way...	Thomas A. Edison	edison, failure, inspirational, paraphrased
8	"A woman is like a tea bag; you never know how...	Eleanor Roosevelt	misattributed-eleanor-roosevelt
9	"A day without sunshine is like, you know, nig...	Steve Martin	humor, obvious, simile

Fig 1.6: Displaying the saved json and csv

The scraped data is successfully stored in JSON format using the `json.dump()` function.

The saved CSV file is read using `pandas.read_csv()` and displayed to verify that the data has been stored correctly.

The saved JSON file is read using `json.load()` and the stored records are displayed successfully, confirming correct JSON storage.

Conclusions

The experiment was successfully completed by scraping data from the given webpage using Python. The webpage content was retrieved using the requests library and parsed using BeautifulSoup. The required information such as quotes, authors, and tags was extracted and organized using a Pandas DataFrame. Finally, the collected data was successfully stored in structured CSV and JSON formats for further analysis and use.

References

1. Michael Freeman and Joel Ross, Programming Skills for Data Science: Start Writing Code to Wrangle, Analyze, and Visualize Data with R, Addison-Wesley, 2018
2. Python Program (Google Colab link /Github link)
3. Class lecture link

Student Reg. No: 26MML0031 Name: Hasnain Makada

Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT),
Vellore

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Exercise Program

Date: 06/08/2026

Week No.: 5

Program S. No. 10

Title	Online Job Market Analytics			
Problem Statement	A recruitment agency wants to analyze job opportunities posted on a dynamic job portal. Develop a Python application using Selenium to scrape job listings, extract Job Title, Company Name, Location and Salary (if available), store the collected data in a Pandas DataFrame, save the dataset as a CSV file, and visualize job demand by location.			
Objectives	• To understand web scraping from a dynamic job portal using Selenium. • To extract job title, company name, location and salary information from job listings. • To organize and clean the collected data using a Pandas DataFrame. • To save the structured job dataset in CSV format. • To perform location-wise analysis and visualize job distribution using Matplotlib.			
Dataset URL Link / File	Job Portal: https://www.naukri.com Search role used: Data Scientist Cities: Bangalore, Hyderabad, Pune, Chennai, Mumbai and Delhi			
Functions Details	Sl. No.	Function	Package	Description of function
	1	Firefox()	selenium	Starts a Firefox browser session controlled by Selenium.
	2	driver.get()	selenium	Opens the specified job-portal URL in the automated browser.
	3	execute_script()	selenium	Executes JavaScript to scroll the dynamically loaded webpage.
	4	find_elements()	selenium	Finds all matching job-card elements on the page.
	5	find_element()	selenium	Finds a required field inside an individual job card.
	6	time.sleep()	time	Pauses execution briefly to allow dynamic content to load.
	7	DataFrame()	pandas	Converts the collected job records into tabular form.
	8	drop_duplicates()	pandas	Removes duplicate job records from the DataFrame.
	9	reset_index()	pandas	Resets row indexing after data cleaning.
	10	to_csv()	pandas	Saves the structured job dataset as a CSV file.
	11	value_counts()	pandas	Counts job records for each location for analysis.
	12	bar()/savefig()	matplotlib	Creates and saves the location-wise job-demand bar chart.
Steps/ Process/ Procedure/ Algorithm/ Flowchart/ Model	1. Import Selenium, Pandas, Matplotlib and supporting Python modules. 2. Define the job role, location and maximum number of job listings to collect. 3. Launch the browser using Selenium WebDriver and construct the Naukri search URL. 4. Open the dynamic job-search page and allow sufficient time for the page to load. 5. Scroll the webpage so that dynamically loaded job cards become available. 6. Locate job-card elements using CSS selectors. 7. Extract Job Title, Company Name, Location and Salary from each job card. 8. Store the extracted records in a Python list and convert them into a Pandas DataFrame. 9. Remove duplicate records and save the cleaned dataset as a CSV file. 10. Repeat the search for Bangalore, Hyderabad, Pune, Chennai, Mumbai			

Student Reg. No: 26MML0031

Name: Hasnain Makada

Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT),

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Exercise Program

Date: 06/08/2026

Week No.: 5

Program S. No. 10

	and Delhi. 11. Use value_counts() to prepare location-wise job analysis and create a bar chart. 12. Save the visualization and multi-city CSV file, display the final summary and close the browser. START -> Open Job Portal -> Scrape Listings -> DataFrame -> CSV -> Location Analysis -> Bar Chart -> STOP
Program	<pre> import time import pandas as pd import matplotlib.pyplot as plt from selenium import webdriver from selenium.webdriver.common.by import By from selenium.webdriver.firefox.options import Options from selenium.common.exceptions import NoSuchElementException JOB_ROLE = "data scientist" LOCATION = "bangalore" MAX_JOBS = 30 options = Options() driver = webdriver.Firefox(options=options) driver.set_window_size(1400, 900) def make_slug(text): return "-".join(text.lower().strip().split()) def get_text(card, selectors, default="Not Available"): for selector in selectors: try: element = card.find_element(By.CSS_SELECTOR, selector) text = element.text.strip() if text: return text except NoSuchElementException: pass return default job_slug = make_slug(JOB_ROLE) location_slug = make_slug(LOCATION) url = f"https://www.naukri.com/{job_slug}-jobs-in-{location_slug}" print("Search URL:", url) driver.get(url) time.sleep(5) print("Page Title:", driver.title) print("Current URL:", driver.current_url) </pre>

Student Reg. No: 26MML0031

Name: Hasnain Makada

Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT),

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Exercise Program

Date: 06/08/2026

Week No.: 5

Program S. No. 10

```
for i in range(3):
    driver.execute_script("window.scrollTo(0,
document.body.scrollHeight);")
    time.sleep(2)
driver.execute_script("window.scrollTo(0, 0);")

cards = driver.find_elements(By.CSS_SELECTOR, "div.cust-job-tuple")
print("Number of job cards found:", len(cards))

jobs = []
for card in cards[:MAX_JOBS]:
    title = get_text(card, ["a.title", "[class*='title']"])
    company = get_text(card, ["a.comp-name", ".comp-name"])
    location = get_text(card, ["span.locWdth", ".locWdth",
"[class*='location']"])
    salary = get_text(card, ["span.sal", ".sal", "[class*='salary']"],
default="Not Disclosed")
    jobs.append({
        "Job Title": title,
        "Company Name": company,
        "Location": location,
        "Salary": salary
    })

print("Jobs scraped:", len(jobs))
df = pd.DataFrame(jobs)
df = df.drop_duplicates().reset_index(drop=True)
print("Total unique jobs:", len(df))
print("Dataset Shape:", df.shape)
print(df.head(10))
print("Salary Information:")
print(df["Salary"].value_counts())

df.to_csv("job_market_data.csv", index=False)
print("CSV file saved successfully.")

csv_data = pd.read_csv("job_market_data.csv")
print(csv_data.head())

cities = ["bangalore", "hyderabad", "pune", "chennai", "mumbai",
"delhi"]
all_jobs = []

for city in cities:
    city_slug = make_slug(city)
    url = f"https://www.naukri.com/{job_slug}-jobs-in-{city_slug}"
    print("Scraping:", city)
```

Student Reg. No: 26MML0031

Name: Hasnain Makada

Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT),

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Exercise Program

Date: 06/08/2026

Week No.: 5

Program S. No. 10

```
driver.get(url)

time.sleep(4)

city_cards = driver.find_elements(By.CSS_SELECTOR, "div.cust-job-  
tuple")
print("Jobs found:", len(city_cards))

for card in city_cards[:10]:
    title = get_text(card, ["a.title", "[class*='title']"])
    company = get_text(card, ["a.comp-name", ".comp-name"])
    salary = get_text(card, ["span.sal", ".sal", "[class*='salary']"],  
                        default="Not Disclosed")
    all_jobs.append({
        "Job Title": title,
        "Company Name": company,
        "Location": city.title(),
        "Salary": salary
    })

city_df = pd.DataFrame(all_jobs)
city_df = city_df.drop_duplicates().reset_index(drop=True)
print("Total jobs collected:", len(city_df))

location_analysis = city_df["Location"].value_counts().reset_index()
location_analysis.columns = ["Location", "Number of Jobs"]
print(location_analysis)

plt.figure(figsize=(10, 5))
plt.bar(location_analysis["Location"], location_analysis["Number of  
Jobs"])
plt.title("Data Scientist Job Demand by Location")
plt.xlabel("Location")
plt.ylabel("Number of Job Listings")
plt.xticks(rotation=45)
plt.tight_layout()
plt.savefig("job_demand_by_location.png", dpi=300,  
bbox_inches="tight")
plt.show()

city_df.to_csv("job_market_multi_city.csv", index=False)
print("Multi-city dataset saved successfully.")

print("ONLINE JOB MARKET ANALYTICS")
print("Job Role:", JOB_ROLE)
print("Total Jobs Analyzed:", len(city_df))
print("Cities Analyzed:", len(location_analysis))
```

Student Reg. No: 26MML0031

Name: Hasnain Makada

Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT),

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Exercise Program

Date: 06/08/2026

Week No.: 5

Program S. No. 10

**Output
Snapshots
With
Explanation/
Observations**

```
print(location_analysis.to_string(index=False))

driver.quit()
print("Browser closed successfully.")
```

```
print("Browser closed successfully.")
Firefox started successfully.
[5]: def make_slug(text):
      return "-".join(text.lower().strip().split())

      job_slug = make_slug(JOB_ROLE)
      location_slug = make_slug(LOCATION)

      url = f"https://www.naukri.com/{job_slug}-jobs-in-{location_slug}"
      print("Search URL:", url)

      Search URL: https://www.naukri.com/data-scientist-jobs-in-bangalore
[6]: driver.get(url)

      time.sleep(5)

      print("Page Title:", driver.title)
      print("Current URL:", driver.current_url)

      Page Title: Data Scientist Jobs In Bangalore - 7018 Data Scientist Job Vacancies
      In Bangalore - Naukri.com
      Current URL: https://www.naukri.com/data-scientist-jobs-in-bangalore
[7]: print("Website Title:")
      print(driver.title)
```

Fig. 1.1: Search URL and Job Portal Validation

The Selenium session generated the Naukri search URL for Data Scientist jobs in Bangalore and successfully retrieved the page title and current URL, confirming that the dynamic job portal was opened correctly.

```
)
time.sleep(2)

driver.execute_script(
    "window.scrollTo(0, 0);"
)

print("Page scrolling completed.")
Page scrolling completed.
[9]: cards = driver.find_elements(
      By.CSS_SELECTOR,
      "div.cust-job-tuple"
    )

      print("Number of job cards found:", len(cards))

      Number of job cards found: 20
[10]: print(cards[0].text)

      Data Scientist
      Aditya Birla Management Corporation (ABMCPL)
      4.1
      2366 Reviews
      7-10 Yrs
      Bengaluru
      Position Title - Data Scientist Job Location - BengaluruJob Purpose The role
      focuses o...
      Machine LearningDeep LearningPythonPredictive ModelingPysparkArtificial
      IntelligenceNatural Language ProcessingNeural Networks
```

Fig. 1.2: Detection of Dynamic Job Cards

After scrolling the page, Selenium detected 20 job-card elements. The

Student Reg. No: 26MML0031

Name: Hasnain Makada

Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT),

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Exercise Program

Date: 06/08/2026

Week No.: 5

Program S. No. 10

first job card displays details such as job title, company, experience, location and associated skills.

```
jobs.append({
    "Job Title": title,
    "Company Name": company,
    "Location": location,
    "Salary": salary
})

print("Jobs scraped:", len(jobs))

Jobs scraped: 20

[14]: jobs[:3]

[14]: [{'Job Title': 'Data Scientist',
      'Company Name': 'Aditya Birla Management Corporation (ABMCPL)',
      'Location': 'Bengaluru',
      'Salary': 'Not Disclosed'},
      {'Job Title': 'Data Scientist/Software Developer',
      'Company Name': 'Xylem Water Solutions',
      'Location': 'Bengaluru',
      'Salary': 'Not Disclosed'},
      {'Job Title': 'Data Scientist III',
      'Company Name': 'Walmart',
      'Location': 'Bengaluru',
      'Salary': 'Not Disclosed'}]

[15]: df = pd.DataFrame(jobs)

df

[15]:
```

	Job Title \
0	Data Scientist
1	Data Scientist/Software Developer
2	Data Scientist III
3	Data Scientist
4	Data Scientist Python GenAI LLM RAG
5	Machine Learning Data Scientist
6	Advanced Data Scientist
7	Data Scientist
8	Data Scientist GenAI Agentic AI LLM RAG
9	Data Scientist AIML
10	Data Scientist/Python/MLOps- 4th Sep(Fri)-6+ y...
11	Data scientist III
12	Data Scientist
13	Data Scientist/ Data Analyst
14	Walk-in Data Scientist

Fig. 1.3: Scraped Job Records and Pandas DataFrame

The scraper collected 20 listings and extracted the required Job Title, Company Name, Location and Salary fields. The records were then converted into a structured Pandas DataFrame.

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Exercise Program

Date: 06/08/2026

Week No.: 5

Program S. No. 10

```
7           Pune, Bengaluru Not Disclosed
8 Hybrid - Pune, Bengaluru, Delhi / NCR Not Disclosed
9           Hyderabad, Chennai, Bengaluru Not Disclosed
```

```
[18]: print("Dataset Shape:", df.shape)

      print("\nColumn Names:")
      print(df.columns.tolist())

      print("\nTotal Records:", len(df))
```

Dataset Shape: (20, 4)

Column Names:
['Job Title', 'Company Name', 'Location', 'Salary']

Total Records: 20

```
[19]: print("Salary Information:")

      print(df["Salary"].value_counts())
```

Salary Information:
Salary
Not Disclosed 16
20-30 Lacs PA 1
15-20 Lacs PA 1
14-20 Lacs PA 1
0-18 Lacs PA 1
Name: count, dtype: int64

```
[20]: df.to_csv(
      "job_market_data.csv",
```

Fig. 1.4: Dataset Structure, Salary Availability and CSV Creation

The initial dataset contains 20 records and 4 required columns. Salary information was unavailable for most listings, while a few postings contained salary ranges. The cleaned DataFrame was saved successfully as job_market_data.csv.

```
      index=False
    )
    print("CSV file saved successfully.")
```

CSV file saved successfully.

```
[21]: csv_data = pd.read_csv(
      "job_market_data.csv"
    )

    csv_data.head()
```

```
[21]: Job Title \
0           Data Scientist
1 Data Scientist/Software Developer
2           Data Scientist III
3           Data Scientist
4 Data Scientist | Python | GenAI | LLM | RAG

Company Name \
0 Aditya Birla Management Corporation (ABMCPL)
1           Xylem Water Solutions
2           Walmart
3           Genpact
4           Infosys

Location      Salary
0 Bengaluru Not Disclosed
1 Bengaluru Not Disclosed
2 Bengaluru Not Disclosed
3 Bengaluru 20-30 Lacs PA
4 Bengaluru Not Disclosed
```

Fig. 1.5: Verification of the Saved CSV Dataset

The generated CSV file was read back using pandas.read_csv(), and the displayed rows confirm that the job-listing data was stored correctly in

Student Reg. No: 26MML0031

Name: Hasnain Makada

Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT),

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Exercise Program

Date: 06/08/2026

Week No.: 5

Program S. No. 10

structured form.

```
"Company Name": company,
"Location": city.title(),
"Salary": salary
})

Scraping: bangalore
Jobs found: 20

Scraping: hyderabad
Jobs found: 20

Scraping: pune
Jobs found: 20

Scraping: chennai
Jobs found: 20

Scraping: mumbai
Jobs found: 21

Scraping: delhi
Jobs found: 21

[25]: city_df = pd.DataFrame(all_jobs)

city_df = city_df.drop_duplicates().reset_index(drop=True)

print("Total jobs collected:", len(city_df))

city_df.head(10)

Total jobs collected: 60

[25]:
```

	Job Title \
0	Data Scientist
1	Data Scientist/Software Developer
2	Data Scientist III
3	Data Scientist
4	Data Scientist Python GenAI LLM RAG
5	Machine Learning Data Scientist
6	Advanced Data Scientist
7	Data Scientist

Fig. 1.6: Multi-City Job Data Collection

The same Data Scientist search was executed for Bangalore, Hyderabad, Pune, Chennai, Mumbai and Delhi. A controlled sample of 10 listings per city produced 60 records for location-wise analysis.

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Exercise Program

Date: 06/08/2026

Week No.: 5

Program S. No. 10

4	Infosys	Bangalore	Not Disclosed
5	Hsbc	Bangalore	Not Disclosed
6	ExxonMobil	Bangalore	Not Disclosed
7	VOIS	Bangalore	Not Disclosed
8	Infosys	Bangalore	Not Disclosed
9	Tata Consultancy Services	Bangalore	Not Disclosed

```
[26]: location_analysis = (  
    city_df[["Location"]  
    .value_counts()  
    .reset_index()  
)  
  
location_analysis.columns = [  
    "Location",  
    "Number of Jobs"  
]  
  
location_analysis
```

```
[26]:
```

	Location	Number of Jobs
0	Bangalore	10
1	Hyderabad	10
2	Pune	10
3	Chennai	10
4	Mumbai	10
5	Delhi	10

```
[27]: plt.figure(figsize=(10, 5))
```

Fig. 1.7: Location-Wise Job Analysis

Pandas value_counts() produced the location-wise summary for the six-city job sample.

```
plt.tight_layout()  
plt.show()
```

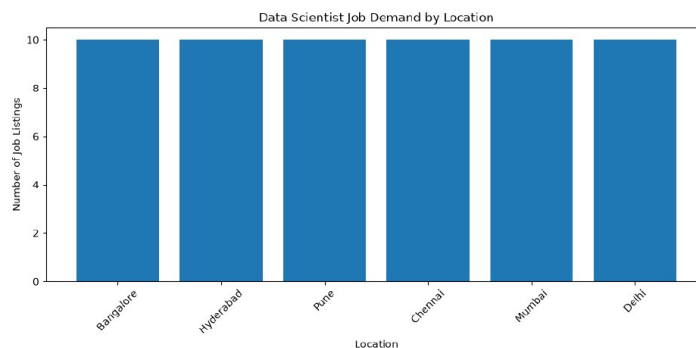


Fig. 1.8: Data Scientist Job Demand Visualization by Location

A Matplotlib bar chart visualizes the number of sampled job listings for each selected city. The graph provides a clear location-wise comparison of the collected job-market sample.

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Exercise Program

Date: 06/08/2026

Week No.: 5

Program S. No. 10

```
[30]: print("ONLINE JOB MARKET ANALYTICS")
      print("-" * 35)

      print("Job Role:", JOB_ROLE)
      print("Total Jobs Analyzed:", len(city_df))
      print("Cities Analyzed:", len(location_analysis))

      print("\nLocation-wise Job Demand:")
      print(location_analysis.to_string(index=False))

      ONLINE JOB MARKET ANALYTICS
      -----
      Job Role: data scientist
      Total Jobs Analyzed: 60
      Cities Analyzed: 6

      Location-wise Job Demand:
      Location  Number of Jobs
      Bangalore          10

      Hyderabad          10
      Pune                10
      Chennai             10
      Mumbai              10
      Delhi               10

[31]: driver.quit()

      print("Browser closed successfully ")
```

Fig. 1.9: Final Job Market Analytics Summary

The final output confirms that the multi-city dataset and graph were saved successfully. The analysis includes 60 sampled job records across 6 cities and displays the location-wise summary.

Conclusions

The exercise successfully demonstrated Selenium-based web scraping on a dynamic job portal. Job title, company name, location and available salary information were extracted from Naukri, organized in Pandas DataFrames, cleaned and stored as CSV files. Location-wise analysis was performed for six cities and visualized using a Matplotlib bar chart. Thus, the required job listings dataset, CSV output, location analysis and demand visualization were generated successfully.

References

1. Michael Freeman and Joel Ross, Programming Skills for Data Science: Start Writing Code to Wrangle, Analyze, and Visualize Data with R, Addison-Wesley, 2018.
2. Python Program (GitHub link): <https://github.com/hasnainmakada-99/Data-Science-Codes/blob/main/Lab%20Codes/Week5-EP.ipynb>
3. Class lecture link: <https://drive.google.com/file/d/1HjCacfaFs49XEnuM3idduuKB0Tgjb4xI/view>

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Practice Program

Date: 20/08/2026

Week No.: 6

Program S. No. 11

Title	Interactive Bar Chart with Plotly			
Problem Statement	Create an interactive bar chart showing the population of different cities.			
Objectives	• To understand the creation of interactive data visualizations using Plotly. • To organize city population data in a Pandas DataFrame. • To create an interactive bar chart for comparing population across different cities. • To sort and analyze population data to identify the highest, lowest, and average values. • To save the interactive chart as an HTML file and the dataset as a CSV file.			
Dataset URL Link / File	City		Population (Lakhs)	
	Mumbai		204	
	Delhi		167	
	Bengaluru		136	
	Hyderabad		108	
	Chennai		115	
	Kolkata		149	
	Dataset source: Manually defined in the Jupyter Notebook for this exercise; no external dataset URL is required.			
Functions Details	Sl. No.	Function	Package	Description of function
	1	pd.DataFrame()	pandas	Creates a structured DataFrame from the city and population data.
	2	px.bar()	plotly.express	Creates an interactive bar chart from the DataFrame.
	3	update_traces()	plotly	Updates trace properties such as the position of value labels on the bars.
	4	update_layout()	plotly	Updates chart layout properties including axis titles.
	5	show()	plotly	Displays the interactive Plotly figure in the Jupyter Notebook.
	6	sort_values()	pandas	Sorts the DataFrame based on population values in ascending or descending order.
	7	idxmax()	pandas	Returns the index of the row containing

Student Reg. No: 26MML0031 Name: Hasnain Makada

Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT), Vellore

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Practice Program

Date: 20/08/2026

Week No.: 6

Program S. No. 11

				the maximum population value.
	8	idxmin()	pandas	Returns the index of the row containing the minimum population value.
	9	mean()	pandas	Calculates the average population of all cities.
	10	round()	Python	Rounds the calculated average population to the required decimal places.
	11	write_html()	plotly	Saves the interactive Plotly chart as an HTML file.
	12	to_csv()	pandas	Saves the DataFrame as a CSV file without the index column.
Steps/ Process/ Procedure/ Algorithm/ Flowchart/ Model	1. Import the Pandas library and Plotly Express for data handling and interactive visualization. 2. Create a dictionary containing the names of six Indian cities and their population values in lakhs. 3. Convert the dictionary into a Pandas DataFrame and display the dataset. 4. Check the dataset shape and print the city and population lists to verify the input data. 5. Create an interactive bar chart using Plotly Express with City on the X-axis and Population (Lakhs) on the Y-axis. 6. Display population values above the bars and provide appropriate chart and axis titles. 7. Sort the DataFrame in descending order of population using <code>sort_values()</code>. 8. Create and display a second interactive bar chart using the sorted population data. 9. Use <code>idxmax()</code> and <code>idxmin()</code> to identify the cities with the highest and lowest populations. 10. Calculate the average population using <code>mean()</code> and round the result to two decimal places. 11. Display the final population analysis summary in the notebook. 12. Save the interactive Plotly chart as <code>city_population_chart.html</code> and save the dataset as <code>city_population.csv</code>. Flow: START → Import Libraries → Create Dataset → DataFrame → Verify Data → Interactive Bar Chart → Sort Data → Sorted Chart → Population Statistics → Save HTML/CSV → STOP			
Program	import pandas as pd			

Student Reg. No: 26MML0031 Name: Hasnain Makada

Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT), Vellore

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Practice Program

Date: 20/08/2026

Week No.: 6

Program S. No. 11

```
import plotly.express as px

data = {
    "City": [
        "Mumbai",
        "Delhi",
        "Bengaluru",
        "Hyderabad",
        "Chennai",
        "Kolkata"
    ],
    "Population (Lakhs)": [
        204,
        167,
        136,
        108,
        115,
        149
    ]
}

df = pd.DataFrame(data)
df

print("Dataset Shape:", df.shape)
print("\nCITIES:")
print(df["City"].tolist())
print("\nPOPULATION:")
print(df["Population (Lakhs)"].tolist())

fig = px.bar(
    df,
    x="City",
    y="Population (Lakhs)",
    title="Population of Major Indian Cities",
    text="Population (Lakhs)"
)
fig.update_traces(textposition="outside")
fig.update_layout(
    xaxis_title="City",
    yaxis_title="Population (Lakhs)"
)
fig.show()

sorted_df = df.sort_values(
    by="Population (Lakhs)",
```

Student Reg. No: 26MML0031 Name: Hasnain Makada

Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT), Vellore

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Practice Program

Date: 20/08/2026

Week No.: 6

Program S. No. 11

```
        ascending=False
    )
    sorted_df

    fig = px.bar(
        sorted_df,
        x="City",
        y="Population (Lakhs)",
        title="City-wise Population Analysis",
        text="Population (Lakhs)"
    )
    fig.update_traces(textposition="outside")
    fig.update_layout(
        xaxis_title="City",
        yaxis_title="Population (Lakhs)"
    )
    fig.show()

    highest = df.loc[
        df["Population (Lakhs)"].idxmax()
    ]
    print("City with Highest Population:")
    print(highest)

    lowest = df.loc[
        df["Population (Lakhs)"].idxmin()
    ]
    print(
        "Lowest Population:",
        lowest["City"],
        "-",
        lowest["Population (Lakhs)"],
        "Lakhs"
    )

    average_population = df["Population (Lakhs)"].mean()
    print(
        "Average Population:",
        round(average_population, 2),
        "Lakhs"
    )

    print("CITY POPULATION ANALYSIS")
    print("-" * 35)
    print("Total Cities:", len(df))
    print(
```

Student Reg. No: 26MML0031 Name: Hasnain Makada

Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology
(VIT), Vellore

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Practice Program

Date: 20/08/2026

Week No.: 6

Program S. No. 11

```
"Highest Population:",
df.loc[df["Population (Lakhs)"].idxmax(), "City"]
)
print(
    "Lowest Population:",
    df.loc[df["Population (Lakhs)"].idxmin(), "City"]
)
print(
    "Average Population:",
    round(df["Population (Lakhs)"].mean(), 2),
    "Lakhs"
)

fig.write_html(
    "city_population_chart.html"
)
print("Interactive chart saved successfully.")

df.to_csv(
    "city_population.csv",
    index=False
)
print("Dataset saved successfully.")
```

**Output
Snapshots
With
Explanation/
Observations**

The following output snapshots are taken from the completed Jupyter Notebook. Each image is inserted at full content width so that the dataset, charts, code cells, and output values remain readable and are not cut off.

```
[5]:      City  Population (Lakhs)
0    Mumbai             204
1     Delhi             167
2  Bengaluru             136
3  Hyderabad             108
4     Chennai             115
5    Kolkata             149

[6]: print("Dataset Shape:", df.shape)

print("\nCities:")
print(df["City"].tolist())

print("\nPopulation:")
print(df["Population (Lakhs)"].tolist())

Dataset Shape: (6, 2)

Cities:
['Mumbai', 'Delhi', 'Bengaluru', 'Hyderabad', 'Chennai', 'Kolkata']

Population:
[204, 167, 136, 108, 115, 149]

[8]: fig = px.bar(
      df,
```

Fig. 1.1: Population Dataset and Dataset Structure

Student Reg. No: 26MML0031 Name: Hasnain Makada

Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT), Vellore

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Practice Program

Date: 20/08/2026

Week No.: 6

Program S. No. 11

Explanation/Observation: The DataFrame verifies 6 cities and their corresponding population values.

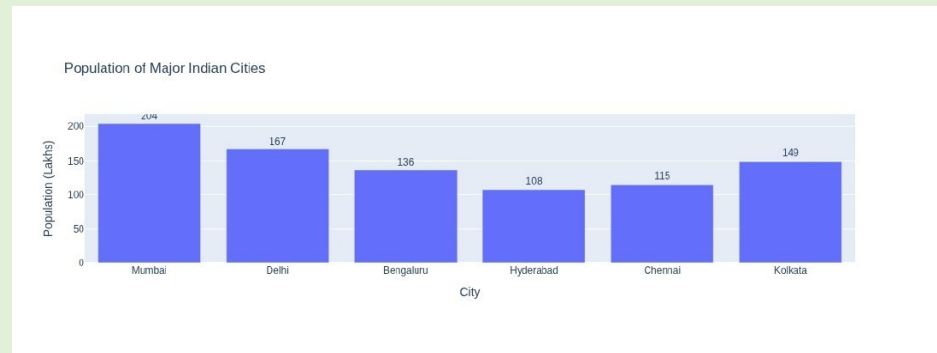


Fig. 1.2: Interactive Bar Chart of Major Indian Cities

Explanation/Observation: Plotly Express generates an interactive bar chart for the original city order. The population values are displayed above the bars, with Mumbai showing the highest value of 204 lakhs in the given dataset.

```
[9]: sorted_df = df.sort_values(
    by="Population (Lakhs)",
    ascending=False
)

sorted_df
```

	City	Population (Lakhs)
0	Mumbai	204
1	Delhi	167
5	Kolkata	149
2	Bengaluru	136
4	Chennai	115
3	Hyderabad	108

Fig. 1.3: Population Data Sorted in Descending Order

Explanation/Observation: The DataFrame is sorted using `sort_values()` in descending order of population. The resulting order is Mumbai, Delhi, Kolkata, Bengaluru, Chennai, and Hyderabad.



Fig. 1.4: City-wise Population Analysis after Sorting

Explanation/Observation: The sorted bar chart displays the cities from

Student Reg. No: 26MML0031 Name: Hasnain Makada

Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT), Vellore

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Practice Program

Date: 20/08/2026

Week No.: 6

Program S. No. 11

highest to lowest population, making comparison and ranking easier.

```
[11]: highest = df.loc[
      df["Population (Lakhs)"].idxmax()
    ]

print("City with Highest Population:")
print(highest)
```

```
City with Highest Population:
City                Mumbai
Population (Lakhs)    204
Name: 0, dtype: object
```

```
[12]: lowest = df.loc[
      df["Population (Lakhs)"].idxmin()
    ]

print(
    "Lowest Population:",
    lowest["City"],
    "-",
    lowest["Population (Lakhs)"],
    "Lakhs"
)
```

```
Lowest Population: Hyderabad - 108 Lakhs
```

```
[13]: average_population = df["Population (Lakhs)"].mean()

print(
    "Average Population:",
    round(average_population, 2),
    "Lakhs"
)
```

```
Average Population: 146.5 Lakhs
```

Fig. 1.5: Highest, Lowest, and Average Population Analysis

Explanation/Observation: The program identifies Mumbai as the city with the highest population (204 lakhs), Hyderabad as the city with the lowest population (108 lakhs), and calculates the average population as 146.5 lakhs.

Student Reg. No: 26MML0031 Name: Hasnain Makada

Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT), Vellore

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Practice Program

Date: 20/08/2026

Week No.: 6

Program S. No. 11

```
[14]: print("CITY POPULATION ANALYSIS")
      print("-" * 35)

      print("Total Cities:", len(df))

      print(
          "Highest Population:",
          df.loc[df["Population (Lakhs)"].idxmax(), "City"]
      )

      print(
          "Lowest Population:",
          df.loc[df["Population (Lakhs)"].idxmin(), "City"]
      )

      print(
          "Average Population:",
          round(df["Population (Lakhs)"].mean(), 2),
          "Lakhs"
      )

CITY POPULATION ANALYSIS
-----
Total Cities: 6
Highest Population: Mumbai
Lowest Population: Hyderabad
Average Population: 146.5 Lakhs

[15]: fig.write_html(
      "city_population_chart.html"
      )

      print("Interactive chart saved successfully.")

Interactive chart saved successfully.

[16]: df.to_csv(
      "city_population.csv",
      index=False
      )

      print("Dataset saved successfully.")

Dataset saved successfully.

[ ]:
```

Fig. 1.6: Final Analysis Summary and File Saving Output

Explanation/Observation: The final summary confirms 6 cities were analyzed, with Mumbai highest and Hyderabad lowest. The interactive chart and CSV dataset are saved successfully.

Conclusions	The experiment was successfully completed using Pandas and Plotly. The population data was organized in a DataFrame, visualized with interactive bar charts, and sorted for comparison. Mumbai had the highest population, Hyderabad the lowest, and the average population was 146.5 lakhs. The interactive chart was saved as HTML and the dataset as CSV.
References	1. Michael Freeman and Joel Ross, Programming Skills for Data Science, Addison-Wesley, 2018. 2. Python Program (GitHub link): https://github.com/hasnainmakada-99/Data-Science-Codes/blob/main/WEEK%206/Week-6-PP.ipynb 3. Class lecture link: Open Lecture File

Student Reg. No: 26MML0031 Name: Hasnain Makada

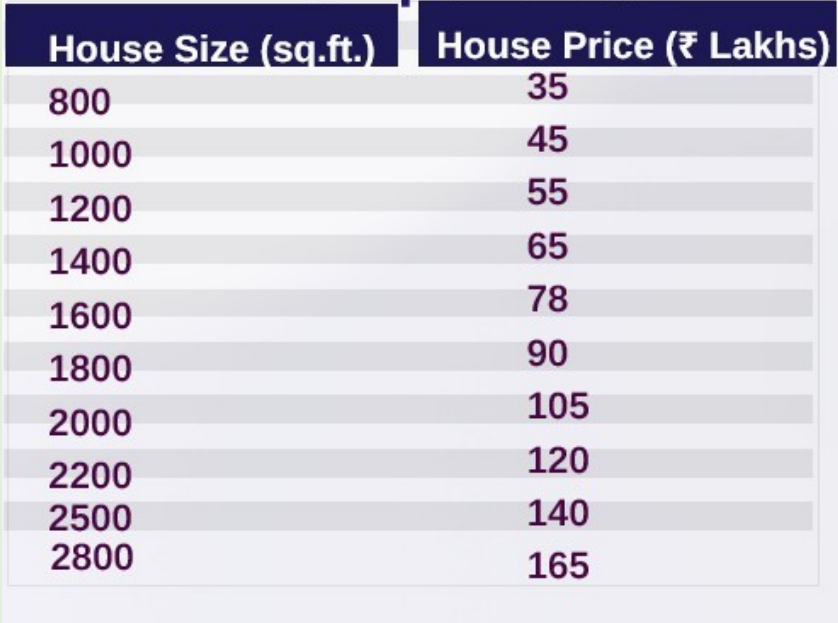
Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT), Vellore

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Practice Program

Date: 20/08/2026

Week No.: 6

Program S. No. 12

Title	Interactive Scatter Plot with Plotly			
Problem Statement	Create an interactive scatter plot showing the relationship between house size and price.			
Objectives	• To understand interactive scatter plot visualization using Plotly. • To organize house size and house price data using a Pandas DataFrame. • To visualize and interpret the relationship between house size and house price. • To compute descriptive measures such as minimum, maximum, average, and correlation. • To create a trendline for the relationship and save the interactive chart and dataset for further use.			
Dataset URL Link / File	 Dataset source: Manually defined in the Jupyter Notebook using the house-size and house-price values shown above; no external dataset URL is required.			
Functions Details	Sl. No.	Function	Package	Description of function
	1	pd.DataFrame()	pandas	Creates a structured DataFrame from the house size and price data.
	2	px.scatter()	plotly.express	Creates an interactive scatter plot from the DataFrame.
	3	update_traces()	plotly	Updates marker size and text-position properties of the plotted points.

Student Reg. No: 26MML0031 Name: Hasnain Makada

Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT), Vellore

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Practice Program

Date: 20/08/2026

Week No.: 6

Program S. No. 12

	4	update_layout()	plotly	Updates the chart layout and axis titles.
	5	show()	plotly	Displays the interactive Plotly chart inside the Jupyter Notebook.
	6	sort_values()	pandas	Sorts the dataset based on house size in ascending order.
	7	idxmax()	pandas	Returns the index of the record with the maximum house price.
	8	idxmin()	pandas	Returns the index of the record with the minimum house price.
	9	mean()	pandas	Calculates the average house price and average house size.
	10	corr()	pandas	Calculates the correlation between house size and house price.
	11	write_html()	plotly	Saves the interactive Plotly scatter plot as an HTML file.
	12	to_csv()	pandas	Saves the house-price DataFrame as a CSV file without the index column.
Steps/ Process/ Procedure/ Algorithm/ Flowchart/ Model		1. Import Pandas and Plotly Express for tabular data handling and interactive visualization. 2. Create a dictionary containing house sizes in square feet and corresponding house prices in lakhs. 3. Convert the dictionary into a Pandas DataFrame and display all ten records. 4. Verify the dataset shape and print the complete house-size and house-price lists. 5. Create a basic interactive scatter plot with house size on the X-axis and house price on the Y-axis. 6. Create an improved scatter plot with larger markers, value labels, title, and proper axis labels. 7. Sort the DataFrame by house size in ascending order and display the sorted dataset. 8. Use idxmax() and idxmin() to identify the most expensive and least expensive houses. 9. Calculate the average house price and average house size using mean(). 10. Calculate the correlation between house size and price using corr() to measure the strength of their relationship. 11. Create a scatter plot with an OLS trendline to visually represent the positive relationship between size and price.		

Student Reg. No: 26MML0031 Name: Hasnain Makada

Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT), Vellore

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Practice Program

Date: 20/08/2026

Week No.: 6

Program S. No. 12

	12. Display the final house-price analysis and save the interactive chart as HTML and the dataset as CSV. Flow: START → Import Libraries → Create Dataset → DataFrame → Verify Data → Scatter Plot → Enhanced Plot → Sort Data → Price Statistics → Average Values → Correlation → Trendline → Save HTML/CSV → STOP
Program	<pre>import pandas as pd import plotly.express as px data = { "House Size (sq.ft)": [800, 1000, 1200, 1400, 1600, 1800, 2000, 2200, 2500, 2800], "House Price (Lakhs)": [35, 45, 55, 65, 78, 90, 105, 120, 140, 165] } df = pd.DataFrame(data) df print("Dataset Shape:", df.shape) print("\nHouse Sizes:") print(df["House Size (sq.ft.)"].tolist()) print("\nHouse Prices:") print(df["House Price (Lakhs)"].tolist())</pre>

Student Reg. No: 26MML0031 Name: Hasnain Makada

Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT), Vellore

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Practice Program

Date: 20/08/2026

Week No.: 6

Program S. No. 12

```
fig = px.scatter(
    df,
    x="House Size (sq.ft.)",
    y="House Price (Lakhs)",
    title="House Size vs House Price"
)
fig.show()

fig = px.scatter(
    df,
    x="House Size (sq.ft.)",
    y="House Price (Lakhs)",
    title="Relationship Between House Size and Price",
    text="House Price (Lakhs)"
)
fig.update_traces(
    marker=dict(size=12),
    textposition="top center"
)
fig.update_layout(
    xaxis_title="House Size (sq.ft.)",
    yaxis_title="House Price (Lakhs)"
)
fig.show()

sorted_df = df.sort_values(
    by="House Size (sq.ft.)",
    ascending=True
)
sorted_df

highest = df.loc[
    df["House Price (Lakhs)"].idxmax()
]
print("Most Expensive House:")
print(highest)

lowest = df.loc[
    df["House Price (Lakhs)"].idxmin()
]
print(
    "Least Expensive House:",
    lowest["House Size (sq.ft.)"],
    "sq.ft. - ",
    lowest["House Price (Lakhs)"],
```

Student Reg. No: 26MML0031 Name: Hasnain Makada

Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology
(VIT), Vellore

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Practice Program

Date: 20/08/2026

Week No.: 6

Program S. No. 12

```
"Lakhs"
)

average_price = df["House Price (Lakhs)"].mean()
print(
    "Average House Price:",
    round(average_price, 2),
    "Lakhs"
)

average_size = df["House Size (sq.ft.)"].mean()
print(
    "Average House Size:",
    round(average_size, 2),
    "sq.ft."
)

correlation = df[
    "House Size (sq.ft.)"
].corr(
    df["House Price (Lakhs)"]
)
print(
    "Correlation between House Size and Price:",
    round(correlation, 3)
)

fig = px.scatter(
    df,
    x="House Size (sq.ft.)",
    y="House Price (Lakhs)",
    title="House Size vs Price with Trendline",
    trendline="ols"
)
fig.update_traces(
    marker=dict(size=12)
)
fig.update_layout(
    xaxis_title="House Size (sq.ft.)",
    yaxis_title="House Price (Lakhs)"
)
fig.show()

print("HOUSE PRICE ANALYSIS")
print("-" * 40)
print("Total Houses:", len(df))
```

Student Reg. No: 26MML0031 Name: Hasnain Makada

Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology
(VIT), Vellore

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Practice Program

Date: 20/08/2026

Week No.: 6

Program S. No. 12

	<pre> print("Smallest House:", df["House Size (sq.ft.)"].min(), "sq.ft.") print("Largest House:", df["House Size (sq.ft.)"].max(), "sq.ft.") print("Lowest Price: ", df["House Price (Lakhs)"].min(), "Lakhs") print("Highest Price: ", df["House Price (Lakhs)"].max(), "Lakhs") print("Average Price: ", round(df["House Price (Lakhs)"].mean(), 2), "Lakhs") print("Correlation:", round(correlation, 3)) fig.write_html("house_size_price_scatter.html") print("Interactive scatter plot saved successfully.") df.to_csv("house_price_data.csv", index=False) print("Dataset saved successfully.") </pre>
Output Snapshots With Explanation/ Observations	The following snapshots are taken from the completed Jupyter Notebook. The images retain the complete visible code/output blocks from each source page; only unused white margins were removed so no program output is cut off.

Student Reg. No: 26MML0031 Name: Hasnain Makada

Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT), Vellore

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Practice Program

Date: 20/08/2026

Week No.: 6

Program S. No. 12

```
165
    ]
}

[3]: df = pd.DataFrame(data)

df

[3]:   House Size (sq.ft.)  House Price (Lakhs)
0                800             35
1               1000             45
2               1200             55
3               1400             65
4               1600             78
5               1800             90
6               2000            105
7               2200            120
8               2500            140
9               2800            165

[4]: print("Dataset Shape:", df.shape)

print("\nHouse Sizes:")
print(df["House Size (sq.ft.)"].tolist())

print("\nHouse Prices:")
print(df["House Price (Lakhs)"].tolist())

Dataset Shape: (10, 2)

House Sizes:
[800, 1000, 1200, 1400, 1600, 1800, 2000, 2200, 2500, 2800]

House Prices:
[35, 45, 55, 65, 78, 90, 105, 120, 140, 165]

[5]: fig = px.scatter(
    df,
    x="House Size (sq.ft.)",
    y="House Price (Lakhs)",
    title="House Size vs House Price"
)

fig.show()
```

2

Fig. 1.1: DataFrame Creation, Dataset Verification, and Basic Scatter Plot Code

Explanation/Observation: The dataset is converted into a Pandas DataFrame with 10 rows and 2 columns. The complete house-size and house-price lists are verified, and the code for the initial interactive scatter plot is executed.

Student Reg. No: 26MML0031 Name: Hasnain Makada

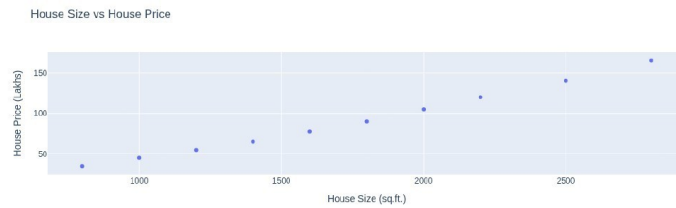
Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT), Vellore

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Practice Program

Date: 20/08/2026

Week No.: 6

Program S. No. 12



```
[6]: fig = px.scatter(  
    df,  
    x="House Size (sq.ft.)",  
    y="House Price (Lakhs)",  
    title="Relationship Between House Size and Price",  
    text="House Price (Lakhs)"  
)  
  
fig.update_traces(  
    marker=dict(size=12),  
    textposition="top center"  
)  
  
fig.update_layout(  
    xaxis_title="House Size (sq.ft.)",  
    yaxis_title="House Price ( Lakhs)"  
)  
  
fig.show()
```



3

Fig. 1.2: Interactive and Enhanced Scatter Plots

Explanation/Observation: The first scatter plot shows house size versus price. The enhanced Plotly chart adds larger markers and price labels, making the strong upward relationship between size and price visually clear.

Student Reg. No: 26MML0031 Name: Hasnain Makada

Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT), Vellore

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Practice Program

Date: 20/08/2026

Week No.: 6

Program S. No. 12

```
[7]: sorted_df = df.sort_values(
      by="House Size (sq.ft.)",
      ascending=True
    )

sorted_df

[7]:   House Size (sq.ft.)  House Price (Lakhs)
0              800             35
1             1000             45
2             1200             55
3             1400             65
4             1600             78
5             1800             90
6             2000            105
7             2200            120
8             2500            140
9             2800            165

[8]: highest = df.loc[
      df["House Price (Lakhs)"].idxmax()
    ]

print("Most Expensive House:")
print(highest)

Most Expensive House:
House Size (sq.ft.)    2800
House Price (Lakhs)    165
Name: 9, dtype: int64

[9]: lowest = df.loc[
      df["House Price (Lakhs)"].idxmin()
    ]

print(
    "Least Expensive House:",
    lowest["House Size (sq.ft.)"],
    "sq.ft. - ",
    lowest["House Price (Lakhs)"],
    "Lakhs"
)

Least Expensive House: 800 sq.ft. - 35 Lakhs

[10]: average_price = df["House Price (Lakhs)"].mean()

print()
```

4

Fig. 1.3: Sorted Dataset and Extreme Price Analysis

Explanation/Observation: The dataset is sorted by house size in ascending order. The program identifies the 2800 sq.ft. house priced at 165 lakhs as the most expensive and the 800 sq.ft. house priced at 35 lakhs as the least expensive.

Student Reg. No: 26MML0031 Name: Hasnain Makada

Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT), Vellore

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Practice Program

Date: 20/08/2026

Week No.: 6

Program S. No. 12

```
"Average House Price:",
round(average_price, 2),
"Lakhs"
)

Average House Price: 89.8 Lakhs

[11]: average_size = df["House Size (sq.ft.)"].mean()

print(
    "Average House Size:",
    round(average_size, 2),
    "sq.ft."
)

Average House Size: 1730.0 sq.ft.

[12]: correlation = df[
    "House Size (sq.ft.)"
].corr(
    df["House Price (Lakhs)"]
)

print(
    "Correlation between House Size and Price:",
    round(correlation, 3)
)

Correlation between House Size and Price: 0.996

[14]: fig = px.scatter(
    df,
    x="House Size (sq.ft.)",
    y="House Price (Lakhs)",
    title="House Size vs Price with Trendline",
    trendline="ols"
)

fig.update_traces(
    marker=dict(size=12)
)

fig.update_layout(
    xaxis_title="House Size (sq.ft.)",
    yaxis_title="House Price ( Lakhs)"
)

fig.show()
```

5

Fig. 1.4: Average Values, Correlation, and Trendline Program

Explanation/Observation: The average house price is 89.8 lakhs and the average house size is 1730 sq.ft. The calculated correlation is 0.996, indicating a very strong positive relationship. The trendline scatter-plot code is also shown.

Student Reg. No: 26MML0031 Name: Hasnain Makada

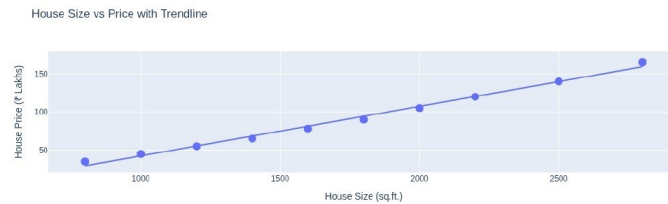
Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT), Vellore

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Practice Program

Date: 20/08/2026

Week No.: 6

Program S. No. 12



```
[15]: print("HOUSE PRICE ANALYSIS")
      print("-" * 40)

      print("Total Houses:", len(df))

      print(
        "Smallest House:",
        df["House Size (sq.ft.)"].min(),
        "sq.ft."
      )

      print(
        "Largest House:",
        df["House Size (sq.ft.)"].max(),
        "sq.ft."
      )

      print(
        "Lowest Price: ",
        df["House Price (Lakhs)"].min(),
        "Lakhs"
      )

      print(
        "Highest Price: ",
        df["House Price (Lakhs)"].max(),
        "Lakhs"
      )

      print(
        "Average Price: ",
        round(df["House Price (Lakhs)"].mean(), 2),
        "Lakhs"
      )
```

6

Fig. 1.5: Scatter Plot with OLS Trendline and Final Analysis Code

Explanation/Observation: The Plotly scatter plot with an OLS trendline confirms that house price rises strongly with house size. The final analysis code computes the smallest/largest houses and lowest/highest/average prices.

Student Reg. No: 26MML0031 Name: Hasnain Makada

Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT), Vellore

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Practice Program

Date: 20/08/2026

Week No.: 6

Program S. No. 12

```
)

print(
    "Correlation:",
    round(correlation, 3)
)

HOUSE PRICE ANALYSIS
-----

Total Houses: 10
Smallest House: 800 sq.ft.
Largest House: 2800 sq.ft.
Lowest Price: 35 Lakhs
Highest Price: 165 Lakhs
Average Price: 89.8 Lakhs
Correlation: 0.996

[16]: fig.write_html(
        "house_size_price_scatter.html"
    )

print("Interactive scatter plot saved successfully.")

Interactive scatter plot saved successfully.

[17]: df.to_csv(
        "house_price_data.csv",
        index=False
    )

print("Dataset saved successfully.")

Dataset saved successfully.

[ ]:
```

7

Fig. 1.6: Final House Price Analysis and File Saving Output

Explanation/Observation: The final output confirms 10 houses, sizes from 800 to 2800 sq.ft., prices from 35 to 165 lakhs, an average price of 89.8 lakhs, and correlation of 0.996. The interactive HTML chart and CSV dataset are saved successfully.

Conclusions

The experiment was successfully completed using Pandas and Plotly. House size and price data were organized in a Pandas DataFrame and visualized through interactive scatter plots. The analysis found an average house size of 1730 sq.ft., an average price of 89.8 lakhs, and a correlation of 0.996, showing a very strong positive relationship between house size and price. An OLS trendline further confirmed this relationship. The interactive

Student Reg. No: 26MML0031 Name: Hasnain Makada

Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT), Vellore

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Practice Program

Date: 20/08/2026

Week No.: 6

Program S. No. 12

	chart and dataset were successfully saved as HTML and CSV files.
References	1. Michael Freeman and Joel Ross, Programming Skills for Data Science: Start Writing Code to Wrangle, Analyze, and Visualize Data with R, Addison-Wesley, 2018. 2. Python Program (GitHub link): https://github.com/hasnainmakada-99/Data-Science-Codes/blob/main/WEEK%206/Week-6-PP2.ipynb 3. Class lecture link: Open Lecture File

Student Reg. No: 26MML0031 Name: Hasnain Makada

Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT), Vellore

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Practice Program

Date: 27/08/2026

Week No.: 7

Program S. No. 13

Title	Interactive Scatter Plot with Bokeh			
Problem Statement	Create an interactive scatter plot showing the relationship between petal length and petal width from the Iris dataset using Bokeh.			
Objectives	• To understand interactive scatter plot visualization using Bokeh. • To load and organize the Iris dataset using Pandas and scikit-learn. • To visualize the relationship between petal length and petal width. • To differentiate Iris species using separate colors and an interactive legend. • To use Bokeh hover, zoom, pan, reset and save tools for interactive analysis.			
Dataset URL Link / File	Dataset: Built-in Iris dataset loaded using sklearn.datasets.load_iris(). Selected attributes: Petal Length (cm), Petal Width (cm), and Species.			
	Species		Samples	
	setosa		50	
	versicolor		50	
	virginica		50	
Functions Details	Sl. No.	Function	Package	Description of function
	1	load_iris()	sklearn	Loads the built-in Iris dataset.
	2	rename()	Pandas	Renames dataset columns for clearer labels.
	3	map()	Pandas	Maps numeric target values to species names.
	4	DataFrame()	Pandas	Represents the selected Iris attributes in tabular form.
	5	ColumnData Source()	Bokeh	Creates a Bokeh-friendly data source.
	6	figure()	Bokeh	Creates the interactive plotting canvas.
	7	scatter()	Bokeh	Plots petal length against petal width.
	8	HoverTool()	Bokeh	Displays species and measurements on hover.
	9	add_tools()	Bokeh	Adds the hover tool to the Bokeh figure.
	10	value_counts()	Pandas	Counts the records belonging to each Iris species.
	11	corr()	Pandas	Calculates correlation between petal length and width.
	12	output_file()/save()	Bokeh	Saves the interactive visualization as an HTML file.

Student Reg. No: 26MML0031 Name: Hasnain Makada

Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT), Vellore

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Practice Program

Date: 27/08/2026

Week No.: 7

Program S. No. 13

Steps/ Process/ Procedure/ Algorithm/ Flowchart/ Model	1. Import Pandas, the Iris dataset utility, and required Bokeh modules. 2. Load the Iris dataset and rename the petal measurement columns for clarity. 3. Convert numeric target values into the species names setosa, versicolor and virginica. 4. Create a DataFrame containing Petal Length, Petal Width and Species. 5. Display sample records, dataset shape, species counts and correlation for verification. 6. Create a Bokeh figure with interactive pan, zoom, reset and save tools. 7. Plot one scatter series for each Iris species using a ColumnDataSource. 8. Add HoverTool so that species and measurements appear when the pointer is placed over a point. 9. Enable an interactive legend so species can be hidden or displayed. 10. Save the interactive Bokeh chart as an HTML file and verify successful execution. Flow: START -> Load Iris Dataset -> Prepare DataFrame -> ColumnDataSource -> Bokeh Figure -> Scatter Plot -> Hover/Legend -> Save HTML -> STOP
Program	<pre>import pandas as pd from sklearn.datasets import load_iris from bokeh.plotting import figure, output_file, save from bokeh.models import ColumnDataSource, HoverTool from bokeh.palettes import Category10 # Load the Iris dataset iris = load_iris(as_frame=True) df = iris.frame.rename(columns={ "petal length (cm)": "Petal Length (cm)", "petal width (cm)": "Petal Width (cm)" }) df["Species"] = df["target"].map(dict(enumerate(iris.target_names))) df = df[["Petal Length (cm)", "Petal Width (cm)", "Species"]] print(df.head(10)) print("Dataset Shape:", df.shape) print("Species Counts:") print(df["Species"].value_counts()) correlation = df["Petal Length (cm)"].corr(df["Petal Width (cm)"]) print("Correlation:", round(correlation, 3)) # Create the interactive Bokeh figure p = figure(title="Interactive Scatter Plot: Iris Petal Length vs Petal Width", x_axis_label="Petal Length (cm)", y_axis_label="Petal Width (cm)",</pre>

Student Reg. No: 26MML0031 Name: Hasnain Makada

Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT), Vellore

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Practice Program

Date: 27/08/2026

Week No.: 7

Program S. No. 13

```
width=900,
height=520,
tools="pan,wheel_zoom,box_zoom,reset,save"
)

colors = Category10[3]
for color, species in zip(colors, iris.target_names):
    subset = df[df["Species"] == species]
    source = ColumnDataSource(subset)
    p.scatter(
        x="Petal Length (cm)",
        y="Petal Width (cm)",
        source=source,
        size=9,
        color=color,
        alpha=0.75,
        legend_label=species
    )

hover = HoverTool(tooltips=[
    ("Species", "@Species"),
    ("Petal Length", "@{Petal Length (cm)}{0.0} cm"),
    ("Petal Width", "@{Petal Width (cm)}{0.0} cm")
])
p.add_tools(hover)
p.legend.title = "Species"
p.legend.location = "top_left"
p.legend.click_policy = "hide"

output_file("program13_iris_scatter.html")
save(p)
print("Interactive scatter plot saved successfully.")
```

**Output
Snapshots
With
Explanation/
Observations**

The following outputs were generated by executing the complete Program 13 code. The static chart image represents the interactive Bokeh visualization; the HTML file retains hover, zoom, pan and reset functionality.

Jupyter Output - Iris Dataset Summary

```
First 10 rows: Petal Length (cm), Petal Width (cm), Species
1.4  0.2  setosa
1.4  0.2  setosa
1.3  0.2  setosa
1.5  0.2  setosa
1.4  0.2  setosa
...
Dataset Shape: (150, 3)
Species Counts: setosa 50 | versicolor 50 | virginica 50
Correlation (Petal Length vs Petal Width): 0.963
```

Fig. 1.1: Iris Dataset and Correlation Summary

Student Reg. No: 26MML0031 Name: Hasnain Makada

Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT), Vellore

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Practice Program

Date: 27/08/2026

Week No.: 7

Program S. No. 13

The output verifies that 150 Iris samples were loaded, with 50 records for each species. The petal length-petal width correlation is approximately 0.963, indicating a strong positive relationship.

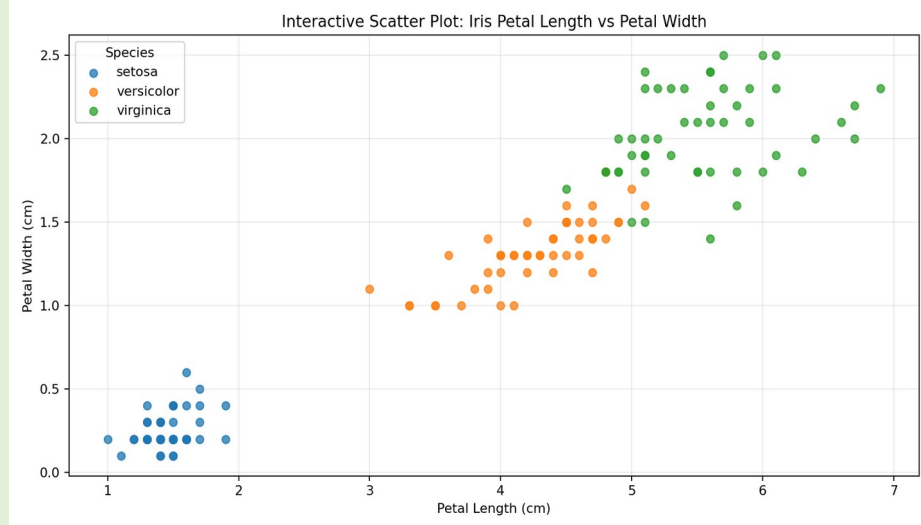


Fig. 1.2: Interactive Scatter Plot Output

The scatter plot shows three clearly distinguishable Iris species. Setosa forms a compact cluster at lower petal measurements, while versicolor and virginica occupy progressively larger petal-length and petal-width ranges.

Jupyter Output - Program 13 Final Status

```
Interactive scatter plot created successfully using Bokeh.
HTML file saved: program13_iris_scatter.html
Total records visualized: 150
Species displayed: setosa, versicolor, virginica
Hover, zoom, pan, reset and save tools are enabled.
```

Fig. 1.3: Program 13 Execution and HTML Save Confirmation

The final execution confirms successful generation of the interactive Bokeh scatter plot and its HTML output. Hover, zoom, pan, reset and save tools are available in the saved interactive file.

Conclusions

The experiment was completed successfully using Pandas, scikit-learn and Bokeh. The Iris dataset was loaded and organized into a DataFrame containing petal length, petal width and species. The interactive scatter plot clearly demonstrated a strong positive relationship between petal length and petal width and visually separated the three Iris species. Bokeh interaction tools further enabled hovering, zooming, panning and legend-based filtering, making the visualization suitable for exploratory analysis.

References

1. Bokeh Documentation: [Open Link](#)
2. Pandas Documentation: [Open Link](#)
3. scikit-learn Iris Dataset Documentation: [Open Link](#)
4. Python Program (GitHub link):
5. Class lecture link: [Open Lecture Link](#)

Student Reg. No: 26MML0031 Name: Hasnain Makada

Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT), Vellore

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Exercise Program

Date: 27/08/2026

Week No.: 7

Program S. No. 14

Title	Interactive Line Plot with Bokeh			
Problem Statement	Create an interactive line plot showing the daily temperatures over a week using Bokeh.			
Objectives	• To understand interactive line visualization using Bokeh. • To organize daily temperature values in a Pandas DataFrame. • To visualize day-wise temperature changes over one week. • To identify the highest, lowest and average weekly temperature. • To use hover, zoom, pan, reset and save tools for interactive time-series analysis.			
Dataset URL Link / File	Dataset: Manually defined weekly temperature readings for seven consecutive days.			
	Day		Temperature (°C)	
	Monday		29	
	Tuesday		31	
	Wednesday		30	
	Thursday		32	
	Friday		34	
	Saturday		33	
	Sunday		31	
Functions Details	Sl. No.	Function	Package	Description of function
	1	DataFrame()	Pandas	Creates a tabular dataset from the day and temperature values.
	2	ColumnData Source()	Bokeh	Converts the DataFrame into a Bokeh data source.
	3	figure()	Bokeh	Creates the interactive Bokeh plotting canvas.
	4	line()	Bokeh	Draws the line connecting daily temperatures.
	5	scatter()	Bokeh	Adds visible markers for each day.
	6	HoverTool()	Bokeh	Displays day and temperature values on hover.
	7	add_tools()	Bokeh	Adds the hover tool to the figure.
	8	idxmax()	Pandas	Locates the day with the maximum temperature.
	9	idxmin()	Pandas	Locates the day with the minimum temperature.
	10	mean()	Pandas	Calculates the average weekly temperature.
	11	output_file()	Bokeh	Defines the HTML output filename.
	12	save()	Bokeh	Saves the interactive line plot as an HTML file.

Student Reg. No: 26MML0031 Name: Hasnain Makada

Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT), Vellore

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Exercise Program

Date: 27/08/2026

Week No.: 7

Program S. No. 14

Steps/ Process/ Procedure/ Algorithm/ Flowchart/ Model	1. Import Pandas and the required Bokeh modules. 2. Define the seven days of the week and their corresponding temperature values. 3. Create a Pandas DataFrame and display it for verification. 4. Calculate the highest, lowest and average temperature values. 5. Convert the DataFrame into a Bokeh ColumnDataSource. 6. Create a Bokeh figure using the days as categorical X-axis values. 7. Plot the temperature trend using a line and add markers for each day. 8. Add HoverTool to display the exact day and temperature interactively. 9. Save the completed visualization as an HTML file and verify successful execution. Flow: START -> Create Weekly Dataset -> DataFrame -> Temperature Statistics -> ColumnDataSource -> Bokeh Figure -> Line + Markers -> Hover -> Save HTML -> STOP
Program	<pre>import pandas as pd from bokeh.plotting import figure, output_file, save from bokeh.models import ColumnDataSource, HoverTool # Weekly temperature dataset days = ["Monday", "Tuesday", "Wednesday", "Thursday", "Friday", "Saturday", "Sunday"] temperatures = [29, 31, 30, 32, 34, 33, 31] df = pd.DataFrame({"Day": days, "Temperature (°C)": temperatures}) print(df) highest_row = df.loc[df["Temperature (°C)"].idxmax()] lowest_row = df.loc[df["Temperature (°C)"].idxmin()] average_temp = df["Temperature (°C)"].mean() print("Highest Temperature:", highest_row["Day"], "-", highest_row["Temperature (°C)"], "°C") print("Lowest Temperature:", lowest_row["Day"], "-", lowest_row["Temperature (°C)"], "°C") print("Average Temperature:", round(average_temp, 2), "°C") source = ColumnDataSource(df) p = figure(x_range=days, title="Interactive Line Plot: Daily Temperature Over a Week", x_axis_label="Day", y_axis_label="Temperature (°C)", width=900, height=520, tools="pan,wheel_zoom,box_zoom,reset,save")</pre>

Student Reg. No: 26MML0031 Name: Hasnain Makada

Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT), Vellore

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Exercise Program

Date: 27/08/2026

Week No.: 7

Program S. No. 14

```
p.line(x="Day", y="Temperature (°C)", source=source, line_width=3)
p.scatter(x="Day", y="Temperature (°C)", source=source, size=10)

hover = HoverTool(tooltips=[
    ("Day", "@Day"),
    ("Temperature", "@{Temperature (°C)} °C")
])
p.add_tools(hover)
p.xgrid.grid_line_color = None

output_file("program14_temperature_line.html")
save(p)
print("Interactive line plot saved successfully.")
```

**Output
Snapshots
With
Explanation/
Observations**

The following outputs were generated from the completed Program 14. The chart image is a static view of the interactive Bokeh line plot, while the saved HTML output contains the full interactive tools.

Jupyter Output - Weekly Temperature Summary

Weekly Temperature Dataset:
Monday 29 °C | Tuesday 31 °C | Wednesday 30 °C
Thursday 32 °C | Friday 34 °C | Saturday 33 °C | Sunday 31 °C
Highest Temperature: Friday - 34 °C
Lowest Temperature: Monday - 29 °C
Average Temperature: 31.43 °C

Fig. 2.1: Weekly Temperature Dataset and Statistical Summary

The dataset contains seven daily readings. Friday records the highest temperature of 34 °C, Monday records the lowest temperature of 29 °C, and the weekly average is approximately 31.43 °C.

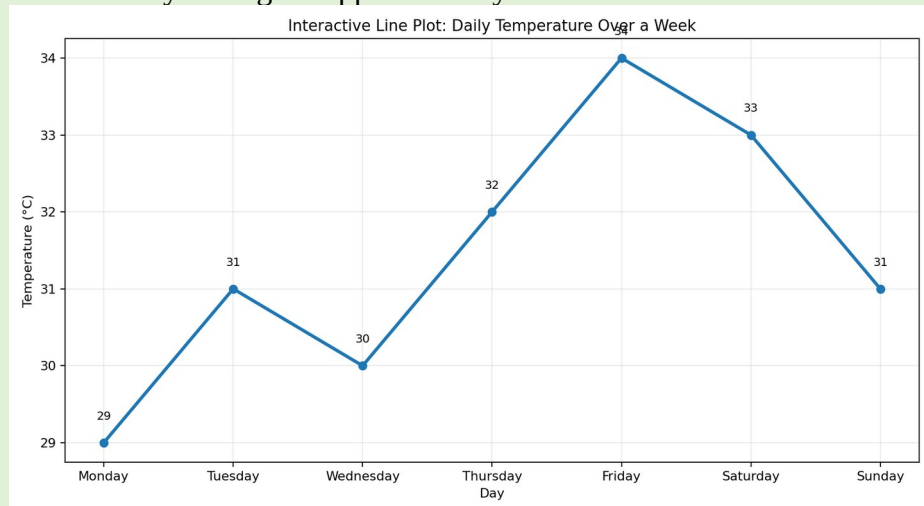


Fig. 2.2: Interactive Line Plot Output

The line plot shows the day-wise temperature variation across the week. The temperature rises overall from Monday to Friday, reaches its maximum on Friday, and decreases during the weekend.

Student Reg. No: 26MML0031 Name: Hasnain Makada

Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT), Vellore

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Exercise Program

Date: 27/08/2026

Week No.: 7

Program S. No. 14

	Jupyter Output - Program 14 Final Status <pre>Interactive line plot created successfully using Bokeh. HTML file saved: program14_temperature_line.html Days visualized: 7 Hover, zoom, pan, reset and save tools are enabled.</pre> Fig. 2.3: Program 14 Execution and HTML Save Confirmation The execution confirms successful creation and saving of the Bokeh line plot. The resulting HTML file supports hover, zoom, pan, reset and save interactions.
Conclusions	The exercise was completed successfully using Pandas and Bokeh. Daily temperature values for one week were organized in a DataFrame and visualized using an interactive line plot. Statistical analysis identified Friday as the hottest day, Monday as the coolest day, and the average weekly temperature as approximately 31.43 °C. The interactive Bokeh output makes the weekly trend easy to inspect using hover and navigation tools.
References	1. Bokeh Documentation: Open Link 2. Pandas Documentation: Open Link 3. Python Program (GitHub link): 4. Class lecture link: Open Lecture Link

Student Reg. No: 26MML0031 Name: Hasnain Makada

Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT), Vellore

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Exercise Program

Date: 27/08/2026

Week No.: 7

Program S. No. 15

Title	Interactive Bar Chart with Bokeh			
Problem Statement	Create an interactive bar chart showing the sales figures of different products using Bokeh.			
Objectives	• To understand interactive categorical bar-chart visualization using Bokeh. • To organize product-wise sales values in a Pandas DataFrame. • To compare sales performance across different products. • To calculate total, highest and average product sales. • To use Bokeh hover, zoom, pan, reset and save tools for interactive analysis.			
Dataset URL Link / File	Dataset: Manually defined product-wise sales values for six products.			
	Product		Sales (Units)	
	Laptop		125	
	Smartphone		210	
	Tablet		95	
	Headphones		170	
	Smartwatch		140	
	Camera		80	
Functions Details	Sl. No.	Function	Package	Description of function
	1	DataFrame()	Pandas	Creates a structured product-sales DataFrame.
	2	ColumnData Source()	Bokeh	Converts the DataFrame into a Bokeh data source.
	3	figure()	Bokeh	Creates the categorical plotting canvas.
	4	vbar()	Bokeh	Draws one vertical bar for each product.
	5	HoverTool()	Bokeh	Displays product and sales values on hover.
	6	add_tools()	Bokeh	Adds the hover tool to the figure.
	7	idxmax()	Pandas	Locates the product with the highest sales.
	8	max()	Pandas	Returns the highest sales value.
	9	sum()	Pandas	Calculates the total number of units sold.
	10	mean()	Pandas	Calculates the average sales per product.
	11	output_file()	Bokeh	Defines the interactive HTML output filename.
	12	save()	Bokeh	Saves the completed Bokeh bar chart as HTML.

Student Reg. No: 26MML0031 Name: Hasnain Makada

Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT), Vellore

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Exercise Program

Date: 27/08/2026

Week No.: 7

Program S. No. 15

Steps/ Process/ Procedure/ Algorithm/ Flowchart/ Model	1. Import Pandas and the required Bokeh modules. 2. Define the product names and their corresponding sales figures. 3. Create and display a Pandas DataFrame for verification. 4. Calculate total sales, the highest-selling product and average sales. 5. Convert the DataFrame to a Bokeh ColumnDataSource. 6. Create a categorical Bokeh figure with product names on the X-axis. 7. Use vbar() to display the product-wise sales values. 8. Add HoverTool so exact sales values are visible interactively. 9. Save the completed interactive bar chart as an HTML file and verify successful execution. Flow: START -> Create Product Dataset -> DataFrame -> Sales Statistics -> ColumnDataSource -> Bokeh Figure -> Vertical Bars -> Hover -> Save HTML -> STOP
Program	<pre>import pandas as pd from bokeh.plotting import figure, output_file, save from bokeh.models import ColumnDataSource, HoverTool # Product sales dataset products = ["Laptop", "Smartphone", "Tablet", "Headphones", "Smartwatch", "Camera"] sales = [125, 210, 95, 170, 140, 80] df = pd.DataFrame({"Product": products, "Sales (Units)": sales}) print(df) highest_row = df.loc[df["Sales (Units)"].idxmax()] total_sales = df["Sales (Units)"].sum() average_sales = df["Sales (Units)"].mean() print("Total Sales:", total_sales, "units") print("Highest Sales:", highest_row["Product"], "-", highest_row["Sales (Units)"], "units") print("Average Sales:", round(average_sales, 2), "units") source = ColumnDataSource(df) p = figure(x_range=products, title="Interactive Bar Chart: Product Sales", x_axis_label="Product", y_axis_label="Sales (Units)", width=900, height=520, tools="pan,wheel_zoom,box_zoom,reset,save") p.vbar(x="Product", top="Sales (Units)", source=source, width=0.65)</pre>

Student Reg. No: 26MML0031 Name: Hasnain Makada

Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT), Vellore

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Exercise Program

Date: 27/08/2026

Week No.: 7

Program S. No. 15

```
hover = HoverTool(tooltips=[
    ("Product", "@Product"),
    ("Sales", "@{Sales (Units)} units")
])
p.add_tools(hover)
p.xgrid.grid_line_color = None

output_file("program15_product_sales_bar.html")
save(p)
print("Interactive bar chart saved successfully.")
```

**Output
Snapshots
With
Explanation/
Observations**

The following outputs were generated by executing Program 15. The static chart image presents the product-wise sales comparison, while the generated Bokeh HTML file preserves the interactive functionality.

Jupyter Output - Product Sales Summary

```
Product Sales Dataset:
Laptop 125 | Smartphone 210 | Tablet 95
Headphones 170 | Smartwatch 140 | Camera 80
Total Sales: 820 units
Highest Sales: Smartphone - 210 units
Average Sales: 136.67 units
```

Fig. 3.1: Product Sales Dataset and Statistical Summary

The product dataset contains six sales values. Total sales are 820 units, Smartphone has the highest sales at 210 units, and average sales are approximately 136.67 units per product.

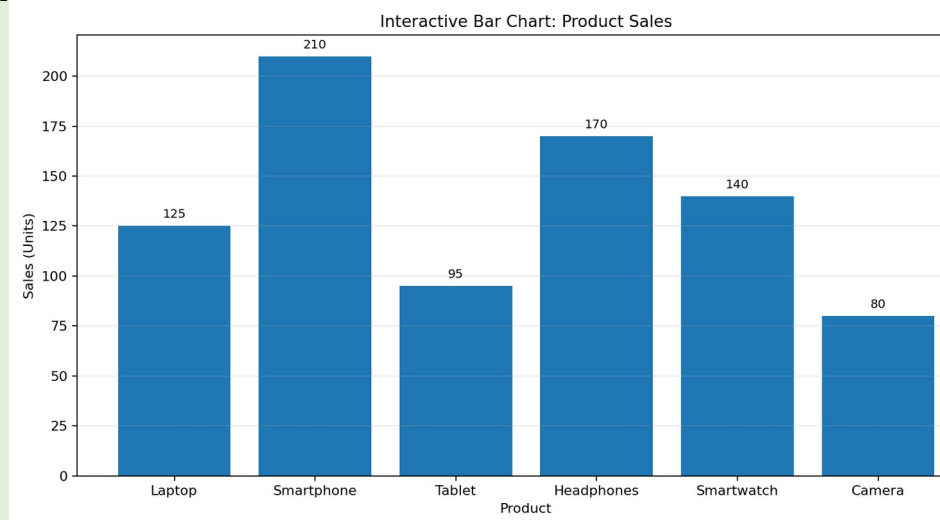


Fig. 3.2: Interactive Bar Chart Output

The bar chart compares sales across all six products. Smartphone is the highest-selling product, followed by Headphones, while Camera records the lowest sales.

Student Reg. No: 26MML0031 Name: Hasnain Makada

Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT), Vellore

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Exercise Program

Date: 27/08/2026

Week No.: 7

Program S. No. 15

	Jupyter Output - Program 15 Final Status Interactive bar chart created successfully using Bokeh. HTML file saved: program15_product_sales_bar.html Products visualized: 6 Hover, zoom, pan, reset and save tools are enabled. Fig. 3.3: Program 15 Execution and HTML Save Confirmation The final execution confirms successful generation and saving of the Bokeh product-sales bar chart. The HTML output supports hover and navigation tools for interactive inspection.
Conclusions	The exercise was completed successfully using Pandas and Bokeh. Product sales data were organized in a DataFrame and visualized through an interactive vertical bar chart. Smartphone recorded the highest sales with 210 units, total sales were 820 units, and average sales were approximately 136.67 units. The visualization provides a clear product-wise comparison and the interactive Bokeh tools improve data exploration.
References	1. Bokeh Documentation: Open Link 2. Pandas Documentation: Open Link 3. Python Program (GitHub link): 4. Class lecture link: Open Lecture Link

Student Reg. No: 26MML0031 Name: Hasnain Makada

Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT), Vellore

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Practice Program

Date: 03/09/2026

Week No.: 8

Program S. No. 16

Title	Visualizing Clusters with scikit-learn and Plotly			
Problem Statement	Perform Principal Component Analysis (PCA) on the Iris dataset and visualize the first two principal components using an interactive 2D scatter plot in Plotly.			
Objectives	• Load and verify the Iris dataset using Pandas and scikit-learn. • Standardize the four measurements before PCA. • Reduce the standardized data to two principal components. • Interpret explained variance and principal-axis coefficients. • Visualize species in an interactive Plotly scatter plot and save the results.			
Dataset URL Link / File	Built-in Iris dataset: sklearn.datasets.load_iris(as_frame=True). 150 samples; four numeric features measured in cm: sepal length, sepal width, petal length and petal width. Species: setosa (50), versicolor (50), virginica (50). Species labels are used for visualization/evaluation, not as input features. Dataset URL link:			
Functions Details	Sl. No.	Function	Package	Description of function
	1	load_iris()	sklearn	Loads 150 Iris samples and species labels.
	2	DataFrame()	Pandas	Creates tables of components and coefficients.
	3	StandardScaler()	sklearn	Defines mean-zero, unit-variance scaling.
	4	fit_transform()	sklearn	Fits a transformer and transforms the data.
	5	PCA()	sklearn	Defines principal component analysis.
	6	concat()	Pandas	Combines components and original features.
	7	scatter()	Plotly	Creates an interactive 2D scatter plot.
	8	bar()	Plotly	Creates the explained-variance bar chart.
	9	groupby()	Pandas	Groups principal-component scores by species.
	10	mean()/std()	Pandas	Summarizes species centroids and spread.
	11	write_html()/write_image()	Plotly	Exports interactive HTML and static charts.
	12	to_csv()	Pandas	Exports the PCA scores and measurements.

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Practice Program

Date: 03/09/2026

Week No.: 8

Program S. No. 16

Steps/ Process/ Procedure/ Algorithm/ Flowchart/ Model	1. Import Pandas, NumPy, Plotly and scikit-learn utilities. 2. Load Iris, rename the four feature columns and map target IDs to species. 3. Inspect sample rows, dataset shape, species counts and descriptive statistics. 4. Standardize the four numeric columns with StandardScaler; verify zero means and unit population standard deviations. 5. Apply PCA(n_components=2, random_state=42) to the standardized matrix. 6. Create a DataFrame of PC1 and PC2 scores, species and original measurements. 7. Inspect explained-variance percentages and principal-axis coefficients (pca.components_.T). 8. Plot PC1 versus PC2 using Plotly, with species colours, symbols and hover data. 9. Fit a four-component PCA to prepare the supporting variance bar chart. 10. Calculate species centroids and standard deviations in principal-component space. 11. Save the interactive scatter plot, static images and PCA CSV; print the analysis summary. Flow: START → Load Iris → Standardize → PCA → 2D scatter / variance chart → Save results → STOP
Program	Complete notebook code (formatting only adjusted for the page). Run in Jupyter with pandas, numpy, scikit-learn and plotly; static PNG export additionally requires a compatible Kaleido installation. <pre># Step 1: Import the required libraries import pandas as pd import numpy as np import plotly.express as px import plotly.io as pio from sklearn.datasets import load_iris from sklearn.preprocessing import StandardScaler from sklearn.decomposition import PCA import warnings warnings.filterwarnings("ignore") # Plotly renderer: interactive figure inside Jupyter + a static PNG # copy of every chart # (the PNG is what GitHub / PDF previews show, because they cannot run # interactive Plotly) try: import kaleido # noqa: F401 (only needed for the static PNG copies) pio.renderers.default = "plotly_mimetype+png" pio.renderers["png"].width = 900 pio.renderers["png"].height = 550 pio.renderers["png"].scale = 2 except ImportError: pio.renderers.default = "plotly_mimetype+notebook" # Step 2: Load the Iris dataset into a pandas DataFrame iris = load_iris(as_frame=True) df = iris.frame.rename(columns={ "sepal length (cm)": "Sepal Length (cm)", "sepal width (cm)": "Sepal Width (cm)", "petal length (cm)": "Petal Length (cm)", "petal width (cm)": "Petal Width (cm)", }) df["Species"] = df["target"].map(dict(enumerate(iris.target_names)))</pre>

Student Reg. No: 26MML0031 Name: Hasnain Makada

Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT), Vellore

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Practice Program

Date: 03/09/2026

Week No.: 8

Program S. No. 16

```
df = df.drop(columns="target")
feature_columns = [
    "Sepal Length (cm)",
    "Sepal Width (cm)",
    "Petal Length (cm)",
    "Petal Width (cm)",
]
df.head(10)
# Step 3: Basic verification of the dataset
print("Dataset Shape:", df.shape)
print("\nSpecies Counts:")
print(df["Species"].value_counts())
print("\nDescriptive Statistics:")
print(df[feature_columns].describe().round(2))
# Step 4: Standardize the four features (PCA is sensitive to feature
# scale)
scaler = StandardScaler()
X_scaled = scaler.fit_transform(df[feature_columns])
scaled_df = pd.DataFrame(X_scaled, columns=feature_columns)
print("Mean of each standardized feature (should be ~0):")
print(scaled_df.mean().round(3))
print(
    "\nStandard deviation of each standardized feature (should be ~1):"
)
print(scaled_df.std(ddof=0).round(3))
# Step 5: Apply PCA and keep the first two principal components
pca = PCA(n_components=2, random_state=42)
principal_components = pca.fit_transform(X_scaled)
pca_df = pd.DataFrame(
    principal_components, columns=["PC1", "PC2"]
)
pca_df["Species"] = df["Species"]
# keep the original measurements next to the components (used for
# hover information)
pca_df = pd.concat([pca_df, df[feature_columns]], axis=1)
pca_df.head(10)
# Step 6: Explained variance and component loadings
explained = pca.explained_variance_ratio_ * 100
print("Explained Variance Ratio:")
print("PC1 :", round(explained[0], 2), "%")
print("PC2 :", round(explained[1], 2), "%")
print(
    "Total variance captured by PC1 + PC2 :",
    round(explained.sum(), 2),
    "%",
)
loadings = pd.DataFrame(
    pca.components_.T,
    columns=["PC1", "PC2"],
    index=feature_columns,
)
print("\nComponent Loadings (contribution of each feature):")
print(loadings.round(3))
# Step 7: Interactive 2D scatter plot of PC1 vs PC2 coloured by
# species
fig = px.scatter(
    pca_df,
    x="PC1",
    y="PC2",
    color="Species",
    symbol="Species",
    hover_data={
        "PC1": ":.3f",
        "PC2": ":.3f",
        "Sepal Length (cm)": True,
        "Sepal Width (cm)": True,
        "Petal Length (cm)": True,
        "Petal Width (cm)": True,
    },
),
```

Student Reg. No: 26MML0031 Name: Hasnain Makada

Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT), Vellore

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Practice Program

Date: 03/09/2026

Week No.: 8

Program S. No. 16

```
title="PCA of the Iris Dataset: First Two Principal Components",
labels={
    "PC1": f"PC1 ({explained[0]:.2f}% variance)",
    "PC2": f"PC2 ({explained[1]:.2f}% variance)",
},
)
fig.update_traces(
    marker=dict(
        size=10, opacity=0.85, line=dict(width=1, color="white")
    )
)
fig.update_layout(
    legend_title_text="Species",
    width=900,
    height=550,
    template="plotly_white",
)
fig.show()
# Step 8: Scree plot - variance explained by every component
# (supporting chart)
pca_full = PCA(n_components=4, random_state=42).fit(X_scaled)
scree_df = pd.DataFrame(
    {
        "Principal Component": ["PC1", "PC2", "PC3", "PC4"],
        "Explained Variance (%)": (
            pca_full.explained_variance_ratio_ * 100
        ).round(2),
    }
)
scree_df["Cumulative Variance (%)"] = (
    scree_df["Explained Variance (%)"].cumsum().round(2)
)
print(scree_df)
fig_scree = px.bar(
    scree_df,
    x="Principal Component",
    y="Explained Variance (%)",
    text="Explained Variance (%)",
    title="Scree Plot: Variance Explained by Each Principal Component",
)
fig_scree.update_traces(textposition="outside")
fig_scree.update_layout(
    width=900, height=550, template="plotly_white"
)
fig_scree.show()
# Step 9: Analyse the clusters in the principal-component space
centroids = (
    pca_df.groupby("Species")[["PC1", "PC2"]].mean().round(3)
)
print("Centroid of each species in PC space:")
print(centroids)
spread = pca_df.groupby("Species")[["PC1", "PC2"]].std().round(3)
print(
    "\nSpread (standard deviation) of each species in PC space:"
)
print(spread)
# Step 10: Save the interactive chart, a static image and the PCA
# dataset
fig.write_html("program16_pca_scatter.html")
print(
    "Interactive PCA scatter plot saved as program16_pca_scatter.html"
)
try:
    fig.write_image(
        "program16_pca_scatter.png",
        width=900,
        height=550,
        scale=2,
    )
)
```

Student Reg. No: 26MML0031 Name: Hasnain Makada

Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT), Vellore

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Practice Program

Date: 03/09/2026

Week No.: 8

Program S. No. 16

```
fig_scree.write_image(
    "program16_scree_plot.png", width=900, height=550, scale=2
)
print(
    "Static images saved as program16_pca_scatter.png and program16_scree_plot.png"
)
except Exception as error:
    print(
        "Static image not saved (kaleido not available):", error
    )
pca_df.to_csv("program16_iris_pca.csv", index=False)
print("PCA dataset saved as program16_iris_pca.csv")
# Step 11: Summary of the analysis
print("PCA ANALYSIS SUMMARY")
print("-" * 45)
print("Total samples           :", len(pca_df))
print("Original features         :", len(feature_columns))
print("Components retained       : 2")
print("Variance explained by PC1  :", round(explained[0], 2), "%")
print("Variance explained by PC2  :", round(explained[1], 2), "%")
print(
    "Total variance retained   :", round(explained.sum(), 2), "%"
)
print(
    "Most influential feature  :",
    loadings["PC1"].abs().idxmax(),
    "(on PC1)",
)
print("Well separated species    : setosa")
print("Partially overlapping     : versicolor and virginica")
```

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Practice Program

Date: 03/09/2026

Week No.: 8

Program S. No. 16

**Output
Snapshots
With
Explanation/
Observations**

```
Dataset Shape: (150, 5)

Species Counts:
Species
setosa      50
versicolor 50
virginica   50
Name: count, dtype: int64

Descriptive Statistics:
      Sepal Length (cm)  Sepal Width (cm)  Petal Length (cm)  Petal Width (cm)
count              150.00              150.00              150.00              150.00
mean                5.84                3.06                3.76                1.20
std                 0.83                0.44                1.77                0.76
min                 4.30                2.00                1.00                0.10
25%                 5.10                2.80                1.60                0.30
50%                 5.80                3.00                4.35                1.30
75%                 6.40                3.30                5.10                1.80
max                 7.90                4.40                6.90                2.50

Mean of each standardized feature (should be ~0):
Sepal Length (cm)  -0.0
Sepal Width (cm)   -0.0
Petal Length (cm)  -0.0
Petal Width (cm)   -0.0
dtype: float64

Standard deviation of each standardized feature (should be ~1):
Sepal Length (cm)   1.0
Sepal Width (cm)    1.0
Petal Length (cm)   1.0
Petal Width (cm)    1.0
dtype: float64
```

Fig. 1.1: Dataset verification and feature standardization

The dataset contains 150 records and five DataFrame columns: four measurements and one species label. Each species has 50 observations. The transformed feature matrix has zero means (up to floating-point rounding) and population standard deviations of one.

```
Explained Variance Ratio:
PC1 : 72.96 %
PC2 : 22.85 %
Total variance captured by PC1 + PC2 : 95.81 %

Component Loadings (contribution of each feature):
              PC1    PC2
Sepal Length (cm)  0.521  0.377
Sepal Width (cm)   -0.269  0.923
Petal Length (cm)  0.580  0.024
Petal Width (cm)   0.565  0.067
```

Fig. 1.2: Explained variance and principal-axis coefficients

PC1 and PC2 retain 95.81% of the variance. Petal length has the largest absolute PC1 coefficient (0.580). The notebook calls these values loadings; here they are the principal-axis coefficients from `pca.components_.T`, not correlation loadings.

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Practice Program

Date: 03/09/2026

Week No.: 8

Program S. No. 16

PCA of the Iris Dataset: First Two Principal Components

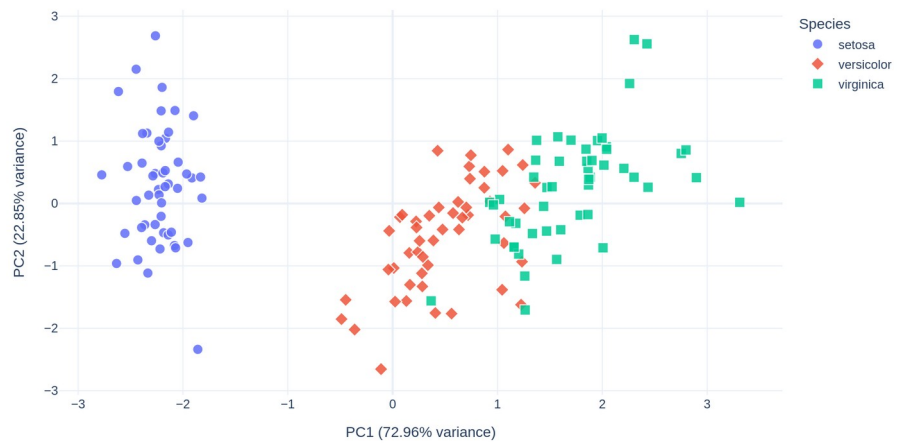


Fig. 1.3: Plotly scatter plot of the first two principal components

Setosa occupies the negative PC1 region and is separated from the other species. Versicolor and virginica overlap partly. Colours and symbols represent known species, not clusters generated by PCA.

Scree Plot: Variance Explained by Each Principal Component

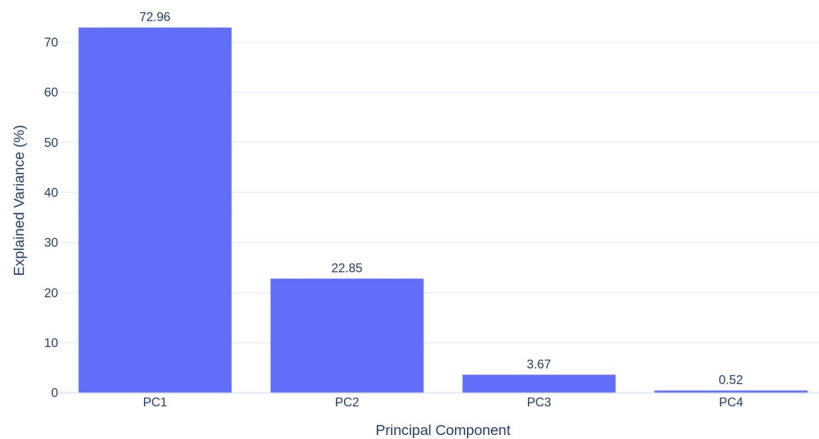


Fig. 1.4: Variance explained by all four principal components

The four variance percentages are 72.96%, 22.85%, 3.67% and 0.52%. Retaining the first two components loses approximately 4.19% of the standardized variance.

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Practice Program

Date: 03/09/2026

Week No.: 8

Program S. No. 16

	Centroid of each species in PC space: <table><thead><tr><th></th><th>PC1</th><th>PC2</th></tr></thead><tbody><tr><td>Species</td><td></td><td></td></tr><tr><td>setosa</td><td>-2.225</td><td>0.289</td></tr><tr><td>versicolor</td><td>0.496</td><td>-0.550</td></tr><tr><td>virginica</td><td>1.728</td><td>0.261</td></tr></tbody></table> Spread (standard deviation) of each species in PC space: <table><thead><tr><th></th><th>PC1</th><th>PC2</th></tr></thead><tbody><tr><td>Species</td><td></td><td></td></tr><tr><td>setosa</td><td>0.206</td><td>0.934</td></tr><tr><td>versicolor</td><td>0.464</td><td>0.833</td></tr><tr><td>virginica</td><td>0.571</td><td>0.876</td></tr></tbody></table> Fig. 1.5: Species centroids and spread in principal-component space Mean PC1 scores increase from setosa (−2.225) through versicolor (0.496) to virginica (1.728). Species labels are used only after projection for these summaries. The reported spreads are sample standard deviations. Interactive PCA scatter plot saved as program16_pca_scatter.html Static images saved as program16_pca_scatter.png and program16_scree_plot.png PCA dataset saved as program16_iris_pca.csv PCA ANALYSIS SUMMARY ----- <table><tbody><tr><td>Total samples</td><td>: 150</td></tr><tr><td>Original features</td><td>: 4</td></tr><tr><td>Components retained</td><td>: 2</td></tr><tr><td>Variance explained by PC1</td><td>: 72.96 %</td></tr><tr><td>Variance explained by PC2</td><td>: 22.85 %</td></tr><tr><td>Total variance retained</td><td>: 95.81 %</td></tr><tr><td>Most influential feature</td><td>: Petal Length (cm) (on PC1)</td></tr><tr><td>Well separated species</td><td>: setosa</td></tr><tr><td>Partially overlapping</td><td>: versicolor and virginica</td></tr></tbody></table> Fig. 1.6: File export confirmation and PCA summary Execution produced the interactive scatter HTML, two static charts and a CSV containing PC1, PC2, species and the four original measurements. The summary confirms 150 observations and 95.81% retained variance.		PC1	PC2	Species			setosa	-2.225	0.289	versicolor	0.496	-0.550	virginica	1.728	0.261		PC1	PC2	Species			setosa	0.206	0.934	versicolor	0.464	0.833	virginica	0.571	0.876	Total samples	: 150	Original features	: 4	Components retained	: 2	Variance explained by PC1	: 72.96 %	Variance explained by PC2	: 22.85 %	Total variance retained	: 95.81 %	Most influential feature	: Petal Length (cm) (on PC1)	Well separated species	: setosa	Partially overlapping	: versicolor and virginica
	PC1	PC2																																															
Species																																																	
setosa	-2.225	0.289																																															
versicolor	0.496	-0.550																																															
virginica	1.728	0.261																																															
	PC1	PC2																																															
Species																																																	
setosa	0.206	0.934																																															
versicolor	0.464	0.833																																															
virginica	0.571	0.876																																															
Total samples	: 150																																																
Original features	: 4																																																
Components retained	: 2																																																
Variance explained by PC1	: 72.96 %																																																
Variance explained by PC2	: 22.85 %																																																
Total variance retained	: 95.81 %																																																
Most influential feature	: Petal Length (cm) (on PC1)																																																
Well separated species	: setosa																																																
Partially overlapping	: versicolor and virginica																																																
Conclusions	The Iris dataset was successfully standardized and reduced from four measurements to two principal components. PC1 explains 72.96% and PC2 explains 22.85% of the standardized variance, retaining 95.81% in total. Setosa is clearly separated in the projection, while versicolor and virginica partly overlap. PCA is an unsupervised dimensionality-reduction method, not a clustering algorithm; species colours are reference labels. Plotly provides an interactive view, and the saved HTML and CSV support further exploration.																																																

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Practice Program

Date: 03/09/2026

Week No.: 8

Program S. No. 16

References

1. scikit-learn PCA documentation:
2. scikit-learn Iris dataset / StandardScaler documentation:
3. Plotly Python documentation:
4. Pandas documentation:
5. Python notebook (GitHub link): [Open Program 16 on GitHub](#)
6. Class lecture link: [Open Week 8 lecture](#)

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Exercise Program

Date: 03/09/2026

Week No.: 8

Program S. No. 17

Title	Visualizing Clusters with scikit-learn and Plotly			
Problem Statement	Perform K-means clustering on the Iris dataset and visualize the clusters using an interactive 3D scatter plot in Plotly.			
Objectives	• Prepare and standardize the four Iris measurements. • Compare inertia and silhouette scores for candidate cluster counts. • Fit K-means with three clusters and inspect their composition. • Evaluate clusters using silhouette score and adjusted Rand index. • Visualize clusters and their centres in an interactive Plotly 3D scatter plot.			
Dataset URL Link / File	Built-in Iris dataset: sklearn.datasets.load_iris(as_frame=True). 150 samples; four numeric features measured in cm: sepal length, sepal width, petal length and petal width. Species: setosa (50), versicolor (50), virginica (50). Species labels are used for visualization/evaluation, not as input features. Dataset URL link:			
Functions Details	Sl. No.	Function	Package	Description of function
	1	load_iris()	sklearn	Loads the Iris dataset and labels.
	2	StandardScaler()	sklearn	Standardizes all four numeric features.
	3	KMeans()	sklearn	Defines the K-means clustering model.
	4	fit_predict()	sklearn	Fits the model and returns cluster labels.
	5	silhouette_score()	sklearn	Measures cluster cohesion and separation.
	6	adjusted_rand_score()	sklearn	Compares clusters with known species labels.
	7	crosstab()	Pandas	Tabulates cluster and species membership.
	8	inverse_transform()	sklearn	Converts centres back to original cm units.
	9	line()	Plotly	Plots inertia for the candidate k values.
	10	scatter_3d()/Scatter3d()	Plotly	Plots observations and cluster centres in 3D.
	11	write_html()/write_image()	Plotly	Saves interactive HTML and static charts.
	12	to_csv()	Pandas	Saves original features and cluster labels.

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Exercise Program

Date: 03/09/2026

Week No.: 8

Program S. No. 17

Steps/ Process/ Procedure/ Algorithm/ Flowchart/ Model	1. Import Pandas, NumPy, Plotly and the scikit-learn clustering/evaluation utilities. 2. Load Iris, rename the feature columns and map the known species labels. 3. Standardize only the four measurements using StandardScaler. 4. Fit K-means for $k = 1$ to 10; collect inertia and silhouette scores (silhouette is undefined for $k = 1$). 5. Plot inertia versus k. Use $k = 3$ to compare with the three known species; do not claim it maximizes the silhouette score. 6. Fit KMeans($n_clusters=3$, $init="k-means++"$, $n_init=10$, $random_state=42$) and assign cluster labels. 7. Compute cluster sizes, a cluster/species cross-tabulation, silhouette score and adjusted Rand index. 8. Convert the cluster centres from standardized units back to cm. 9. Plot sepal length, sepal width and petal length in 3D; use colour for cluster, symbol for species and hover data for petal width. 10. Overlay cluster centres, save the interactive HTML, static charts and clustered CSV, and print the summary. Flow: START → Load Iris → Standardize → Compare k → Fit K-means → Evaluate → 3D scatter → Save results → STOP
Program	Complete notebook code (formatting only adjusted for the page). Run in Jupyter with pandas, numpy, scikit-learn and plotly; static PNG export additionally requires a compatible Kaleido installation. <pre># Step 1: Import the required libraries import pandas as pd import numpy as np import plotly.express as px import plotly.graph_objects as go import plotly.io as pio from sklearn.datasets import load_iris from sklearn.preprocessing import StandardScaler from sklearn.cluster import KMeans from sklearn.metrics import silhouette_score, adjusted_rand_score import warnings warnings.filterwarnings("ignore") # Plotly renderer: interactive figure inside Jupyter + a static PNG # copy of every chart # (the PNG is what GitHub / PDF previews show, because they cannot run # interactive Plotly) try: import kaleido # noqa: F401 (only needed for the static PNG copies) pio.renderers.default = "plotly_mimetype+png" pio.renderers["png"].width = 900 pio.renderers["png"].height = 550 pio.renderers["png"].scale = 2 except ImportError: pio.renderers.default = "plotly_mimetype+notebook" # Step 2: Load the Iris dataset into a pandas DataFrame iris = load_iris(as_frame=True) df = iris.frame.rename(columns={ "sepal length (cm)": "Sepal Length (cm)", "sepal width (cm)": "Sepal Width (cm)", "petal length (cm)": "Petal Length (cm)", "petal width (cm)": "Petal Width (cm)", }) df["Species"] = df["target"].map(</pre>

Student Reg. No: 26MML0031 Name: Hasnain Makada

Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT), Vellore

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Exercise Program

Date: 03/09/2026

Week No.: 8

Program S. No. 17

```
dict(enumerate(iris.target_names))
)
df = df.drop(columns="target")
feature_columns = [
    "Sepal Length (cm)",
    "Sepal Width (cm)",
    "Petal Length (cm)",
    "Petal Width (cm)",
]
print("Dataset Shape:", df.shape)
print("\nSpecies Counts:")
print(df["Species"].value_counts())
df.head(10)
# Step 3: Standardize the features so that every measurement has equal
# weight in the distance calculation
scaler = StandardScaler()
X_scaled = scaler.fit_transform(df[feature_columns])
print("Scaled feature matrix shape:", X_scaled.shape)
print("First 5 scaled rows:")
print(np.round(X_scaled[:5], 3))
# Step 4: Elbow method and silhouette score to choose the number of
# clusters (k)
k_values = range(1, 11)
inertia = []
silhouette = []
for k in k_values:
    model = KMeans(
        n_clusters=k, init="k-means++", n_init=10, random_state=42
    )
    labels = model.fit_predict(X_scaled)
    inertia.append(model.inertia_)
    silhouette.append(
        silhouette_score(X_scaled, labels) if k > 1 else np.nan
    )
elbow_df = pd.DataFrame(
    {
        "k": list(k_values),
        "Inertia (WCSS)": np.round(inertia, 3),
        "Silhouette Score": np.round(silhouette, 3),
    }
)
print(elbow_df.to_string(index=False))
fig_elbow = px.line(
    elbow_df,
    x="k",
    y="Inertia (WCSS)",
    markers=True,
    title="Elbow Method: Inertia vs Number of Clusters (k)",
)
fig_elbow.update_layout(
    width=900,
    height=550,
    template="plotly_white",
    xaxis=dict(dtick=1),
)
fig_elbow.show()
# Step 5: Fit K-means with k = 3 (three Iris species) and assign
# cluster labels
kmeans = KMeans(
    n_clusters=3, init="k-means++", n_init=10, random_state=42
)
df["Cluster"] = kmeans.fit_predict(X_scaled)
df["Cluster"] = "Cluster " + df["Cluster"].astype(str)
print("Number of iterations to converge:", kmeans.n_iter_)
print("Final inertia (WCSS):", round(kmeans.inertia_, 3))
print("\nCluster Sizes:")
print(df["Cluster"].value_counts().sort_index())
df.head(10)
# Step 6: Compare the discovered clusters with the actual species
```

Student Reg. No: 26MML0031 Name: Hasnain Makada

Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT), Vellore

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Exercise Program

Date: 03/09/2026

Week No.: 8

Program S. No. 17

```
comparison = pd.crosstab(df["Cluster"], df["Species"])
print("Cluster vs Species Cross-tabulation:")
print(comparison)
sil = silhouette_score(X_scaled, kmeans.labels_)
ari = adjusted_rand_score(iris.target, kmeans.labels_)
print("\nSilhouette Score (k = 3) : ", round(sil, 3))
print("Adjusted Rand Index vs species:", round(ari, 3))
# percentage of points whose cluster matches the dominant species of
# that cluster
correctly_grouped = comparison.max(axis=1).sum()
print(
    "Points grouped with their dominant species:",
    correctly_grouped,
    "of",
    len(df),
    "(",
    round(correctly_grouped / len(df) * 100, 2),
    "% )",
)
# Step 7: Cluster centres converted back to the original units (cm)
centers_cm = scaler.inverse_transform(kmeans.cluster_centers_)
centers_df = pd.DataFrame(
    centers_cm, columns=feature_columns
).round(3)
centers_df.insert(
    0, "Cluster", ["Cluster 0", "Cluster 1", "Cluster 2"]
)
print("Cluster Centres (original units):")
print(centers_df.to_string(index=False))
# Step 8: Interactive 3D scatter plot of the clusters (with cluster
# centres marked)
fig = px.scatter_3d(
    df.sort_values("Cluster"),
    x="Sepal Length (cm)",
    y="Sepal Width (cm)",
    z="Petal Length (cm)",
    color="Cluster",
    symbol="Species",
    hover_data={"Petal Width (cm)": True},
    title="K-means Clustering of the Iris Dataset (k = 3): Interactive 3D View",
    opacity=0.8,
)
fig.update_traces(marker=dict(size=4))
# add the three cluster centres as large black diamonds
fig.add_trace(
    go.Scatter3d(
        x=centers_df["Sepal Length (cm)"],
        y=centers_df["Sepal Width (cm)"],
        z=centers_df["Petal Length (cm)"],
        mode="markers",
        marker=dict(size=8, color="black", symbol="diamond"),
        hovertext=centers_df["Cluster"] + " centre",
        name="Cluster Centres",
    )
)
fig.update_layout(
    width=900,
    height=650,
    legend_title_text="Cluster / Species",
    margin=dict(l=0, r=0, b=0, t=60),
    scene=dict(
        xaxis_title="Sepal Length (cm)",
        yaxis_title="Sepal Width (cm)",
        zaxis_title="Petal Length (cm)",
        aspectmode="cube",
        camera=dict(eye=dict(x=1.7, y=1.5, z=0.9)),
    ),
    template="plotly_white",
)
fig.show()
```

Student Reg. No: 26MML0031 Name: Hasnain Makada

Faculty: Dr Rajasekhara Babu M, Professor, School of CSE, Vellore Institute of Technology (VIT), Vellore

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Exercise Program

Date: 03/09/2026

Week No.: 8

Program S. No. 17

```
# Step 9: Save the interactive chart, static images and the clustered
# dataset
fig.write_html("program17_kmeans_3d_scatter.html")
print(
    "Interactive 3D scatter plot saved as program17_kmeans_3d_scatter.html"
)
try:
    fig.write_image(
        "program17_kmeans_3d_scatter.png",
        width=900,
        height=650,
        scale=2,
    )
    fig_elbow.write_image(
        "program17_elbow_plot.png", width=900, height=550, scale=2
    )
    print(
        "Static images saved as program17_kmeans_3d_scatter.png and program17_elbow_plot.png"
    )
except Exception as error:
    print(
        "Static image not saved (kaleido not available):", error
    )
df.to_csv("program17_iris_kmeans_clusters.csv", index=False)
print(
    "Clustered dataset saved as program17_iris_kmeans_clusters.csv"
)
# Step 10: Summary of the analysis
print("K-MEANS CLUSTERING SUMMARY")
print("-" * 45)
print("Total samples           :", len(df))
print(
    "Features used           :",
    len(feature_columns),
    "(standardized)",
)
print("Number of clusters (k)    : 3")
print(
    "Cluster sizes           :",
    df["Cluster"].value_counts().sort_index().tolist(),
)
print("Silhouette score         :", round(sil, 3))
print("Adjusted Rand Index      :", round(ari, 3))
print(
    "Agreement with species   :",
    round(correctly_grouped / len(df) * 100, 2),
    "%",
)
print("Perfectly separated species : setosa")
print("Overlapping species       : versicolor and virginica")
```

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Exercise Program

Date: 03/09/2026

Week No.: 8

Program S. No. 17

**Output
Snapshots
With
Explanation/
Observations**

Dataset Shape: (150, 5)

Species Counts:

Species

setosa 50

versicolor 50

virginica 50

Name: count, dtype: int64

Scaled feature matrix shape: (150, 4)

First 5 scaled rows:

```
[[ -0.901  1.019 -1.34  -1.315]
```

```
 [ -1.143 -0.132 -1.34  -1.315]
```

```
 [ -1.385  0.328 -1.397 -1.315]
```

```
 [ -1.507  0.098 -1.283 -1.315]
```

```
 [ -1.022  1.249 -1.34  -1.315]]
```

Fig. 2.1: Dataset and standardized feature matrix

All 150 observations and four numeric features are used for clustering. Species labels are kept separately for plotting and evaluation; they are not included in the distance calculations.

k	Inertia (WCSS)	Silhouette Score
1	600.000	NaN
2	222.362	0.582
3	139.820	0.460
4	114.093	0.387
5	90.928	0.346
6	81.544	0.317
7	72.631	0.320
8	62.541	0.339
9	55.119	0.342
10	47.391	0.352

Fig. 2.2: Inertia and silhouette scores for k = 1 to 10

Inertia decreases as more clusters are added. The highest silhouette score among the tested values is at k = 2 (0.582), not k = 3 (0.460). The notebook uses k = 3 for comparison with the three known Iris species. A silhouette score is undefined for a single cluster.

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Exercise Program

Date: 03/09/2026

Week No.: 8

Program S. No. 17

Elbow Method: Inertia vs Number of Clusters (k)

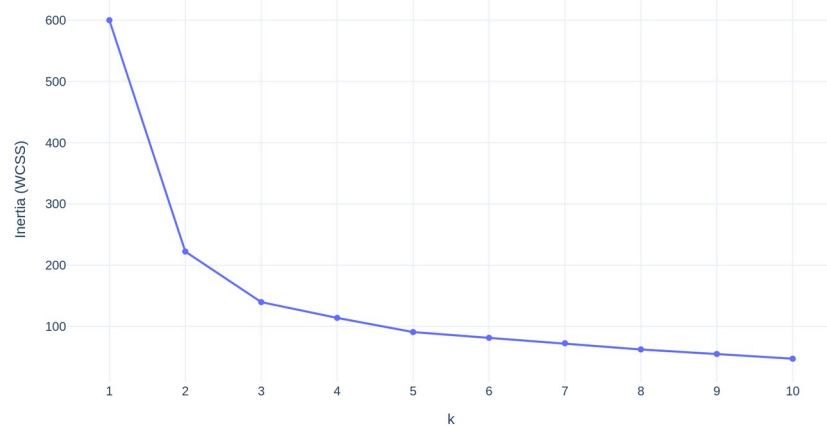


Fig. 2.3: Plotly elbow chart

The large early reductions in inertia are followed by progressively smaller improvements. This chart supports exploring a small number of clusters but does not prove that three is uniquely optimal.

Number of iterations to converge: 4
Final inertia (WCSS): 139.82

Cluster Sizes:

Cluster

Cluster 0 53

Cluster 1 50

Cluster 2 47

Name: count, dtype: int64

Cluster vs Species Cross-tabulation:

Species	setosa	versicolor	virginica
Cluster 0	0	39	14
Cluster 1	50	0	0
Cluster 2	0	11	36

Silhouette Score (k = 3) : 0.46

Adjusted Rand Index vs species: 0.62

Points grouped with their dominant species: 125 of 150 (83.33 %)

Fig. 2.4: Cluster sizes, species cross-tabulation and evaluation

Cluster 1 contains all 50 setosa samples. Cluster 0 contains 39 versicolor and 14 virginica; Cluster 2 contains 11 versicolor and 36 virginica. Cluster IDs are arbitrary. The dominant-species counts are 39, 50 and 36 for Clusters 0, 1 and 2, respectively: 125/150 (83.33%) in total.

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Exercise Program

Date: 03/09/2026

Week No.: 8

Program S. No. 17

Cluster Centres (original units):

Cluster	Sepal Length (cm)	Sepal Width (cm)	Petal Length (cm)	Petal Width (cm)
Cluster 0	5.802	2.674	4.370	1.413
Cluster 1	5.006	3.428	1.462	0.246
Cluster 2	6.781	3.096	5.511	1.972

Fig. 2.5: Cluster centres in centimetres

Inverse transformation makes each centre interpretable in original measurement units. All four dimensions contribute to model fitting, even though only three dimensions can be displayed in the scatter plot.

K-means Clustering of the Iris Dataset (k = 3): Interactive 3D View

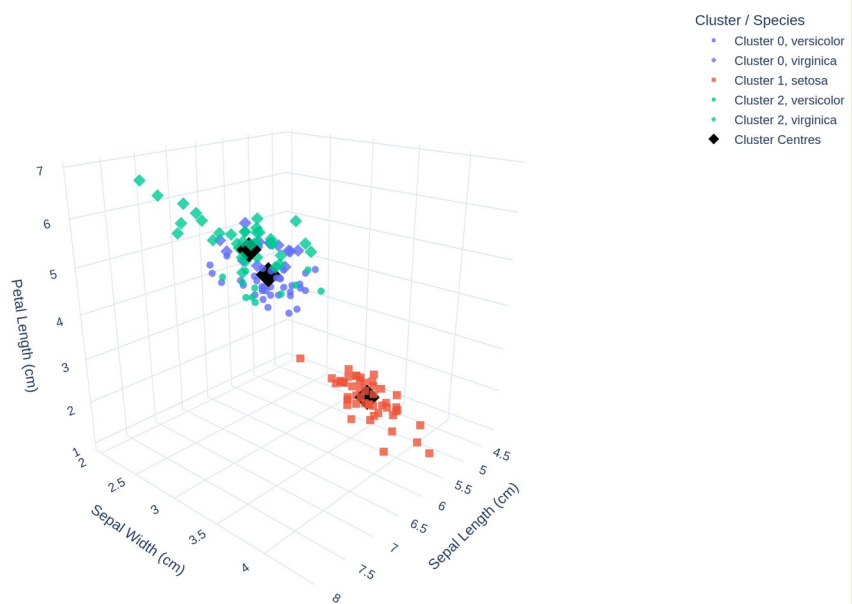


Fig. 2.6: Interactive 3D cluster plot with centres

The static view shows sepal length, sepal width and petal length. Colour encodes cluster and symbol encodes species. Black diamonds mark cluster centres. Petal width is available on hover in the interactive HTML.

2026 FALL SEMESTER
Programming for Data Science (MACSE502) Lab Record
Exercise Program

Date: 03/09/2026

Week No.: 8

Program S. No. 17

	Interactive 3D scatter plot saved as program17_kmeans_3d_scatter.html Static images saved as program17_kmeans_3d_scatter.png and program17_elbow_plot.png Clustered dataset saved as program17_iris_kmeans_clusters.csv K-MEANS CLUSTERING SUMMARY <pre> ----- Total samples : 150 Features used : 4 (standardized) Number of clusters (k) : 3 Cluster sizes : [53, 50, 47] Silhouette score : 0.46 Adjusted Rand Index : 0.62 Agreement with species : 83.33 % Perfectly separated species : setosa Overlapping species : versicolor and virginica </pre> Fig. 2.7: File export confirmation and K-means summary Execution converged in four iterations with inertia 139.820. It saved the interactive 3D scatter, two static charts and the clustered dataset. The output confirms cluster sizes 53, 50 and 47, silhouette 0.460 and adjusted Rand index 0.620.
Conclusions	K-means successfully grouped 150 standardized Iris observations into three clusters of sizes 53, 50 and 47. The final inertia is 139.820, the silhouette score is 0.460 and the adjusted Rand index is 0.620. All 50 setosa samples form one cluster; versicolor and virginica overlap. Dominant-species agreement is 125/150 (83.33%), a descriptive cluster-purity measure rather than supervised test accuracy. The 3D Plotly chart displays three original measurements, while clustering uses all four standardized features. The result demonstrates useful but imperfect agreement between unsupervised groups and biological species.
References	1. scikit-learn K-means documentation: 2. scikit-learn Iris dataset / StandardScaler documentation: 3. Plotly Python documentation: 4. Pandas documentation: 5. Python notebook (GitHub link): Open Program 17 on GitHub 6. Class lecture link: Open Week 8 lecture